

구현하며 배우는 Transformer

Transformer를 위한 사전 지식 - NLP 기반과 RNN 이해

딥러닝 기반의 NLP 주요 요소

- 딥러닝 이전에는 텍스트를 의미와 문맥에 기반하여 표현하고 학습하기 어려움. 때문에 질의 응답, 문서 요약, 번역등을(또는 인간과 유사한 자연스러운 상호작용이 필요한 작업) 위한 언어 모델 구현이 어려움.
- 딥러닝 등장 후 단어의 의미와 문장의 문맥을 이해하고 표현하는 NLP 모델 구현이 가능

단어의 의미와 문장의 문맥 이해에 기반한 NLP

Tokenization

문장을 작은 토큰 단위(단어등)
로 분할하여 단어 사전 구축

Embedding

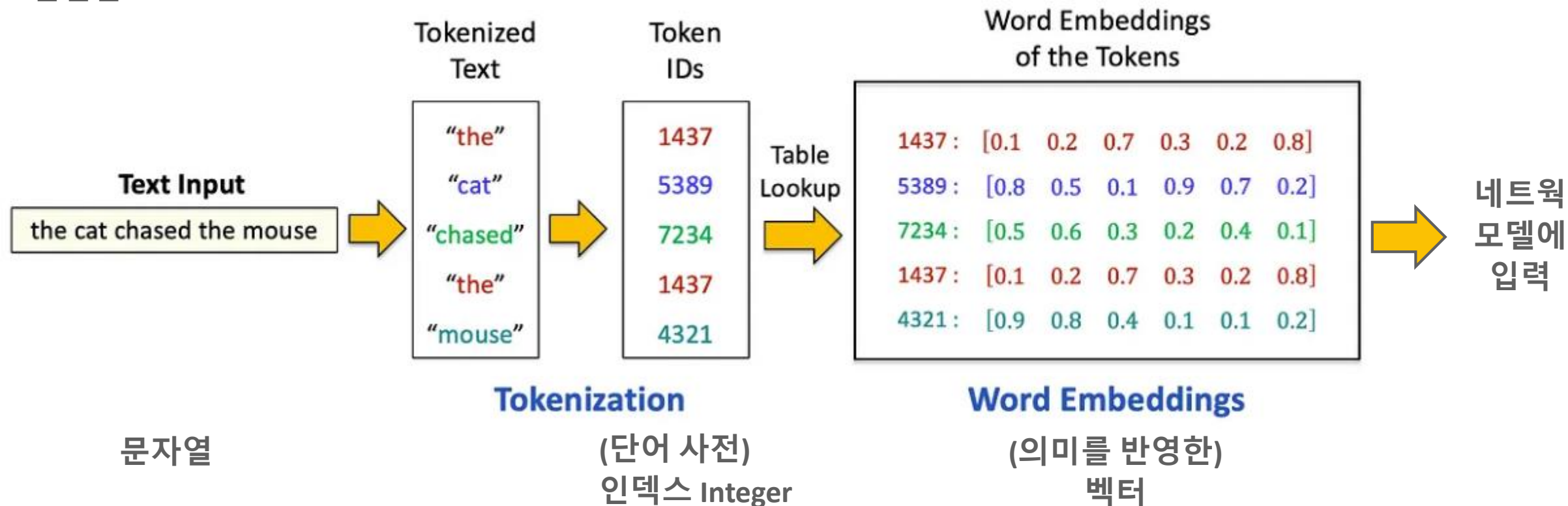
토큰(단어등)의 의미를 다차원
의 Vector값으로 표현

딥러닝 모델

다차원의 Embedding 데이터를 입력
받아 문장내 문맥을 이해하고 학습
할 수 있는 딥러닝 모델의 등장(RNN,
Transformer)

Tokenization(토큰화)와 Embedding 개요

- Tokenization(토큰화)는 텍스트를 **토큰** 단위로 분할 하고, 개별 토큰을 **index 값으로 매핑하는 어휘 사전** 구축 작업
- Embedding은 Tokenizer를 통해 만들어진 토큰값을 단어 의미를 가지는 다차원의 embedding vector로 생성
- Tokenizer의 **token index**를 기반으로 Embedding은 embedding vector를 반환하므로 Tokenizer와 Embedding은 Loosely 연결됨.



텍스트 토큰화(Tokenization) 이해

텍스트 토큰화(Tokenization)는 컴퓨터가 텍스트를 보다 쉽게 처리하고 이해할 수 있게 만들어 주는 NLP의 첫번째 단계로서 텍스트(문장)를 **토큰 단위로** 분해하고 개별 토큰을 **index 값으로 매핑하는 어휘 사전** 구축 작업을 의미

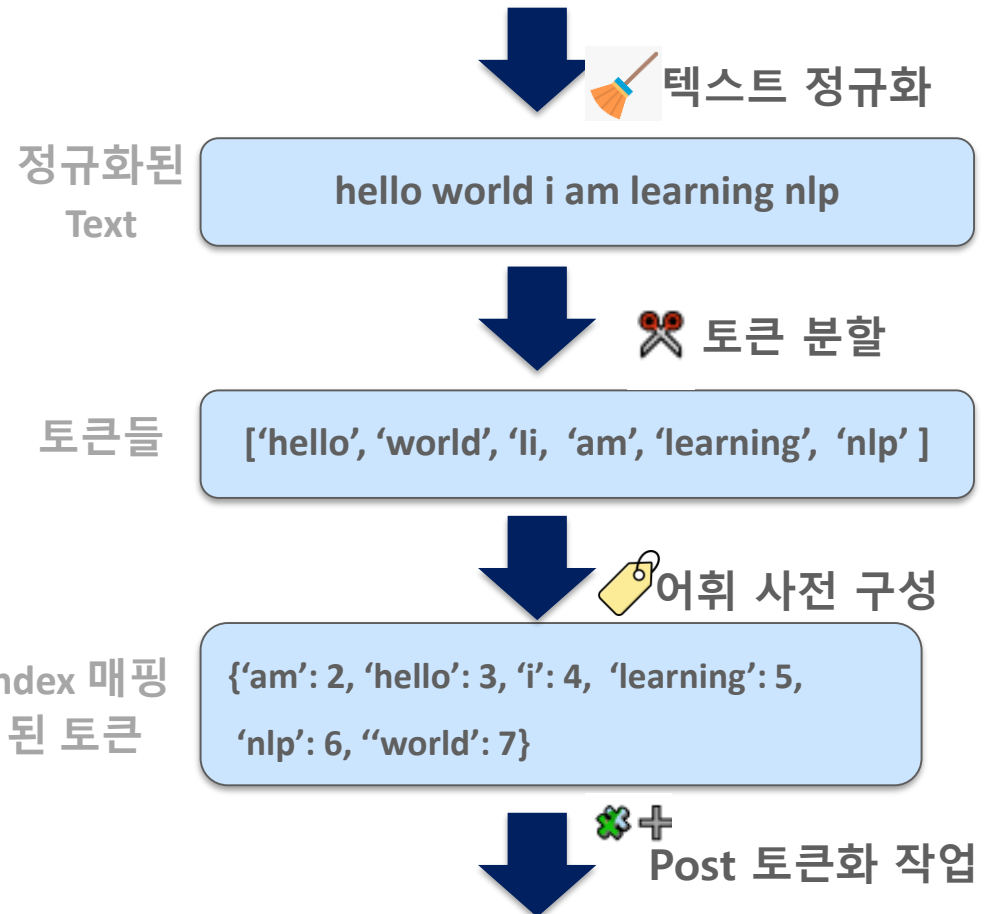
원본 Text

Hello, world! I am learning NLP.

토큰(Token)은 아래와 같은 유형으로 나뉠 수 있음.

- 단어: "I love NLP" -> ["I", "love", "NLP"]
- 서브워드: BPE나 WordPiece 등 "unhappiness" -> ["un", "happiness"]
- 그밖에 문자단위, 문장단위 토큰화도 있음

문장을 토큰 단위로 쪼개어서 모델이 개별 토큰 단위에 보다 집중하고 패턴을 배우도록 도와줌.

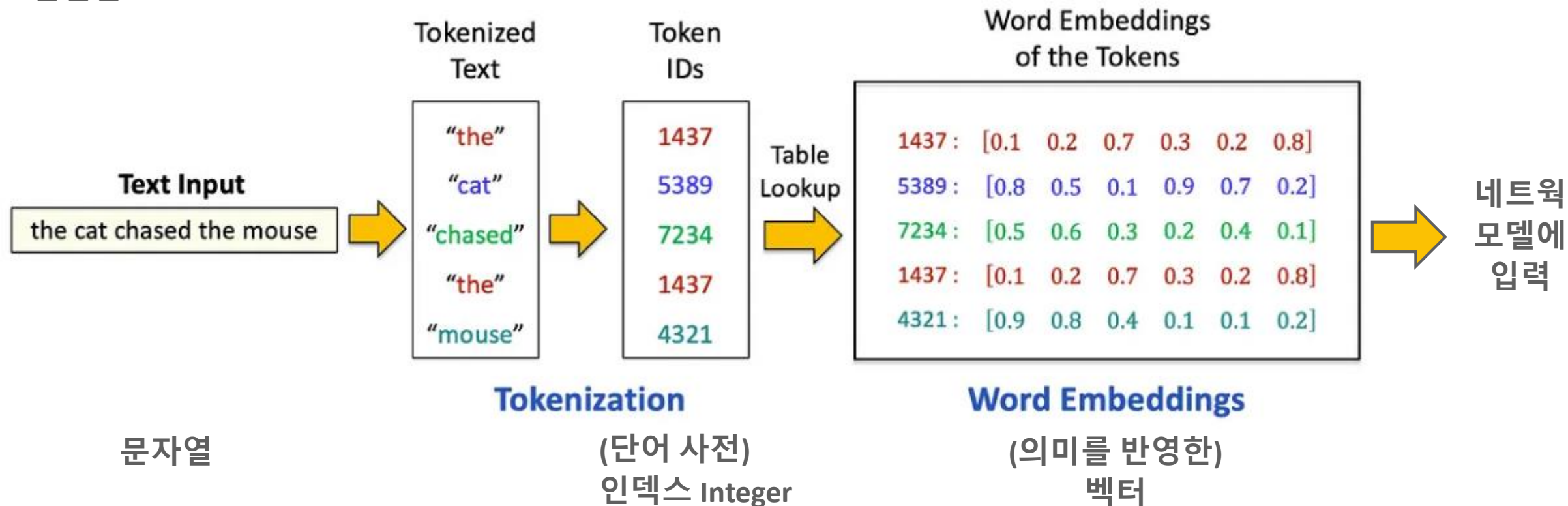


토큰화(Tokenization)의 주요 프로세스



Tokenization(토큰화)와 Embedding 개요

- Tokenization(토큰화)는 텍스트를 **토큰** 단위로 분할 하고, 개별 토큰을 **index 값으로 매핑하는 어휘 사전** 구축 작업
- Embedding은 Tokenizer를 통해 만들어진 토큰값을 단어 의미를 가지는 다차원의 embedding vector로 생성
- Tokenizer의 **token index**를 기반으로 Embedding은 embedding vector를 반환하므로 Tokenizer와 Embedding은 Loosely 연결됨.



의미와 문맥을 반영한 단어(또는 문장)의 수치화 어려움

- 인간은 의미와 문맥에 기반하여 단어와 문장을 이해. 하지만 기계는 이러한 방식으로 단어와 문장을 표현하기 어려움.

"The cat chased the mouse across the kitchen floor."



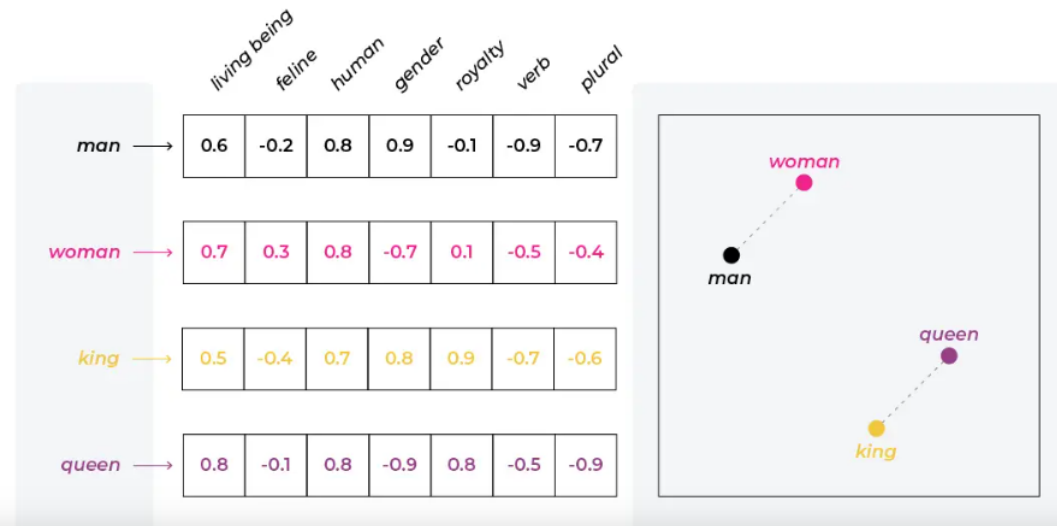
개별 단어(예: cat) 의미를 어떻게 수치화 할 것인가?
(의미가 수치화 될 수 있는가?)

단어와 단어 사이의 다양한 관계, 문장의 문맥을 효과적으로 이해 할 수 있도록 어떻게 개별 단어를 표현 할 것인가?

딥러닝을 활용하여 단어의 의미를 다차원 벡터로 변환

어떻게 단어를 언어 모델이 잘 이해할 수 있도록 표현할 것인가?

- 딥러닝 이전까지는 단어가 가지는 문맥상 의미를 표현할 적절한 방법을 찾지 못함
- 과거에는 문장들/문서들에서 단어가 나타나는 횟수등을 그 단어가 가지는 중요한 특성으로 간주하여 encoding/embedding 변환 (Bag of words, TF-IDF)
- 딥러닝 연구가 활발해 지면서 단어를 문맥적인 의미를 가지는 다차원 벡터에 매핑하는 Word Embedding 방식이 보편화 되고
- 이러한 Embedding 방식은 문장내 문맥을 이해하고 학습할 수 있는 딥러닝 모델의 (RNN, Transformer) NLP에서 주도적으로 활용될 수 있는 시발점 역할을 수행

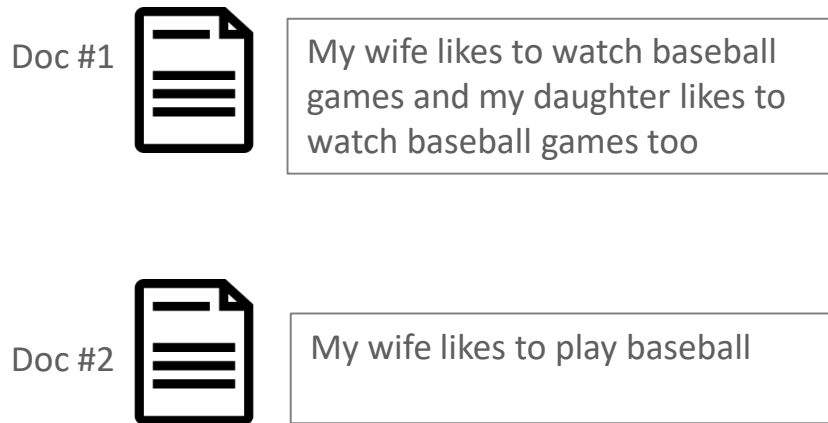


딥러닝 이전 - Bag of Words와 Frequency Based Encoding

Bag of Words 모델은 문서가 가지는 모든 단어(Words)를 문맥이나 순서를 무시하고 일괄적으로 단어에 대해 빈도 값을 부여해 피쳐 값을 추출하는 형태의 변환. 대표적으로 Count와 TF-IDF가 있음

해당 방식은 단어의 문맥적인 특성이 고려 되지 않으므로 번역, 문장 요약복잡한 언어 모델을 구현할 수 없음.

여러 Document(문서) 들과 Text 들



← 단어 feature 들 →

	단어 #1	단어 #2	단어 #3		단어 #n
Doc #1	3	4	3	0	1
.....	..	..	..	..	..
Doc #m	0	5	0	4	3

↑
각 문서에서 해당 단어의 횟수나 정규화 변환된 횟수

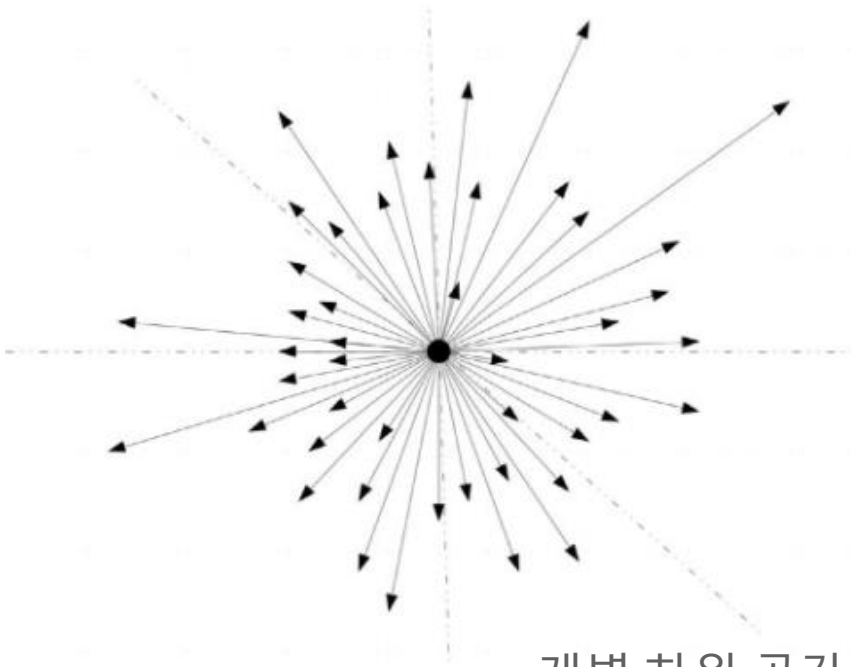
Count, TF-IDF

Document Term Matrix: 개별 문서(또는 문장)을 단어들의 횟수나 정규화 변환된 횟수로 표현

Word Embedding 개요

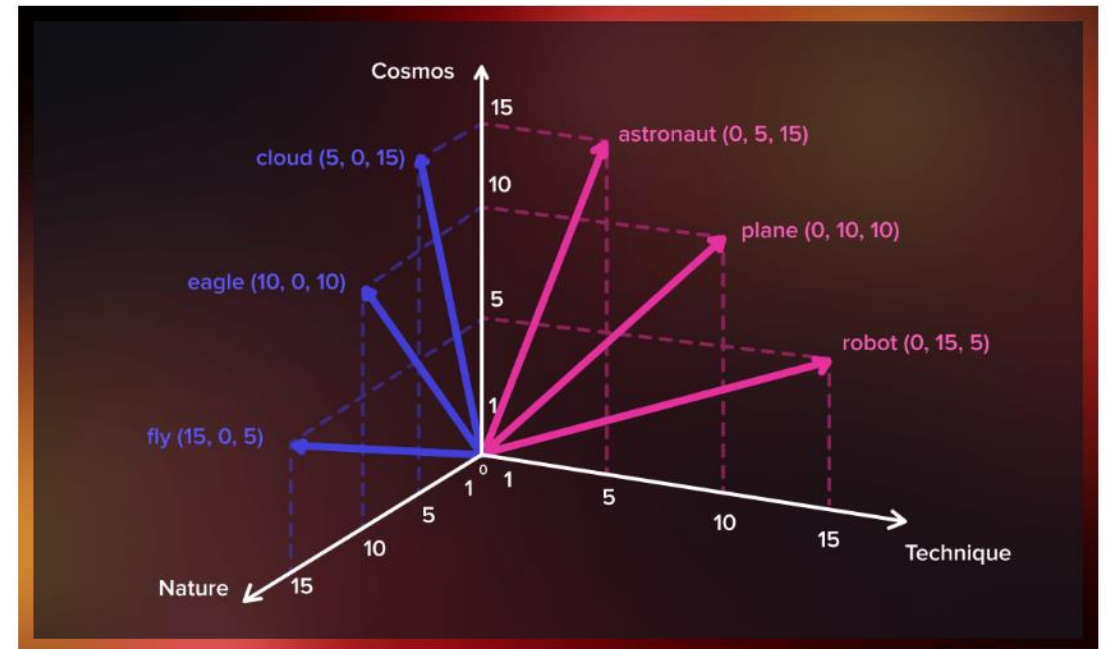
- Word Embedding은 단어를 다차원의 벡터로 매핑하는 것
- 개별 차원들이 **문맥적인 함축 의미**를 가지는 **다차원 공간 영역**에 단어(Token)를 벡터 값으로 매핑하여 단어의 의미를 표현할 수 있게 변환(**비슷한** 의미를 가지는 단어는 **가깝게** 모여 있음)

문맥축을 가지는 다차원 공간 영역
(수백 차원 이상)



개별 차원 공간은 함축적 Latent Space

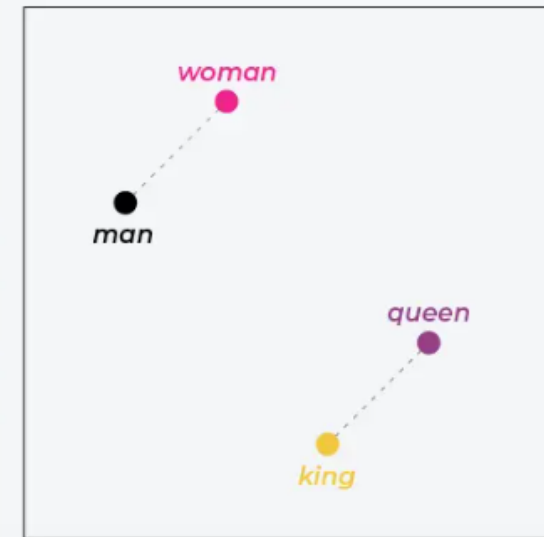
개별 단어를 다차원 공간영역에 벡터 변환
(아래는 예시를 위해 3차원 공간으로 표현)



Word Embedding의 주요 특징

- 단어의 의미 표현. 비슷한 단어는 비슷한 위치에(Semantic similarity)
- 단어들간의 관계 표현 가능
- 문장속 단어들간의 문맥을 모델이 쉽게 이해
- 텍스트의 이해뿐 아니라 텍스트의 생성에도 유용하게 사용(번역, 질의응답등)
- Dense Vector 형태로 연산 복잡도 및 메모리 사용 효율화 개선

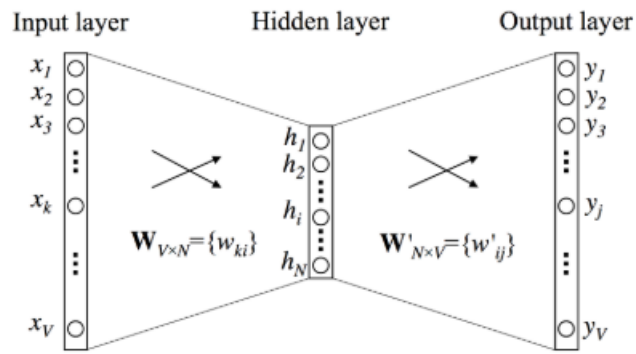
	living	being	feline	human	gender	royalty	verb	plural
man →	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7	
woman →	0.7	0.3	0.8	-0.7	0.1	-0.5	-0.4	
king →	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6	
queen →	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9	



$$\text{king} - \text{man} + \text{woman} = \text{queen}$$
$$\text{Paris} - \text{France} + \text{Germany} = \text{Berlin}$$

딥러닝 기반 임베딩(Embedding)의 유형

언어 모델(RNN, Transformer 등)과 **별도로** 미리 학습된 **정적**(Pretrained Static) 임베딩



- 대규모 말뭉치 데이터에서 딥러닝으로 미리 학습되어 제공되는 임베딩
- 단어의 문맥을 다차원 벡터로 표현 가능하게 한 **선구자적** 딥러닝 모델
- 다른 말뭉치 데이터 세트로 언어 모델을 학습 시킬 때 embedding값 재 학습의 어려움
- 현재 딥러닝 기반 주요 언어 모델(Transformer 등)에서는 더 이상 사용되지 않음.

언어 모델(RNN, Transformer 등)과 **함께 학습되는** 임베딩

0	$x_{0,0}$	$x_{0,1}$	$x_{0,2}$	...	$x_{0,768}$	...	$x_{0,n}$
1	$x_{1,0}$	$x_{1,1}$	$x_{1,2}$	...	$x_{1,768}$	...	$x_{1,n}$
2	$x_{2,0}$	$x_{2,1}$	$x_{2,2}$	...	$x_{2,768}$	...	$x_{2,n}$
...	...	...	...	...	...	...	...
768	$x_{768,0}$	$x_{768,1}$	$x_{768,2}$	...	$x_{768,768}$	...	$x_{768,n}$
...	...	...	...	...	...	...	...
n	$x_{n,0}$	$x_{n,1}$	$x_{n,2}$	...	$x_{n,768}$	...	$x_{n,n}$

- 언어 모델(RNN, Transformer 등) 학습과 함께 학습되는 임베딩
- Nn.Embedding은 **2차원의 Weight Matrix** 형태로 되어 있으며 초기에는 random 다차원 벡터값을 가지고 있지만, 모델이 학습되면서 **단어의 문맥**을 표현하는 다차원 벡터값으로 학습됨
- 오늘날 주요 딥러닝 기반 언어 모델에서 사용하는 임베딩 방법

nn.Embedding의 이해

- nn.Embedding은 **Learnable Lookup Table**.
- Tokenizer 단어 사전의 token 개수와 embeddin의 차원 수 Shape 형태의(vocab_size, embed_dim) learnable weight matrix를 가짐(이 weight matrix 자체가 개별 토큰별 embedding vector값을 가지고 있음)
- nn.Embedding 객체는 문장내 토큰들의 **index 값을 인자로 받으면, 해당 위치에 있는** embedding vector 값을 반환해줌.

```
emb_layer = nn.Embedding(  
    num_embeddings, # 단어 사전 수  
    embedding_dim, # embed vector 크기  
    padding_idx # <PAD> token idx 값  
)
```

```
emb_vector = emb_layer(torch.tensor(768))
```

Word king

Embedding
Layer

Token [768]

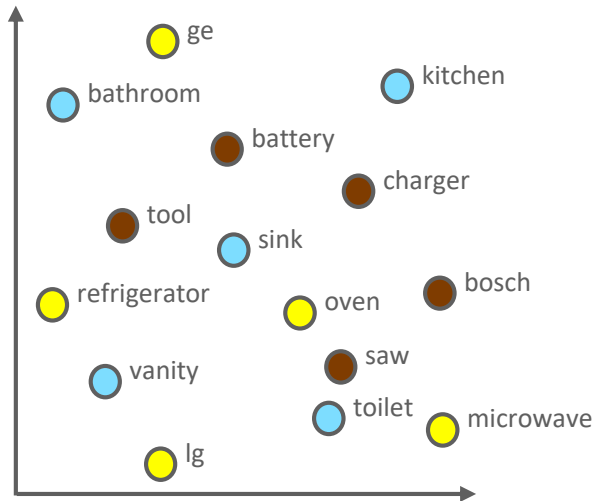
0	$X_{0,0}$	$X_{0,1}$	$X_{0,2}$	...	$X_{0,768}$	...	$X_{0,n}$
1	$X_{1,0}$	$X_{1,1}$	$X_{1,2}$	...	$X_{1,768}$	...	$X_{1,n}$
2	$X_{2,0}$	$X_{2,1}$	$X_{2,2}$	...	$X_{2,768}$	...	$X_{2,n}$
...	...	...	...	...	...	...	...
768	$X_{768,0}$	$X_{768,1}$	$X_{768,2}$	...	$X_{768,768}$	...	$X_{768,n}$
...	...	...	...	...	...	...	...
n	$X_{n,0}$	$X_{n,1}$	$X_{n,2}$	...	$X_{n,768}$	...	$X_{n,n}$

토큰 index값을 생성된 생성된 nn.Embedding 객체의 인자로 입력하면 해당 token의 embedding vector 값 반환.

딥러닝 학습을 통해 Update되는 Embedding Vector

nn.Embedding내의 Learnable Parameter인 Weight값이 바로 Embedding Vector값이며, 딥러닝 모델이 학습을 하면서 (역전파를 통해) 지속적으로 값을 갱신하게 됨

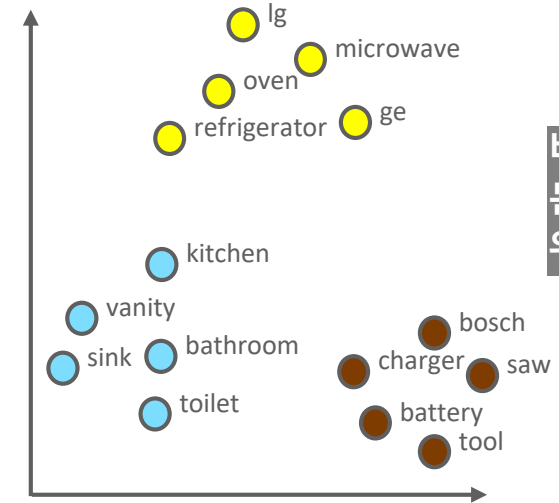
최초 Embedding 값(random)



지속적으로 값 갱신



학습이 완료된 후의 Embedding 값



tokenize

Embedding

신경망

모델 학습

딥러닝 모델이 학습을 하면서 Embedding 역시 계속 값을 갱신

왜 Pretrained NLP 모델은 학습 시 사용된 Tokenizer를 이용해야 하는가?

Pretrained 모델의 학습 파라미터를 100% 활용하기 위해서는 학습 시 사용된 Tokenizer를 이용해야 함

```
from transformers import BertTokenizer, BertForSequenceClassification  
  
model_name = "bert-base-uncased"  
  
bert_tokenizer = BertTokenizer.from_pretrained(pretrained_model_name_or_path=model_name)  
  
bert_model = BertForSequenceClassification.from_pretrained(pretrained_model_name_or_path=model_name)
```

- 특정 Pretrained 모델의 학습(train)시 사용된 특정 Tokenizer는 학습 시 사용된 텍스트들의 **모든 고유한** Token 문자값과 index값 매핑을 **어휘 사전**에 가지고 있으며 Pretrained 모델은 이와 결합되어 **학습된 Embedding**을 가짐.
- 만약 다른 Tokenizer를 이용하여 만들어진 Token index값을 기반으로 모델에 예측(Inference)를 할 경우에는 모델 학습 시 적용된 Token index값 **매핑이 달라져서** 올바른 결과가 나오지 않음
- 또한 Pretrained 모델을 다른 데이터 세트로 재 학습 시에 다른 Tokenizer를 사용하면, Token index값 매핑이 달라지게 되면서 대용량 데이터 세트로 **이미 Pretrained된 Embedding**을 처음부터 다시 학습 시켜야 함.
- 토큰 분할 방식의 차이 및 Pretrained 모델의 성능 재현등의 문제가 발생 할 수 있음

"Welcome to Transformer Course."

BertTokenizer(bert-base-uncased)

Token들

['welcome', 'to', 'transform', '##er', 'course', '.,']

Token index 값

[6160, 2000, 10938, 2121, 2607, 1012]

딥러닝 언어 모델이 해결해야 할 주요 이슈

- 문장에서 달라지는 단어의 뜻, 단어와 단어간 관계, 문장이 가지는 다양한 의미와 문맥을 어떻게 학습해서 이해할 것인가?

"He sat near the **bank** and watched the ducks swim."

"She opened a savings **bank** account yesterday."

같은 bank, 다른 의미

"The cat **chased** the mouse across the kitchen floor."

단어와 단어의 의미적 연결(Cat은 mouse를 쫓는 행동으로 연결)

"I thought the **film** would be boring, but it completely blew me away."

전체 문장에서의 문맥과 의도
(결국 the film은 boring하지 않은, 훌륭한 film)

- 딥러닝 모델의 입력 데이터는 동일한 크기로 입력되어야(적어도 배치 레벨에서는) 하지만 문장들은 서로 다른 길이(토큰의 개수)를 가지고 있음. 어떻게 효율적으로(물리적 메모리 제약등) 이들 데이터를 받아들이고 학습시킬 수 있을 것인가?

Sequence Model 개요

- Sequence Model은 **순서(순차)**적으로 생성되는 Sequence Data를 기반으로 한 모델
- Sequence Data는 순차적 생성이 매우 중요한 요소임. 이전에 생성된 데이터는 현재 생성 데이터에 **밀접한 영향**을 미치며, 과거 데이터는 **업데이트**를 수행하지 않음. Sequence Model은 이러한 **Sequence Data의 특성**에 기반하며 모델링 수행
- 주요 Sequence Data는 텍스트, 스피치등의 언어 데이터, 금융, IoT 센서 데이터와 같은 Time Series 데이터

텍스트

Manchester United produced a stunning comeback from 2-0 down to beat Newcastle United - and potentially save manager José Mourinho from the sack. Second half strikes from Juan Mata and Anthony Martial levelled proceedings, before an injury-time winner from Alexis Sánchez sent the Old Trafford crowd into raptures. After the international break, the Red Devils travel to Stamford Bridge where they face a stern test against unbeaten Chelsea.

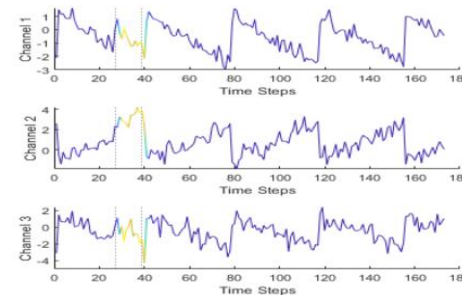
스피치



금융

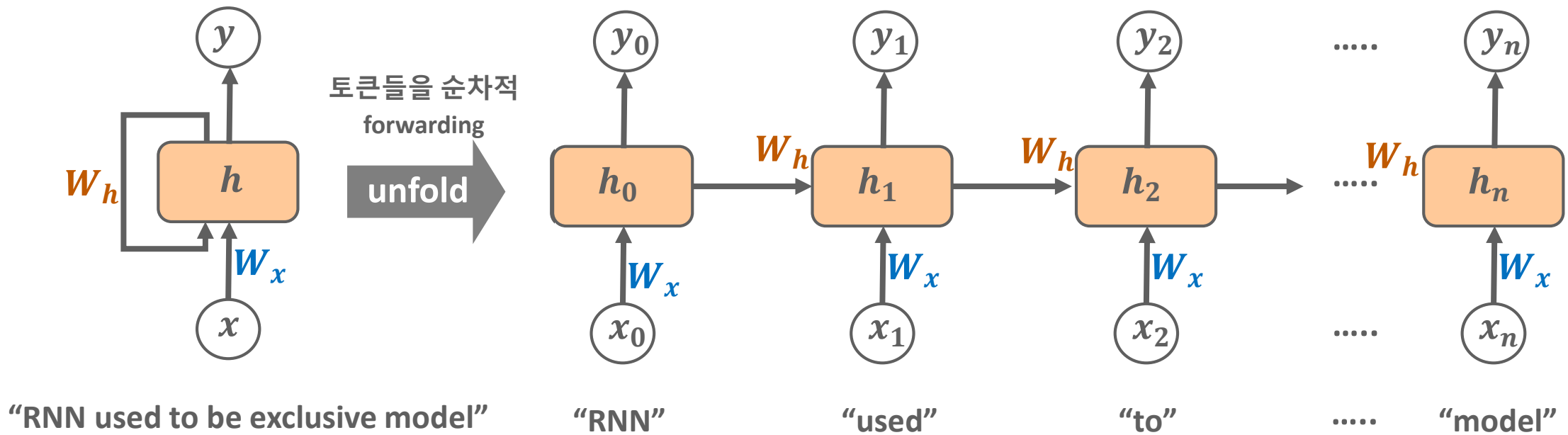


센서



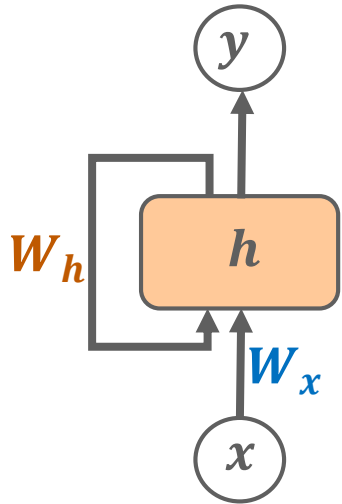
RNN(Recurrent Neural Network) 모델 개요

- RNN은 Sequence 데이터를 시점 순으로 모델에 **순차적으로** 입력하면서 학습하는 모델로서 **동적으로** 변하는 입력 문장의 토큰 개수(문장 길이)에 관계 없이 모델 구성 및 학습 가능
- Sequence데이터의 **현 시점 입력** 데이터와 **이전 시점에서 학습된 Hidden State**를 입력 받아 시점별 Hidden State와 출력을 순차적으로 생성. 각 시점에서 사용되는 **Learnable Parameter** 는 **전 시점에 걸쳐 공유됨**.



RNN 모델의 Forward Pass 방식 및 학습 파라미터

- Sequence 데이터를 순차적으로 Forward Pass 하면서 이전에 생성된 Hidden States 와 Hidden State용 학습 파라미터, 현재 입력 token과 입력 token용 학습 파라미터를 적용하여 현 시점의 Hidden State를 생성. 학습 파라미터는 전 시점에 걸쳐 공유됨.



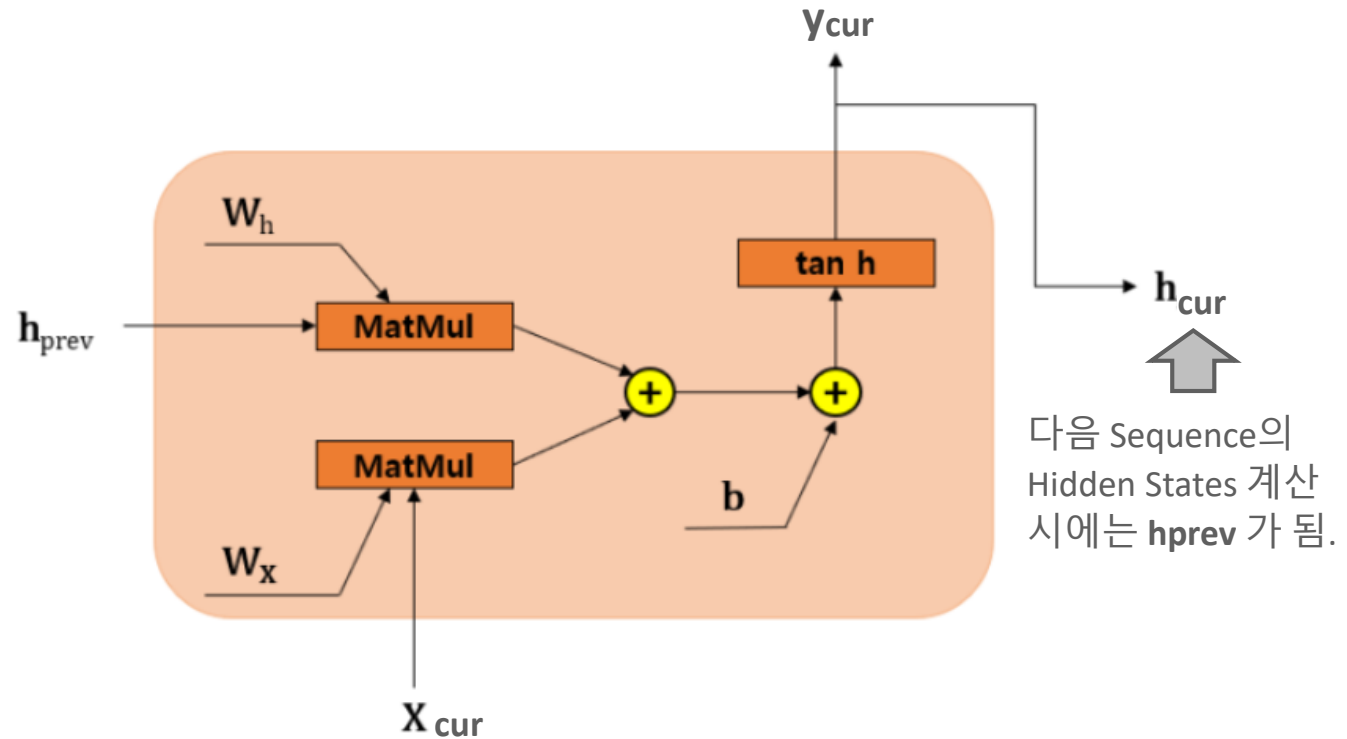
순차적으로 입력을 받아서
순차적으로 Hidden States와
출력 생성

“RNN used to be exclusive model”

W_h: (이전)Hidden State 적용을 위한 Weight Matrix

W_x: (현) input Token 적용을 위한 Weight Matrix

X: Sequence 입력 데이터(b, seq_len, emb_size)



다음 Sequence의
Hidden States 계산
시에는 **h_{prev}** 가 됨.

$$h_{cur} = \tanh(X_{cur} * W_x + b_x + h_{prev} * W_h + b_h)$$

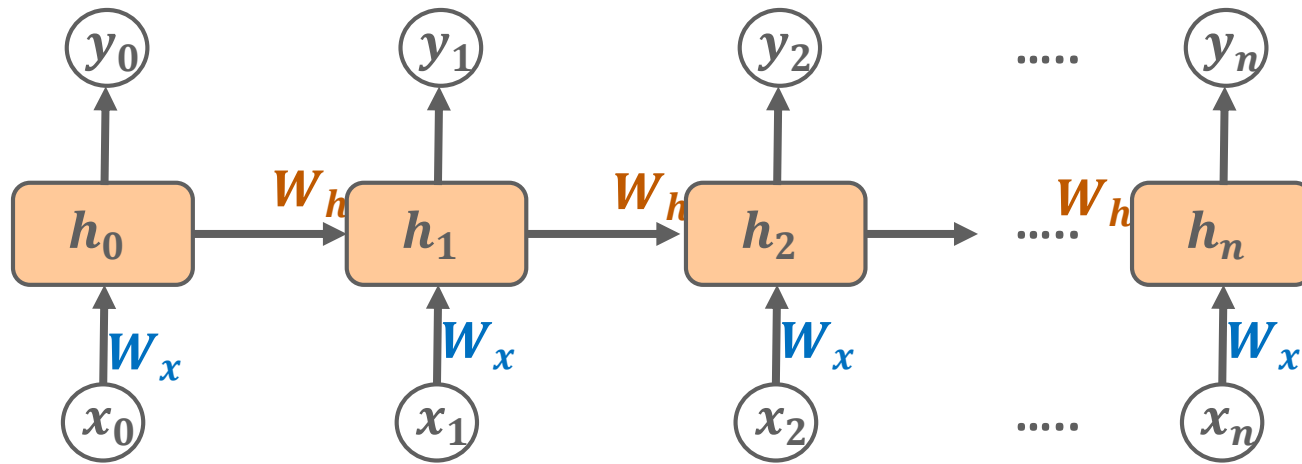
RNN 모델의 학습

Feed Forward

입력 Embedding과 W_h , W_x 에 기반하여 순차적으로 Hidden State 계산

역순으로 Gradient 값을 계산하면서 W_h , W_x , Embedding 값을 Update

오차 역전파



- W_h , W_x , Embedding 토큰값은 모델이 학습하면서 계속 업데이트 됨.
- 학습이 완료되더라도 **동일한 토큰은 여전히 동일한 embedding 값**을 가짐
- 하지만 주변 단어들과 함께 문맥을 반영할 수 있는 W_h , W_x 가 학습되면서 **해당 문맥에 맞는 각기 다른 Hidden State**를 생성

RNN의 문제점

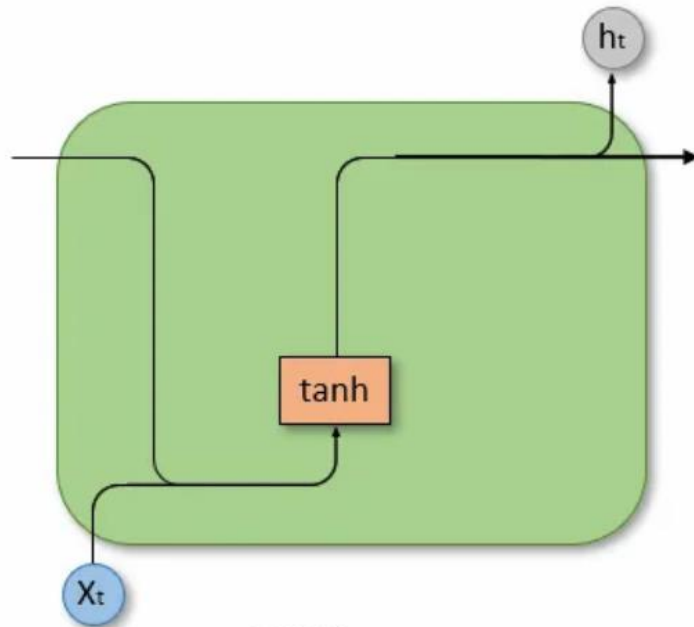
문장은 개별 단어가 **순차적**으로 이어져서 만들어지지만, 문장의 **문맥상 의미**는 반드시 단어 순으로 해석되는 것이 아님

“The **dog** **that** barked loudly and chased the squirrels through the yard and over the fence into the woods **was** very tired”

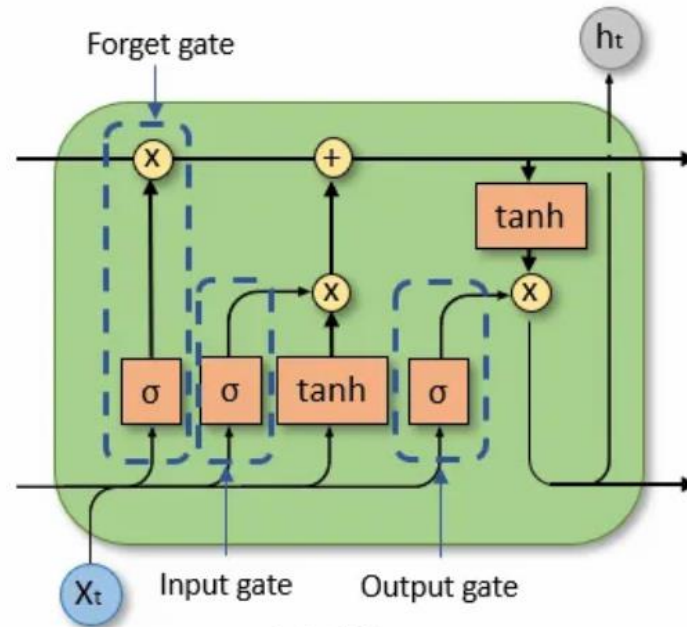
단어 순서에 큰 영향을 받는 Hidden Status	RNN은 단어 생성 순서대로 Hidden State를 생성(bi-directional 제외). 문장내 단어의 순서에 맞게 문맥을 생성하도록 학습되기 때문에 단어 순서에 영향을 덜 받으면서 문맥을 대표할 수 있는 Hidden State 생성이 어려움
장기 기억 문제	입력 시퀀스의 길이가 길어 질 수록 이전 정보가 계속 희석됨 . 즉 오래전 Sequence(문장의 첫 단어)는 현재 Sequence(문장의 마지막 단어)의 hidden state 에 영향을 미치기 어려움(단기 기억에 집중)
Gradient 소실 및 폭발	긴 문장의 경우 최초 토큰의 Gradient를 구하기 위해서는 마지막 토큰에서 부터 역순으로 상당히 많은 연쇄 미분 (Chain Derivatives) 를 순차적으로 해줘야 함. 이 때문에 오래전 Sequence는 Gradient값이 아주 작아지거나 또는 중간에 폭발적으로 커지는 현상이 발생해서 가중치 업데이트가 불안해 질 수 있음.
느린 학습 속도	RNN은 입력 데이터를 단어(토큰) 단위로 순차적 으로 입력해 줘야 함. 문장이 길 수록 순방향 전파, 오차 역전파가 모두 느려지게 되면서 학습 시간이 매우 느려짐(GPU의 병렬 처리 기능을 효과적으로 활용 어려움)

장기 기억 문제등의 개선을 위해서 LSTM, GRU등의 RNN 기반 모델이 있지만, 근본적인 문제 해결은 아님

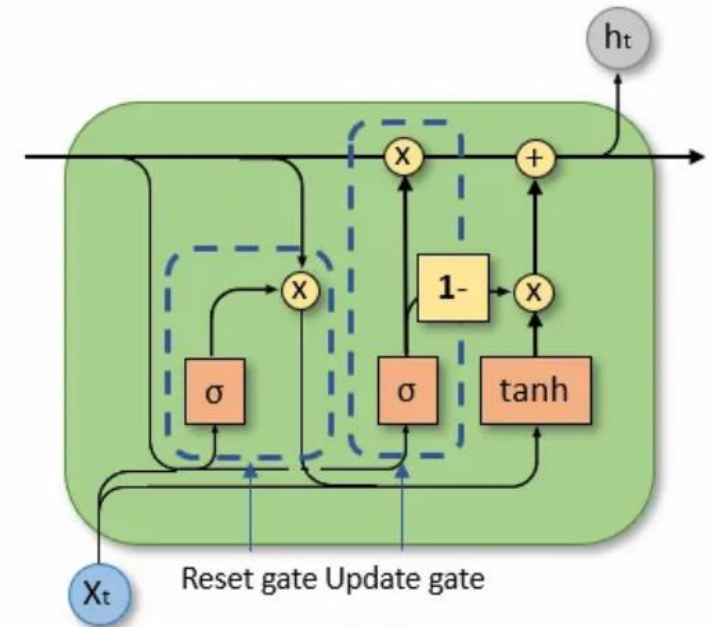
LSTM 과 GRU



RNN



LSTM

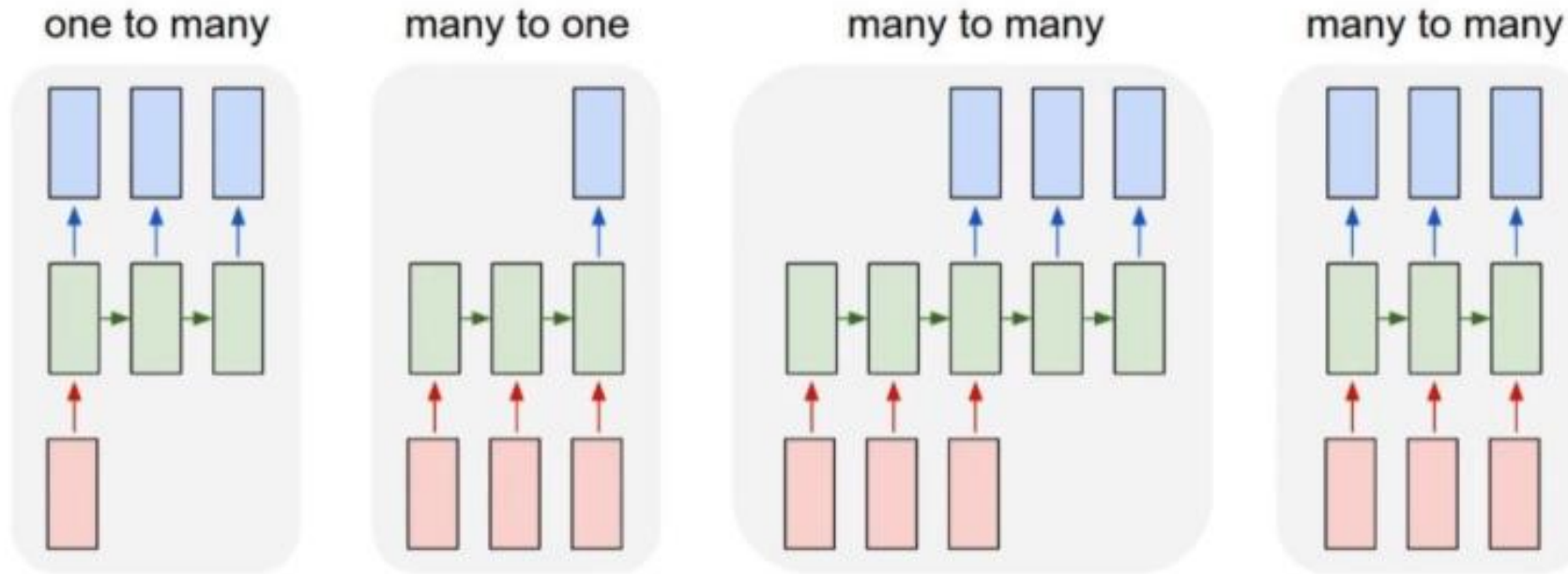


GRU

입력 Sequence와 출력 Sequence 형태별 RNN 구조 유형

- RNN 계열 모델(LSTM, GRU)은 이미지 캡셔닝, 문장 분류(감정 분석등), 품사 태깅, 개체명 인식등의 많은 언어 처리 영역에서 핵심적인 역할을 수행.
- 기존 RNN 모델은 입력 Sequence에 대해서 **동기적으로** 출력 Sequence를 생성하는 구조임. 때문에 **번역(Translation)**등 주요 언어 영역에서 RNN 적용이 어려워지면서 보다 **유연성 있는 모델** 구성 필요성 대두

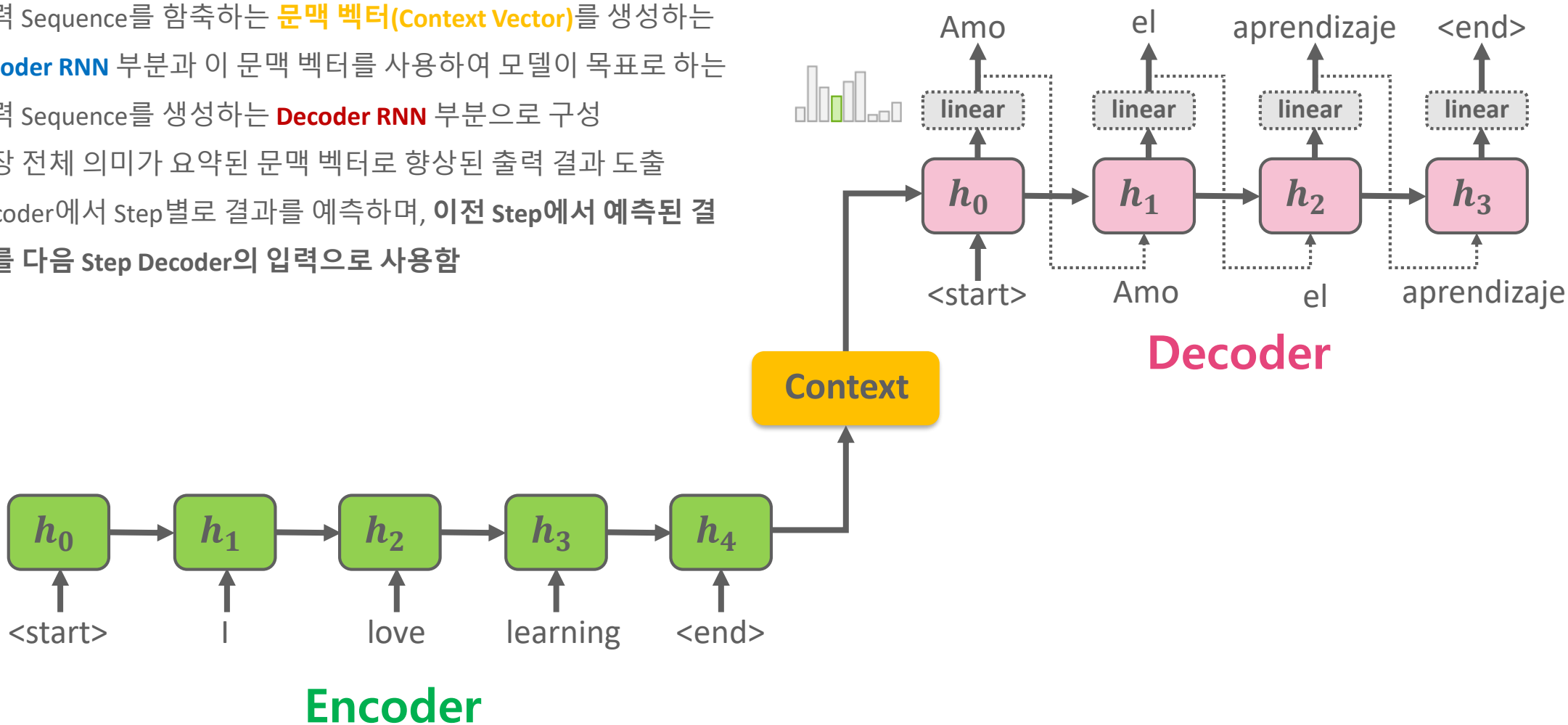
입력 Sequence와 (동기적) 출력 Sequence의 형태에 따른 RNN 구조 유형



Seq2Seq 모델 개요 - RNN 기반

하나의 Sequence를 다른 유형의 Sequence(**다른 길이**)로 매핑하면서 번역등의 NLP를 위한 유연한 모델 구성 제공

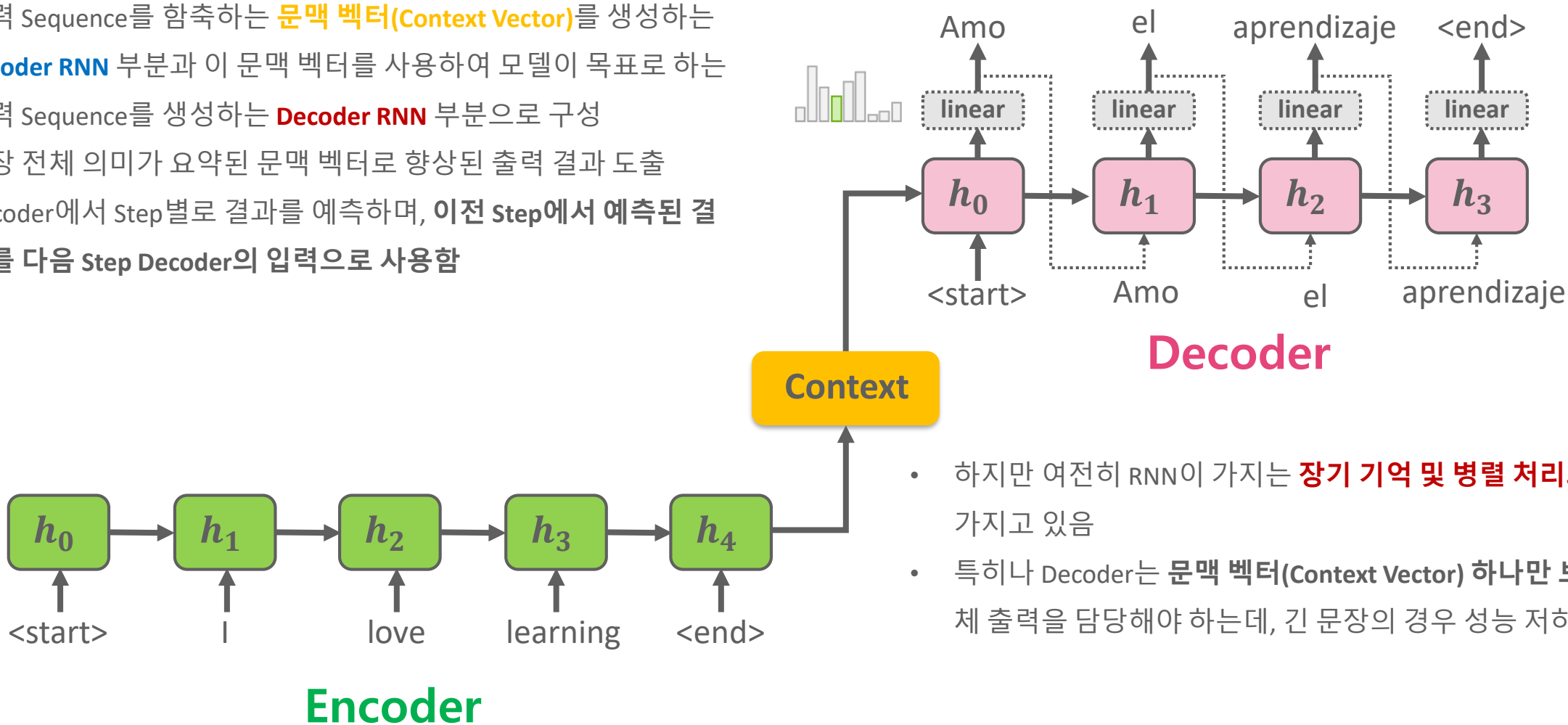
- 입력 Sequence를 함축하는 **문맥 벡터(Context Vector)**를 생성하는 **Encoder RNN** 부분과 이 문맥 벡터를 사용하여 모델이 목표로 하는 출력 Sequence를 생성하는 **Decoder RNN** 부분으로 구성
- 문장 전체 의미가 요약된 문맥 벡터로 향상된 출력 결과 도출
- Decoder에서 Step별로 결과를 예측하며, 이전 Step에서 예측된 결과를 다음 Step Decoder의 입력으로 사용함



Seq2Seq 모델 개요 - RNN 기반

하나의 Sequence를 다른 유형의 Sequence(**다른 길이**)로 매핑하면서 번역등의 NLP를 위한 유연한 모델 구성 제공

- 입력 Sequence를 함축하는 **문맥 벡터(Context Vector)**를 생성하는 **Encoder RNN** 부분과 이 문맥 벡터를 사용하여 모델이 목표로 하는 출력 Sequence를 생성하는 **Decoder RNN** 부분으로 구성
- 문장 전체 의미가 요약된 문맥 벡터로 향상된 출력 결과 도출
- Decoder에서 Step별로 결과를 예측하며, 이전 Step에서 예측된 결과를 다음 Step Decoder의 입력으로 사용함



- 하지만 여전히 RNN이 가지는 **장기 기억 및 병렬 처리의** 문제점을 가지고 있음
- 특히나 Decoder는 **문맥 벡터(Context Vector)** 하나만 보고 필요한 전체 출력을 담당해야 하는데, 긴 문장의 경우 성능 저하 발생

Attention 의 필요성

문장은 개별 단어가 **순차적**으로 이어져서 만들어지지만, 문장의 **문맥상 의미**는 반드시 단어 순으로 해석되는 것이 아님

*“The **dog** **that** barked loudly and chased the squirrels through the yard and over the fence into the woods **was** very tired”*

- 초기 Attention은 Seq2Seq 모델에 보조적인 방식으로 적용되었으며, Decoder의 **Step 별 결과 생성 시** Encoder의 **어떤 hidden state 을 보다 집중해야 하는 지** 학습을 통해 모델에서 참조할 수 있도록 설계됨.

Encoder

“I am going to the market now.”



Decoder

“나는 지금 시장에 가고 있어.”

“I” 에 보다 집중하여 “나는”을 출력

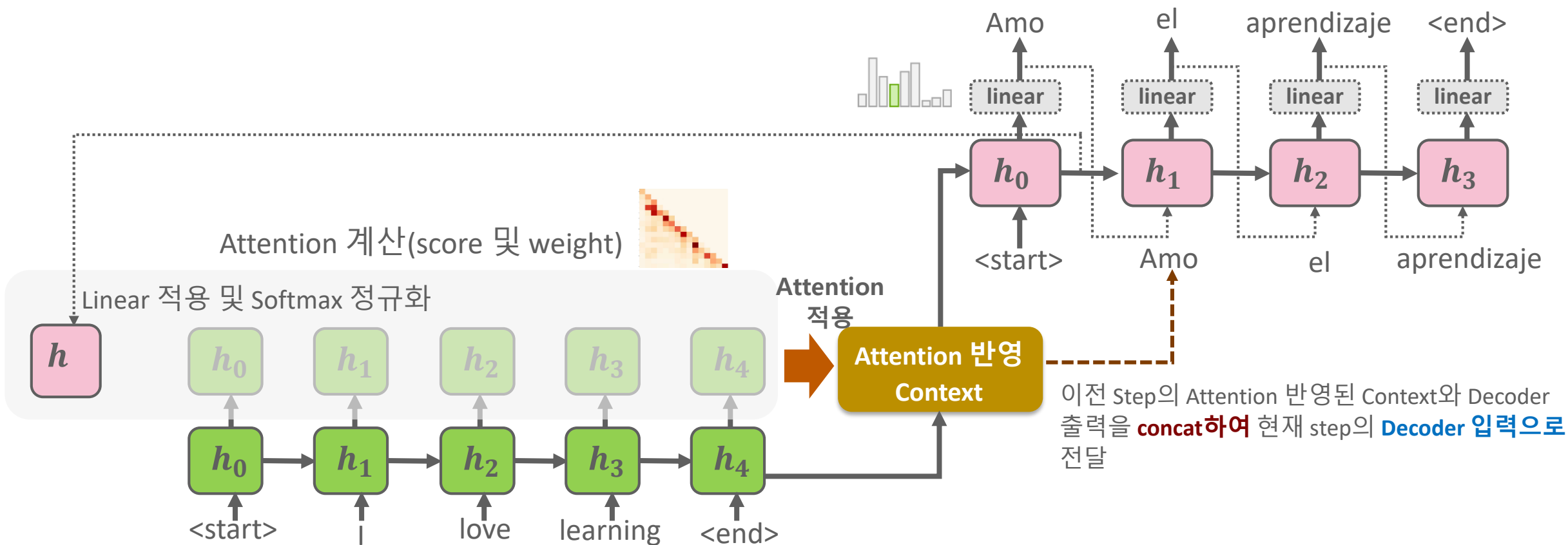
“now” 에 보다 집중하여 “지금”을 출력

“market”(또는 “the market”) 에 보다 집중하여 “시장에”를 출력

“going”(또는 am going) 에 보다 집중하여 “가고”(또는 가고 있어)를 출력

Seq2Seq with Attention 모델 개요 – RNN 기반

- **특정 시점의** Decoder Hidden State 별로 Encoder의 **모든 Hidden states들과** 얼마나 관련되어 있는 지 Attention을 계산
- 이를 위해 Decoder의 Hidden State가 생성 될 때마다 Encoder의 모든 Hidden states와 결합 후 Linear 적용 및 Softmax 정규화하여 Attention Score 및 Attention Weight 생성하며 **Context Vector는 이 Attention Weight를 반영하여 Dynamic 하게 재 생성됨**
- 새롭게 Attention Weight를 반영한 Context Vector는 Decoder 자체의 embedding 입력과 **결합**하여 다음 시점의 Decoder 입력이 됨.



RNN 기반 Seq2Seq with Attention의 문제점

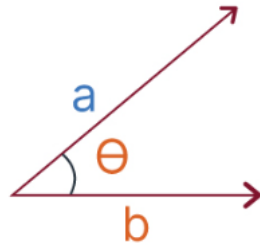
- RNN 기반 Seq2Seq with Attention은 매우 발전된 언어 모델.
- 하지만 여전히 RNN을 사용하고 있기 때문에 (Attention이 있음에도 불구하고) **긴 문장 시** Context Vector 성능 저하(장기 기억 문제), **병렬 처리의** 어려움으로 인한 매우 느린 학습 및 Inference 속도등의 주요 단점을 가지고 있음
- Decoder의 출력 생성 시에만 Decoder의 이전 Hidden states와 Encoder의 모든 Hidden States를 기반으로 Attention 적용. Encoder 또는 Decoder 내부에서 **모든 단어(Token)들 상호간 Attention**이 부재.
- 문장의 문맥 상 의미를 잘 함축하려면(좋은 Context Vector를 만들려면) **모든 단어들 간의 관계를** 잘 파악할 수 있도록 Attention 구현이 필요

코사인 유사도는 왜 Self Attention에 사용되지 않는가?

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

코사인 유사도

$$\cos \Theta = \frac{a \cdot b}{|a||b|}$$



- 코사인 유사도는 벡터간의 방향 유사성을 -1 ~ 1 사이 값으로 나타냄.
- 벡터 크기(magnitude)를 구하기 위한 computing 연산이 더 많이 사용됨.
- Gradient 계산이 더 복잡하고 불안정함
- 단어 벡터 크기는 나름의 의미를 가지고 있지만 이를 반영하지 못함

내적(Dot Product)

$$a \cdot b = |a||b| \cos \Theta$$

- GPU에서 매우 빠른 연산 수행
- 단어 벡터 크기를 반영한 유사도로 향상된 의미 관계 표현
- 보다 빠르고 안정성 있는 Gradient 계산
- Softmax와 잘 결합

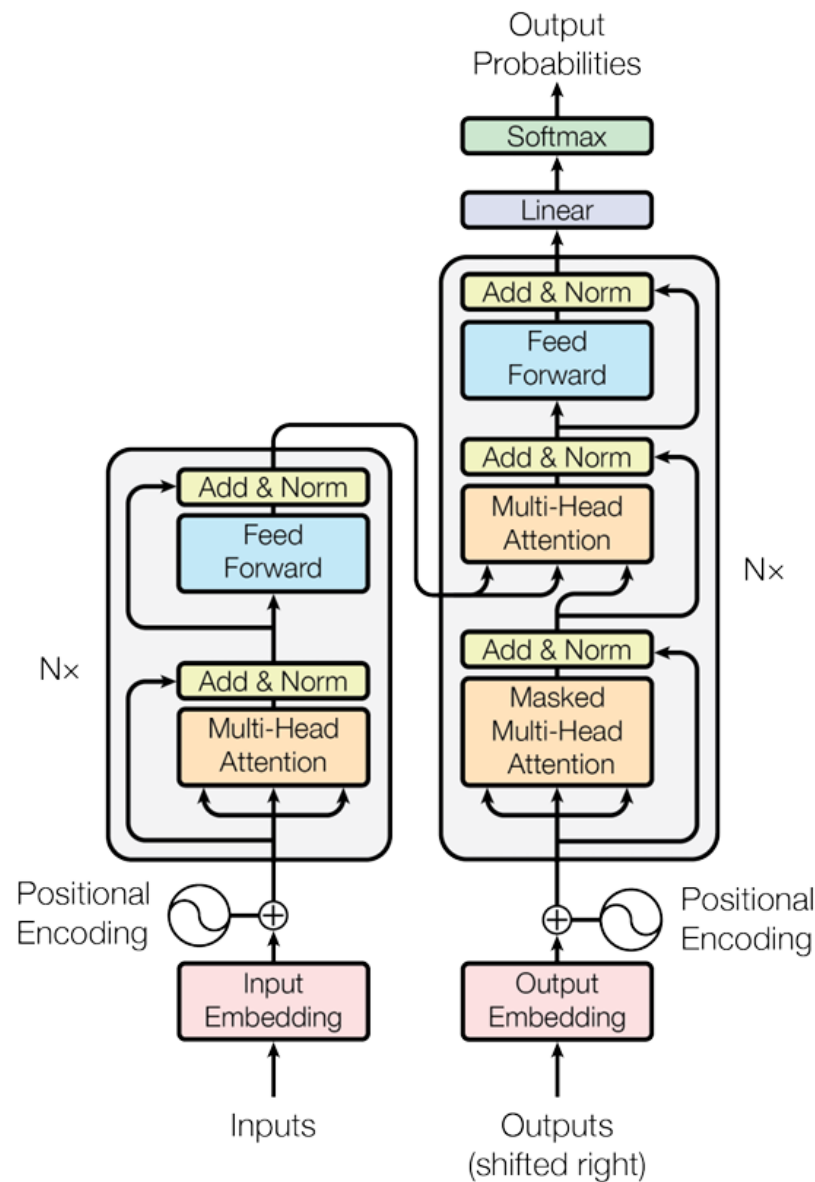
섹션 개요

Transformer – Encoder

Transformer 개요 및 Multi Head Self Attention 구현

Attention is all you need 논문 리뷰

- RNN이 없는 Seq2Seq(Encoder-Decoder)인 **Transformer** 모델 제시
- 입력 문장을 **순차적인 처리에 의존하지 않고**, 모든 단어간 관계를 동시에 고려하면서 **병렬 처리**가 가능한 **(Multi-head) Self Attention**를 기반으로 번역 모델의 성능이 SOTA를 달성하였고, 모델 학습 시간을 획기적으로 단축
- 이후 많은 언어 모델 뿐만 아니라 Vision 영역까지 활용되며, 현재 생성형 AI 붐을 주도하고 있는 LLM(Large Language Model) 과 Multimodal(CLIP, Gemini, GPT-4o) Model등의 기반 아키텍처로 자리잡음.



Transformer 모델 개요

Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Self Attention**

입력 문장내의 개별 단어(입력 토큰)와 다른 단어들과의 관심도에 기반하여 문장의 문맥을 표현하게(Context Vector 생성) 해줌(Query, Key, Value 활용)

- **Multi-Head Attention**

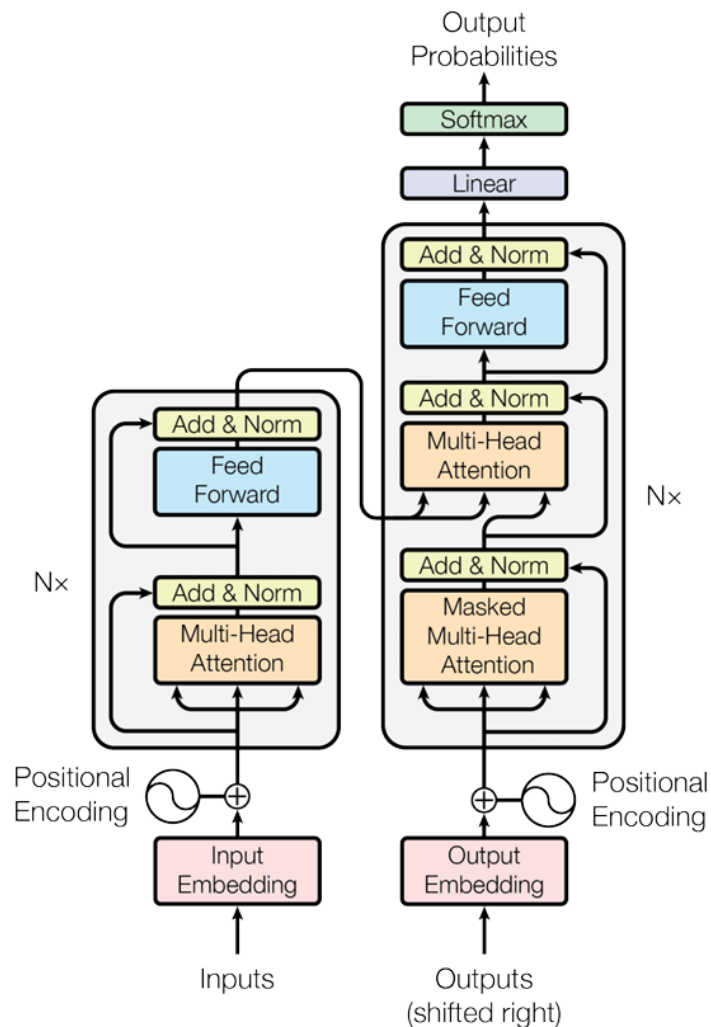
보다 다양한 관점에서의 Attention을 제공하기 위해서 여러 head 별로 self attention을 구한 후 이들을 결합하여 최종 attention을 구성

Positional Encoding

- Attention을 문장내 단어의 순서와 무관하게 병렬로 구하므로 순서를 감안한 문맥 보강을 위해서 개별 단어 Embedding 값에 해당 단어의 위치(Position) 정보를 더해서 Attention 계산에 사용함

Add & Layer Normalization

- 여러 Layer의 반복 연결 시 Gradient 소실등의 문제를 개선하기 위해서 Residual Connection과 Layer Normalization 적용



Transformer 모델 개요

Position-wise Feed Forward

- 개별 단어 토큰들의 의미 표현을 보다 강화. 비선형 activation과 Linear Layer를 개별 단어 토큰 별로 적용.

Encoder(and Decoder) Stack들

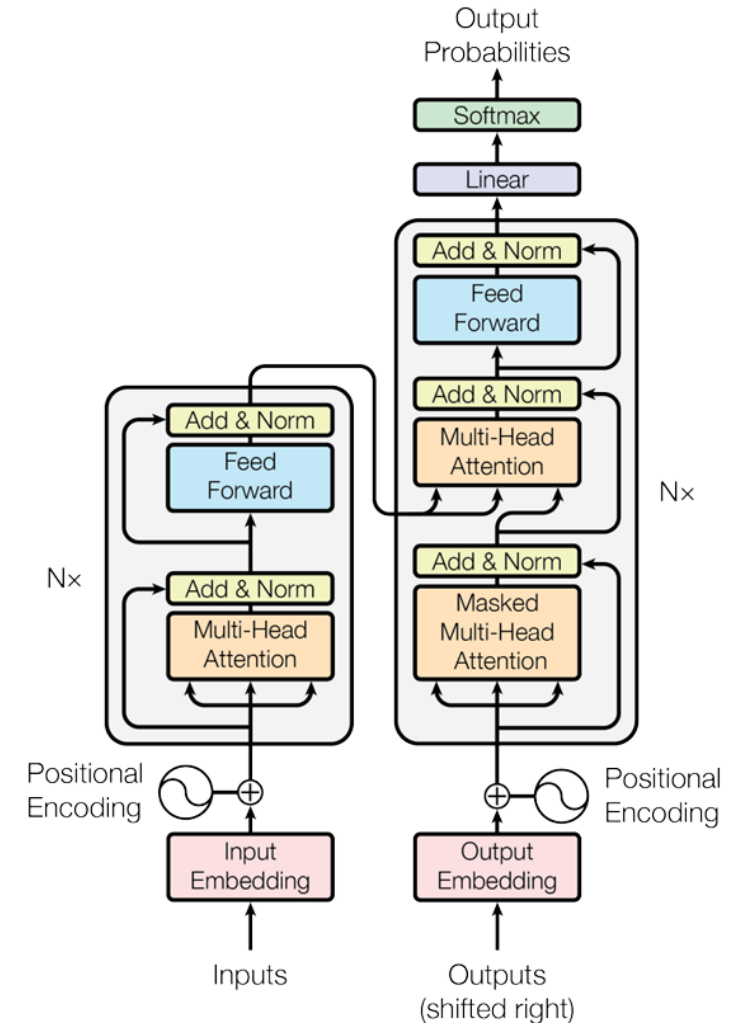
- 보다 뛰어난 문맥 이해와 표현을 위해 개별 Encoder/Decoder Layer(Block)들을 연속적으로 이어서 전체 모델 구성

Masked Multi-Head Self Attention

- Decoder는 전체 문장을 입력으로 받지만, 현 단어 다음의 단어(future word)를 포함한 Attention을 생성하면 모델 학습시 성능이 저하되므로 previous words만 사용하고 future words는 masking으로 Attention에서 제외

Cross Attention(Encoder-Decoder Attention)

- Decoder의 개별 단어 생성 시 Encoder의 출력 중 어느 부분에 집중해야 할지 알기 위해 Decoder의 개별 단어와 Encoder의 모든 출력간 Attention을 계산.



Self Attention 의미

- Self Attention은 문장내에서 개별 단어가 다른 단어들을 **얼마나 관심있게** 집중해야 하는지를 기반으로 문장의 문맥을 표현하는 방식
 - ✓ 주어진 단어에 중요한 다른 단어들은 무엇인가, 문장의 여러 부분에 있는 정보를 어떻게 결합해야 하는가?
- 단어와 다른 단어간의 **관심도(Attention Weights)**는 **관계 유사도(Similarity)**를 바탕으로 계산되며, 이 관심도를 Weighted Sum 형태로 적용하여 문맥을 반영한 단어로 표현

The cat sat on the mat

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

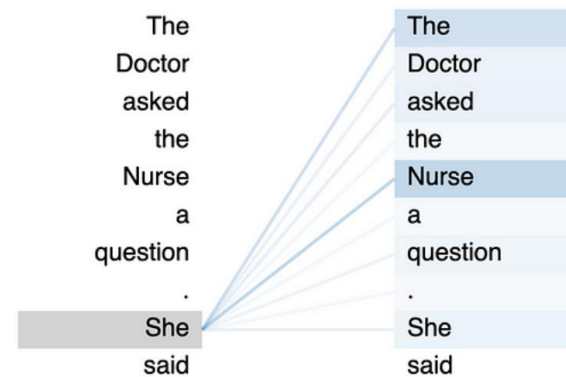
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Query: 관심도 계산을 위한 개별 기준 단어 정보

Key: 관심도 계산을 위해 Query와 연결되어야 할 모든 단어 정보

Value: 관심도가 적용되어야 할 단어 정보

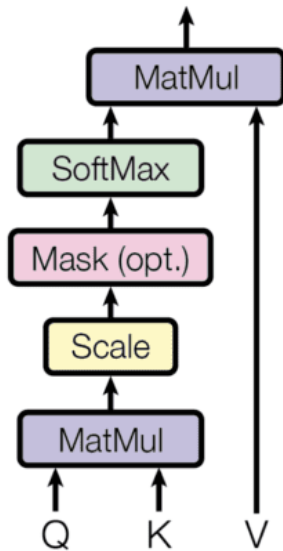
		Attention Weights					
		Key					
		The	cat	sat	on	the	mat
Query	The	0.2	0.2	0.12	0.074	0.2	0.2
	cat	0.11	0.3	0.18	0.11	0.11	0.18
	sat	0.078	0.21	0.21	0.21	0.078	0.21
	on	0.047	0.13	0.21	0.35	0.047	0.21
	the	0.2	0.2	0.12	0.074	0.2	0.2
	mat	0.096	0.16	0.16	0.16	0.096	0.33



벡터 내적에 기반한 유사도 계산

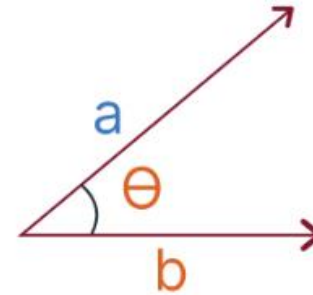
- Transformer의 Self Attention는 단어와 단어들간의 벡터 내적에 기반
- 두 벡터 사이의 내적은 스칼라 값이 되며, 이는 두 벡터 사이의 크기와 방향을 함께 고려하여 유사도(Similarity)를 반영

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

벡터의 내적(dot product)



$$a \cdot b = |a||b| \cos \Theta$$

- 하나의 벡터를 다른 벡터에 투영한 길이로 나타냄.
- 내적값은 두 벡터의 사잇각과 개별 벡터 크기로 계산
- 벡터의 내적은 두 벡터의 크기와 방향 모두를 고려한 유사도. 코사인 유사도는 두 벡터의 방향만을 고려한 유사도.

Query, Key, Value 개요

The cat sat on the mat

Query: Attention Weights를 생성 시 기준 벡터

Key: Attention Weights 생성을 위해 Query가 참조하는 벡터들

Value: Context Vector를 생성하기 위해 Attention Weights를 적용하는 벡터들

		Attention Weights					
		Key					
Query		The	cat	sat	on	the	mat
	The	0.2	0.2	0.12	0.074	0.2	0.2
	cat	0.11	0.3	0.18	0.11	0.11	0.18
	sat	0.078	0.21	0.21	0.21	0.078	0.21
	on	0.047	0.13	0.21	0.35	0.047	0.21
	the	0.2	0.2	0.12	0.074	0.2	0.2
	mat	0.096	0.16	0.16	0.16	0.096	0.33

Self Attention 구현 해보기

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- **학습 파라미터가 없는** Q, K, V 기반의 Self Attention

$$Q = X, \quad K = X, \quad V = X$$

- **학습 파라미터가 적용된** Q, K, V 기반의 Self Attention

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

1. (학습 파라미터가 적용된) 입력 Token들의 행렬 곱(matrix multiplication)으로 **Attention Score**를 계산한다
2. Attention Score에 **Softmax** 정규화를 적용하여 **Attention Weights**를 계산
3. 입력 Token들에 Attention Weights를 **반영하여** 문장 문맥을 담은 Token별 Context Vector를 생성

개별 단어(token)의 Attention Score 구하기 – 내적 Simulation

개별 단어 token과 자신의 token을 포함한 전체 token들(전체 token들의 전치행렬) **행렬곱**(Matrix Multiplication)을 적용하여 **Attention score(값)**을 구함. Attention 값은 벡터의 크기를 반영한 **유사도(similarity)**를 나타냄.

“The cat sat on the mat”

	Emb d0	Emb d1	Emb d2
The	1.0	0.0	0.0
cat	1.0	1.0	0.0
sat	0.5	1.0	0.5
on	0.0	1.0	1.0
the	1.0	0.0	0.0
mat	1.0	0.5	1.0

Emdebbding 차원은 3

cat과 다른 token들과의 Attention score 구하기

행렬곱을 개별 Vector간의 loop 적인 연산으로 표시함.

내적

	Emb d0	Emb d1	Emb d2	Attention 값 (score)
The	1.0	0.0	0.0	1.0
cat	1.0	1.0	0.0	2.0
sat	0.5	1.0	0.5	1.5
on	0.0	1.0	1.0	1.0
the	1.0	0.0	0.0	1.0
mat	1.0	0.5	1.0	1.5

Calculation details for 'cat':

- The: $1.0 \times 1.0 + 0.0 \times 1.0 + 0.0 \times 0.0 = 1$
- cat: $1.0 \times 1.0 + 1.0 \times 1.0 + 0.0 \times 0.0 = 2$
- sat: $1.0 \times 0.5 + 1.0 \times 0.5 + 0.0 \times 0.5 = 1.5$

본 교재와 실습 자료는 다른 강의나 블로그에 활용 하시면 안됩니다.

개별 단어(token)의 Attention Score 구하기 – 행렬곱

개별 단어 token과 자신의 token을 포함한 전체 token들(전체 token들의 전치행렬) **행렬곱**(Matrix Multiplication)을 적용하여 **Attention score(값)**을 구함. Attention 값은 벡터의 크기를 반영한 **유사도(similarity)**를 나타냄.

“The cat sat on the mat”

	Emb d0	Emb d1	Emb d2
The	1.0	0.0	0.0
cat	1.0	1.0	0.0
sat	0.5	1.0	0.5
on	0.0	1.0	1.0
the	1.0	0.0	0.0
mat	1.0	0.5	1.0

Emdebbding 차원은 3

cat과 다른 token들과의 Attention score 구하기

cat	1.0	1.0	0.0
-----	-----	-----	-----

Matmul
(@)

원본 행렬의 전치 행렬
“The cat sat on the mat”.**T**

1.0	1.0	0.5	0.0	1.0	1.0
0.0	1.0	1.0	1.0	0.0	0.5
0.0	0.0	0.5	1.0	0.0	1.0

1.0	2.0	1.5	1.0	1.0	1.5
-----	-----	-----	-----	-----	-----

cat에 대한 Attention **Score**

개별 단어의 Attention Weights 구하기

- 개별 단어 token과 다른 단어들간 Attention Score를 Softmax 함수로 정규화 적용하여 Attention Weights를 구함.
- Softmax는 입력값이 큰 값에 보다 집중하게 만들어 줌. 또한 미분 가능하면서도 안정적인 모델 학습 수행.

“The cat sat on the mat”

The	1.0	0.0	0.0
-----	-----	-----	-----

cat	1.0	1.0	0.0
-----	-----	-----	-----

sat	0.5	1.0	0.5
-----	-----	-----	-----

on	0.0	1.0	1.0
----	-----	-----	-----

the	1.0	0.0	0.0
-----	-----	-----	-----

mat	1.0	0.5	1.0
-----	-----	-----	-----

(다른 토큰과의) Attention을
구하고자 하는 기준 토큰

cat	1.0	1.0	0.0
-----	-----	-----	-----

cat에 대한 Attention **Score**

1.0	2.0	1.5	1.0	1.0	1.5
The	cat	sat	on	the	mat



Softmax 정규화 수행
(logit값을 확률로 변경하여 정규화 수행)

$$S(y)_i = \frac{\exp(y_i)}{\sum_{j=1}^n \exp(y_j)}$$



cat에 대한 Attention **Weights**

0.11	0.30	0.18	0.11	0.11	0.18
The	cat	sat	on	the	mat

개별 단어에 Attention Weights를 반영하기

개별 단어의 Attention Weights를 반영하여 문장 문맥 의미가 적용된 단어 만들기

$0.11 \times 1.0 + 0.3 \times 1.0 + 0.18 \times 0.5 + 0.11 \times 0.0 + 0.11 \times 1.0 + 0.18 \times 1.0 = 0.79$

$0.11 \times 0.0 + 0.3 \times 1.0 + 0.18 \times 1.0 + 0.11 \times 1.0 + 0.11 \times 0.0 + 0.18 \times 0.5 = 0.68$

$0.11 \times 0.0 + 0.3 \times 0.0 + 0.18 \times 0.5 + 0.11 \times 0.0 + 0.11 \times 1.0 + 0.18 \times 1.0 = 0.38$

cat에 대한 Attention Weight

0.11	0.30	0.18	0.11	0.11	0.18
The	cat	sat	on	the	mat

Matmul (@)

	Emb d0	Emb d1	Emb d2
The	1.0	0.0	0.0
cat	1.0	1.0	0.0
sat	0.5	1.0	0.5
on	0.0	1.0	1.0
the	1.0	0.0	0.0
mat	1.0	0.5	1.0

최초 cat token 값

	Emb d0	Emb d1	Emb d2
cat	1.0	1.0	0.0

Attention Weights가 반영된 cat token 값

= cat

0.79	0.68	0.38
------	------	------

Attention weights 반영 이전과 이후 모두 token값의 shape는 동일

문장 내 모든 Token들의 Attention Scores 구하기

- 문장 Token Embedding을 자기 자신과 Matrix Multiplication ($X * X.T$) 수행하여 모든 Token의 Attention Score를 구할 수 있음
- 이때 만들어지는 **Attention Scores의 shape**는 반드시 **Sequence 길이(문장 토큰 수) x Sequence 길이**

X tensor with (6, 3) shape

“The cat sat on the mat”

The	1.0	0.0	0.0
-----	-----	-----	-----

cat	1.0	1.0	0.0
-----	-----	-----	-----

sat	0.5	1.0	0.5
-----	-----	-----	-----

on	0.0	1.0	1.0
----	-----	-----	-----

the	1.0	0.0	0.0
-----	-----	-----	-----

mat	1.0	0.5	1.0
-----	-----	-----	-----

X.T tensor with (3, 6) shape

“The cat sat on the mat”.T

1.0	1.0	0.5	0.0	1.0	1.0
0.0	1.0	1.0	1.0	0.0	0.5
0.0	0.0	0.5	1.0	0.0	1.0

Matmul
(@)

Attention Scores는 (6, 6) Shape

	The	cat	sat	on	the	mat
The	1.0	1.0	0.5	0.0	1.0	1.0
cat	1.0	2.0	1.5	1.0	1.0	1.5
sat	0.5	1.5	1.5	1.5	0.5	1.5
on	0.0	1.0	1.5	0.0	1.0	1.5
the	1.0	1.0	0.5	0.0	1.0	1.0
mat	1.0	1.5	1.5	1.5	1.0	2.2

문장 내 모든 Token들의 Attention Scores 구하기

- 문장 Token Embedding을 자기 자신과 Matrix Multiplication ($X * X.T$) 수행하여 모든 Token의 Attention Score를 구할 수 있음
- 이때 만들어지는 **Attention Scores의 shape**는 반드시 **Sequence 길이(문장 토큰 수) x Sequence 길이**

X tensor with (6, 3) shape

“The cat sat on the mat”

The	1.0	0.0	0.0
-----	-----	-----	-----

cat	1.0	1.0	0.0
-----	-----	-----	-----

sat	0.5	1.0	0.5
-----	-----	-----	-----

on	0.0	1.0	1.0
----	-----	-----	-----

the	1.0	0.0	0.0
-----	-----	-----	-----

mat	1.0	0.5	1.0
-----	-----	-----	-----

Matmul
(@)

X.T tensor with (3, 6) shape

“The cat sat on the mat”.T

1.0	1.0	0.5	0.0	1.0	1.0
0.0	1.0	1.0	1.0	0.0	0.5
0.0	0.0	0.5	1.0	0.0	1.0

Attention Scores는 (6, 6) Shape

	The	cat	sat	on	the	mat
The	1.0	1.0	0.5	0.0	1.0	1.0
cat	1.0	2.0	1.5	1.0	1.0	1.5
sat	0.5	1.5	1.5	1.5	0.5	1.5
on	0.0	1.0	1.5	0.0	1.0	1.5
the	1.0	1.0	0.5	0.0	1.0	1.0
mat	1.0	1.5	1.5	1.5	1.0	2.2

문장 내 모든 Token들의 Attention Weights 구하기

- 2차원 행렬인 문장 Attention Scores 값을 dim=-1을 기준으로 Softmax 함수를 적용하여 Attention Weight를 구함.

Attention Scores는 (6, 6) Shape

	The	cat	sat	on	the	mat
The	1.0	1.0	0.5	0.0	1.0	1.0
cat	1.0	2.0	1.5	1.0	1.0	1.5
sat	0.5	1.5	1.5	1.5	0.5	1.5
on	0.0	1.0	1.5	0.0	1.0	1.5
the	1.0	1.0	0.5	0.0	1.0	1.0
mat	1.0	1.5	1.5	1.5	1.0	2.2

dim = -1

softmax(attn_scores, dim=-1)

Attention Weights는 (6, 6) Shape

	The	cat	sat	on	the	mat
The	0.20	0.20	0.12	0.07	0.20	0.20
cat	0.11	0.30	0.18	0.11	0.11	0.18
sat	0.07	0.21	0.21	0.21	0.07	0.21
on	0.04	0.12	0.21	0.35	0.04	0.21
the	0.20	0.20	0.12	0.07	0.20	0.20
mat	0.09	0.15	0.15	0.15	0.09	0.33

문장 내 모든 Token들에 Attention Weights 반영

- Attention Weights 와 문장의 Token들을 Matrix Multiplication 적용하여 문맥이 반영된 Context Vector들 생성
- 문맥이 반영된 Context Vector의 Shape는 입력 문장 토큰들의 Shape와 동일

Attention Weights(Shape 6x6)

	The	cat	sat	on	the	mat
The	0.20	0.20	0.12	0.07	0.20	0.20
cat	0.11	0.30	0.18	0.11	0.11	0.18
sat	0.07	0.21	0.21	0.21	0.07	0.21
on	0.04	0.12	0.21	0.35	0.04	0.21
the	0.20	0.20	0.12	0.07	0.20	0.20
mat	0.09	0.15	0.15	0.15	0.09	0.33

“The cat sat on the mat”
입력 Token Embedding(Shape 6x3)

	Emb d0	Emb d1	Emb d2
The	1.0	0.0	0.0
cat	1.0	1.0	0.0
sat	0.5	1.0	0.5
on	0.0	1.0	1.0
the	1.0	0.0	0.0
mat	1.0	0.5	1.0

Context Vector
(Shape 6x3)

The	0.86	0.49	0.33
cat	0.79	0.68	0.38
sat	0.66	0.73	0.52
on	0.51	0.78	0.66
the	0.86	0.49	0.33
mat	0.73	0.61	0.55

Matmul (@)

=

Token들의 Context Vector의 Shape는
Token들의 Shape와 동일

Query, Key, Value의 학습 파라미터 의미

- Self Attention의 Q, K, V 텐서는 입력 Tensor(토큰)에 **학습 파라미터**(Learnable Parameter)가 적용되어 선형 변환된 값
- Self Attention은 문맥이 반영 될 수 있도록 문장내 토큰과 토큰간의 **특정 관계(연관) 유사도**를 기반으로 Attention Weights를 계산하고 이를 Context Vector에 반영할 때는 Q, K, V에 부여된 **학습 파라미터로 최적화**(학습)하면서 결정
- 즉 Q, K, V의 학습 파라미터는 문장 문맥의 핵심을 잘 파악한 **특정 관계 유사도**를 찾고, 이를 Context Vector에 최적으로 적용하기 위한 주요 학습 파라미터



$$Q = X @ W_q$$



$$K = X @ W_k$$



$$V = X @ W_v$$

단어간 관계(연관) 유사도 예시

예: 문장의 전반적인 문법 구조 관계에 집중.

- 주어와 동사간의 관계(주체와 행위와의 관계)
- 행위가 일어나는 시간이나 장소와의 연관성

"The cat sat on the mat" sat은 cat에 보다 집중함

"The dog chased the ball." chased는 dog 에 보다 집중함. ball에도 집중함

"He went to the bank to deposit money." 동일 단어, 다른 의미
"He sat on the bank of the river."



의욕이
없어 보여요!

Query, Key, Value 다시 한번

Query: Attention Weights를 생성 시 적용되는 기준 벡터들

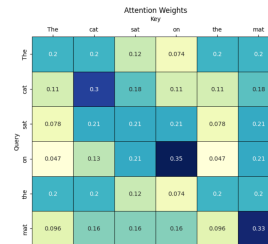
Key: Attention Weights 생성을 위해 Query가 참조하는 벡터들

Value: Context Vector를 생성하기 위해 Attention Weights를 적용하는 벡터들

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$\text{Attention Weights} = \text{Softmax} \left(\frac{\text{Query @ Key}^T}{\sqrt{d_k}} \right)$$

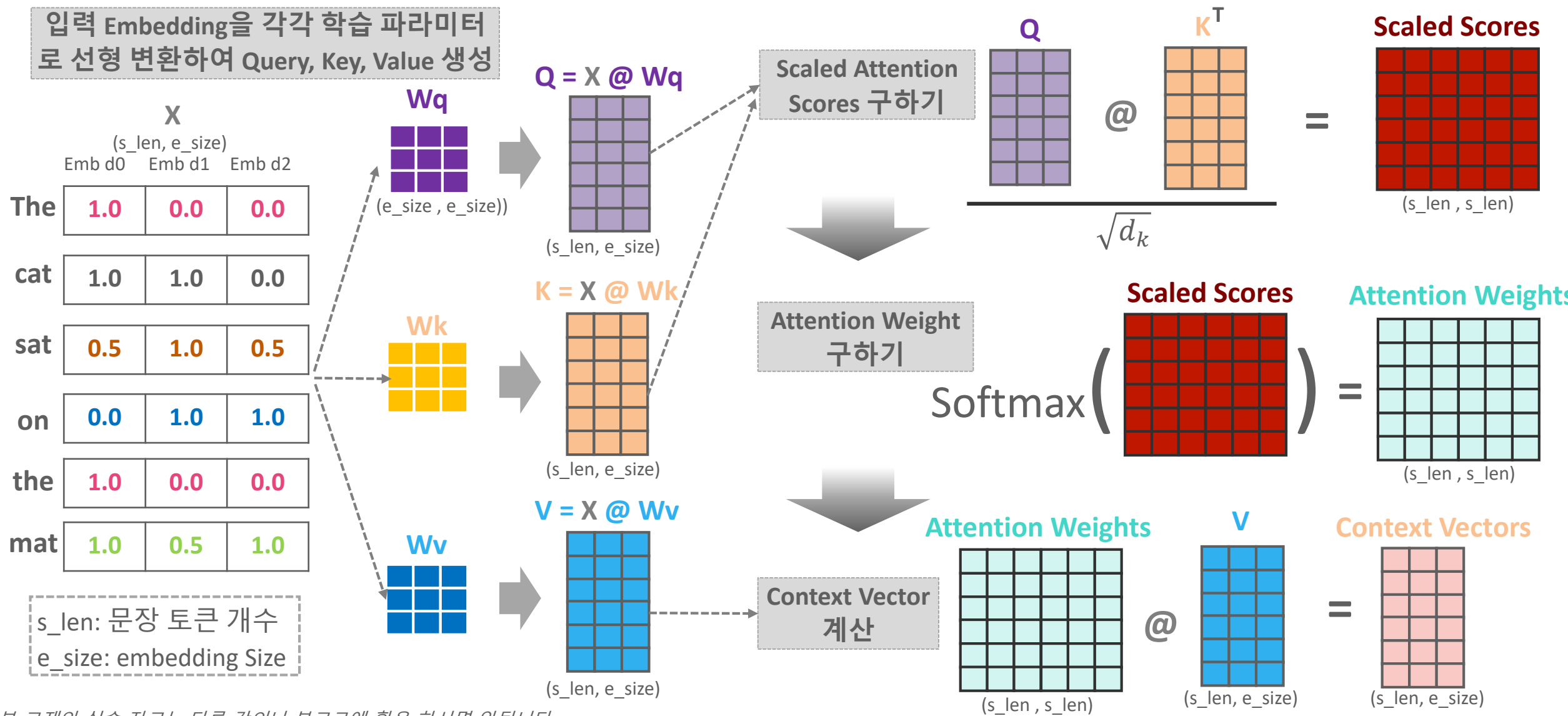
현재 선택된 단어(query)에 대해서 문장내의 어떤 단어들(Key)에 얼마나 집중해야 하는지를 Matrix 형태로 가짐



Context Vector = **Attention Weights** @ **Value**

문장내 단어들(Value)에 Attention Weights에 있는 가중치를 반영하여 문맥 의미를 반영한 Context Vector들을 생성

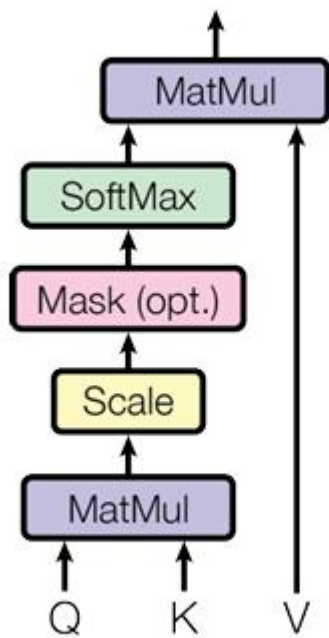
(학습 파라미터 적용된 Q, K, V를 활용한) Self Attention 구현 메커니즘



본 교재와 실습 자료는 다른 강의나 블로그에 활용 하시면 안됩니다.

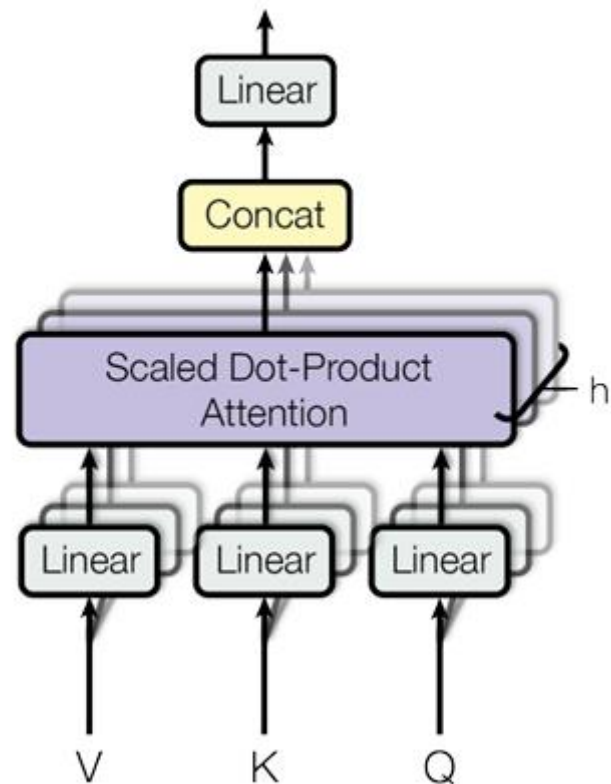
Multi-Head Self Attention 개요

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Multi Head Self Attention 의미

- Single Head Attention은 문장내 문맥을 **특정** 관계(예:명사-동사, 주어-동사-목적어)에 집중해서 attention weights를 적용하기 쉽지만 반면에 Multi-Head Self Attention은 문장내 **다양한 유형의** 관계들에 기반하여 attention weight를 적용하여 보다 **복잡한 문맥을 포괄적으로** 표현할 수 있게 만들어 줌

Multiple heads ➡ Multiple views ➡ 보다 풍부한 문맥 표현 벡터 생성

“Mary said she would arrive late because of traffic”

Head 0 = 주어-동사: Mary -> said

Head 1 = 대명사 해석: She -> Mary

Head 2 = 인과 관계: Arrive late -> because -> traffic

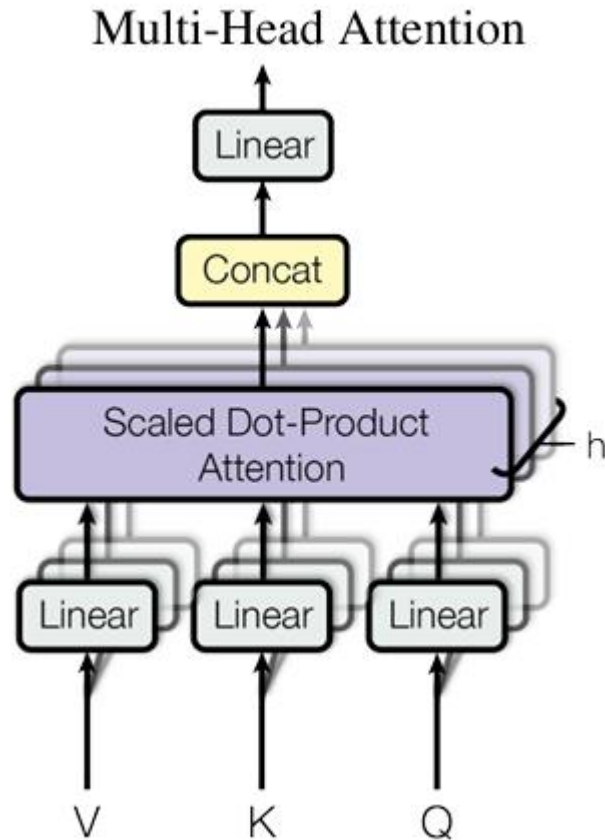
Head 3 = 말의 구조: said -> she would arrive late

Head 4 = 행위의 연결: arrive -> late

문장내 단어(token)들은 여러 관점의 다양한 관계가 있을 수 있으며 Multi-Head Attention은 이를 반영하여 Single Head Attention 보다 성능 향상

- 다양한 관점에서의 여러 Attention Weights 통해 보다 고도화된 Context Vector 생성
- 다양한 관점에서 중시된 focus들을 다 같이 고려하면서 모델 Overfitting 개선 (Generalization)

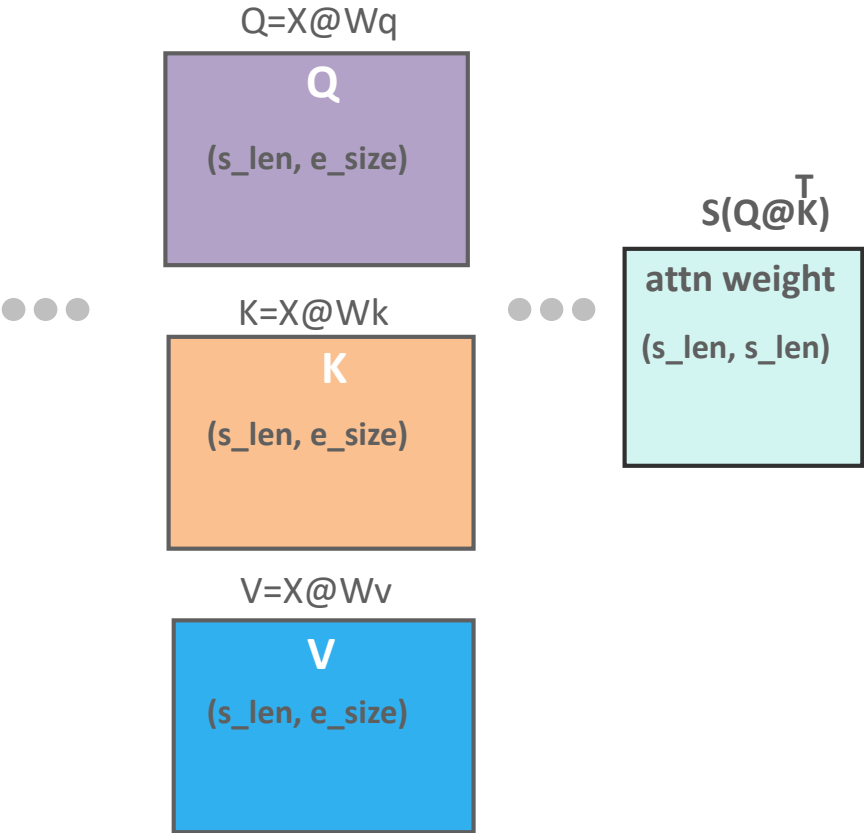
Multi-Head Attention 적용



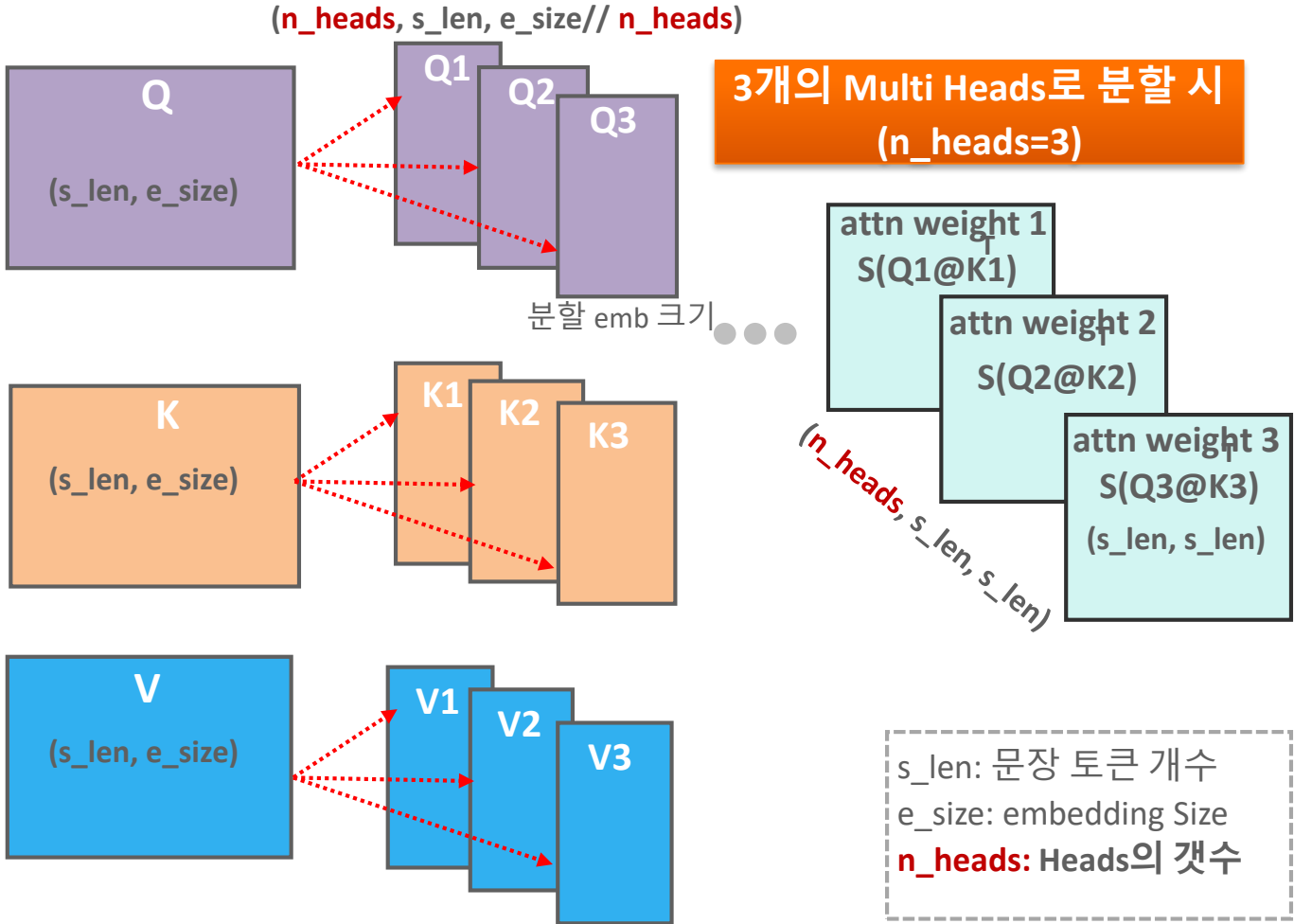
- Q, K, V의 학습 파라미터가(W_q , W_k , W_v)각각 Head의 개수만큼 쪼개져 있는 형태로 표현되어 있지만, 학습 파라미터의 Shape는 여전히 (embedding size, embedding size)이고 입력 텐서에서 학습 파라미터로 선형된 Query, Key, Value 텐서는 입력 텐서와 동일한 Shape임(Sequence length, embedding size)
- 실제 Multi-Head 적용은 Query, Key, Value를 head 레벨로 분할하면서 이루어짐.
- Multi heads 개수 만큼의 차원 축으로 분할하여 재 구성한 Query, Key들로 Attention Weights를 만들고 마찬가지로 재 구성된 Value를 적용해서 context vector를 생성

Single Head vs Multi Head Self Attention 구현 비교

Single Head Self Attention

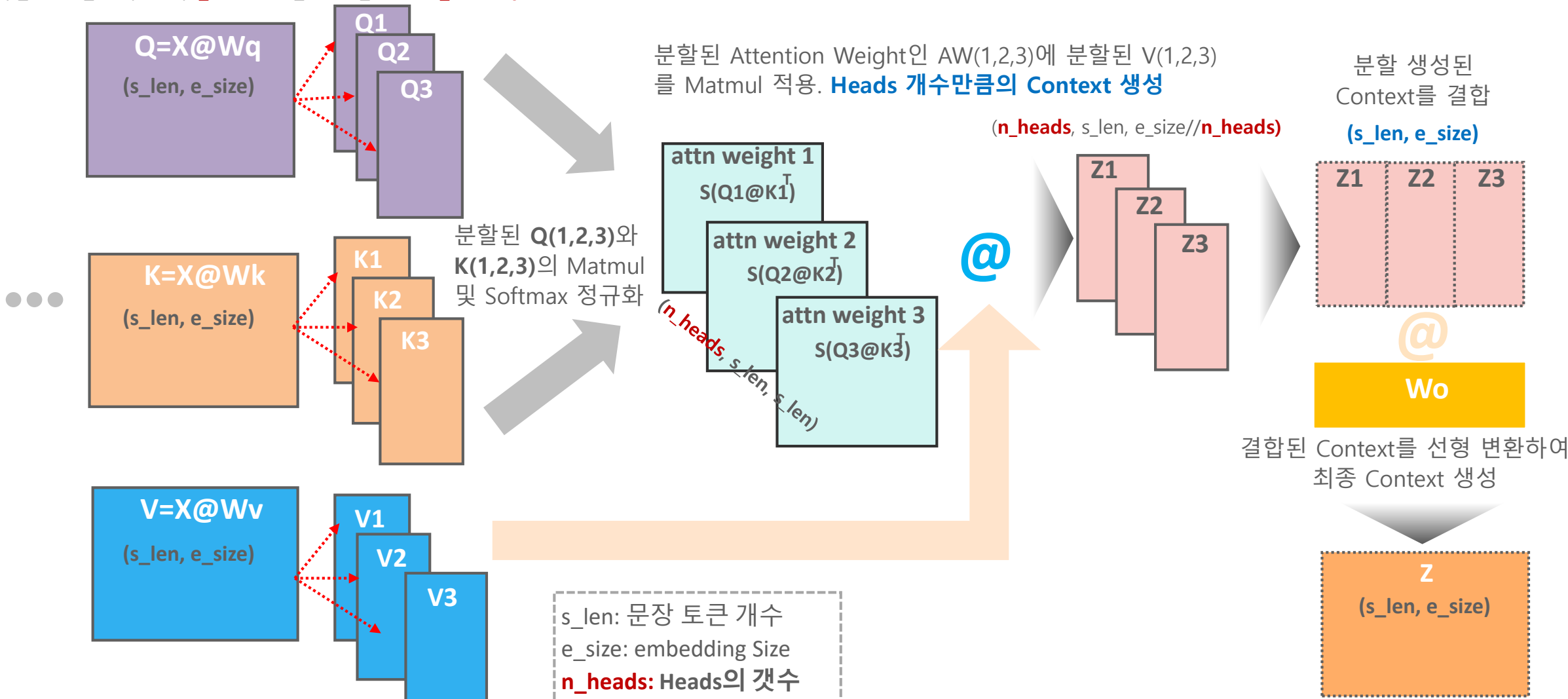


Multi Head Self Attention



Multi Head Self Attention 구현 메커니즘

Q, K, V 각각을 지정된 **Heads의 개수만큼 분할**.
(s_len, e_size) 를 (**n_heads**, s_len, e_size//**n_heads**) 로 분할



Attention Score에 Padding Mask 적용

Attention Scores

Key

Key side
padding
↓

	The	cat	sat	on	the	mat	pad	pad	pad
The	1.0	1.0	0.5	0.0	1.0	1.0	-inf	-inf	-inf
cat	1.0	2.0	1.5	1.0	1.0	1.5	-inf	-inf	-inf
sat	0.5	1.5	1.5	1.5	0.5	1.5	-inf	-inf	-inf
on	0.0	1.0	1.5	0.0	1.0	1.5	-inf	-inf	-inf
the	1.0	1.0	0.5	0.0	1.0	1.0	-inf	-inf	-inf
mat	1.0	1.5	1.5	1.5	1.0	2.2	-inf	-inf	-inf
pad	0.2	0.3	..	..	..	..	-inf	-inf	-inf
pad	..	..	..	..	..	..	-inf	-inf	-inf
pad	0.1	0.2	0.1	0.1	0.2	0.2	-inf	-inf	-inf

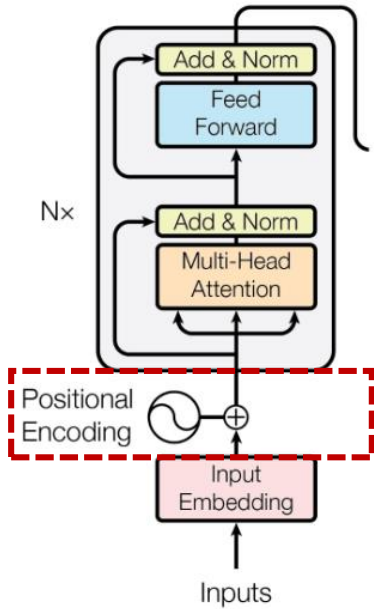
Query

Query side
padding
→

- Batch 내의 여러 개의 문장들은 서로 다른 Token 개수로 구성될 수 있으며, 동일한 Sequence Length를 위해 문장의 끝에 Padding 을 추가
- 문장의 끝에 추가되는 PAD는 **원래 문장에는 없는 Token**이므로 Attention에서 이를 반영하지 않도록 Mask 처리 필요
- Padding Mask는 보통 Attention Score 에 Masking 텐서를 적용. **Masking 처리될 영역의 값을 -inf로 변경하면** Attention Weights에서 softmax가 적용될 때 해당 값은 0으로 변환됨
- 일반적으로 Self Attention은 Key 쪽(Column 쪽)의 Padding만 적용. Query 쪽의 Padding은 별도로 하지 않는 이유는 Encoder 의 최종 출력 또는 Decoder Cross Attention에서 해당 Padding 값이 아예 사용되지 않기 때문

Positional Encoding(위치 인코딩)의 필요성

동일한 단어들(토큰)로 구성된 문장이지만, 위치에 따라 전혀 다른 문맥을 구성



The cat sat on the mat

The **dog** chased another **dog**

The **dog** chased the **cat**

The **cat** chased the **dog**

All students **who** failed the exam received a warning from the teacher.

All students received a warning from the teacher **who** failed the exam.

- 문장내 단어와 다른 단어들간의 관심도(Attention Weight)보다 **문장내 i번째 위치의 단어와 다른 위치의 단어들간**의 관심도가 보다 향상된 문맥 반영
- 이를 위해 문장내 단어(토큰)들의 위치(Position)을 표현하고, 이를 모델에 반영할 수 있는 방안 필요

토큰 Embedding에 Positional Encoding을 적용하는 방식

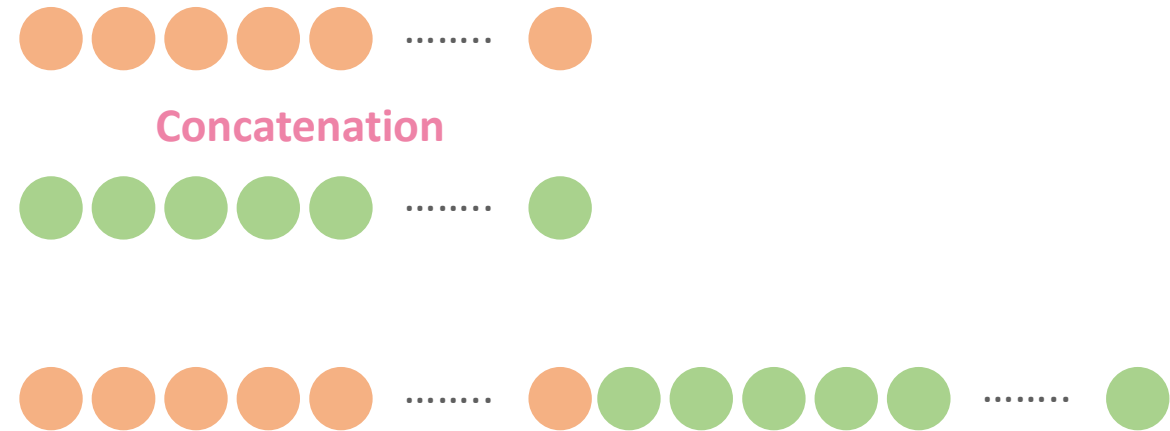
토큰 Embedding에 Positional Encoding을 더하는 방식을 거의 대부분 사용.

토큰 embedding과 Positional Encoding의 값을 **더함**



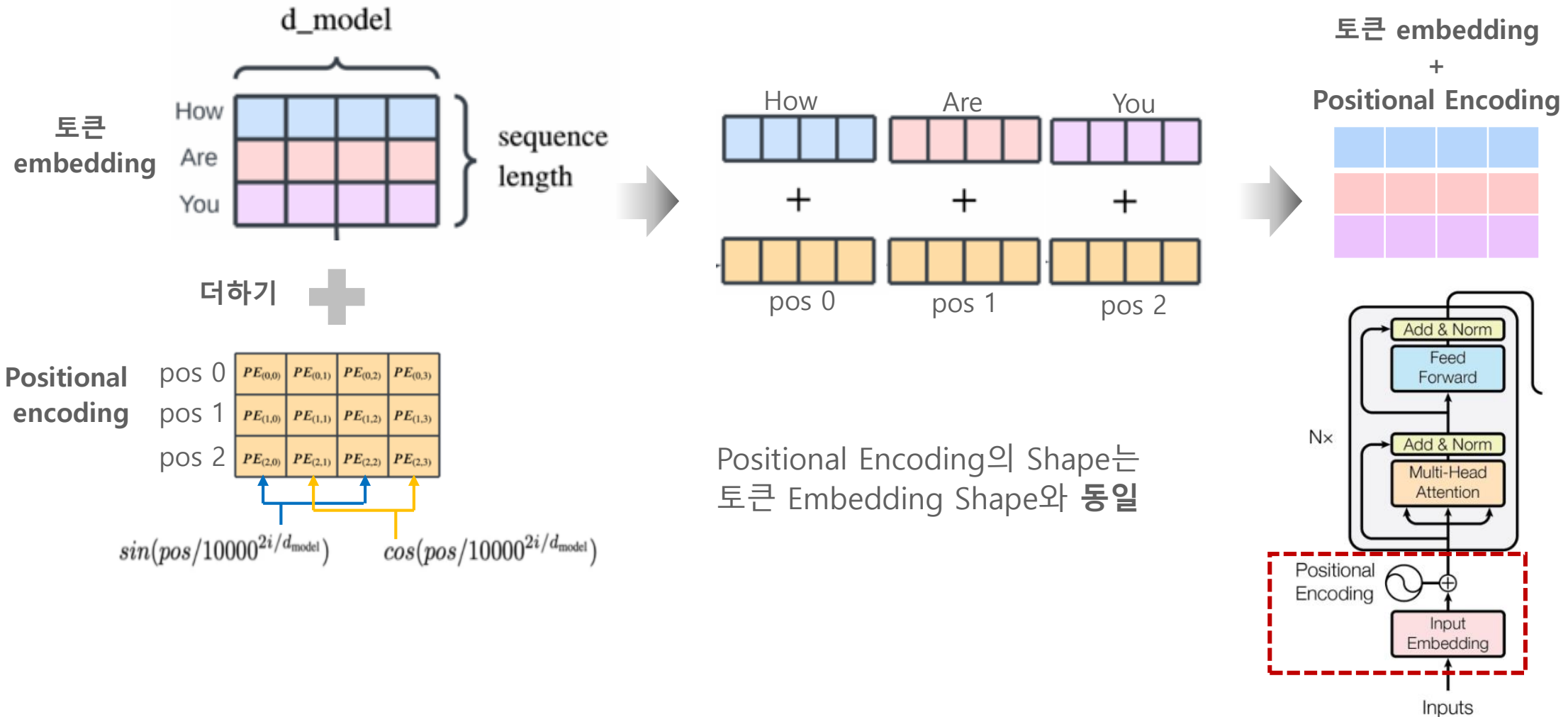
- 토큰 embedding에 Positional Encoding을 더하므로 원래 토큰 embedding 값이 **변경됨**.
- Positional Encoding값은 원래 토큰 embedding이 가지는 의미적인 값의 변경을 **최소화**하면서도 위치가 반영되어 **더욱 향상된 문맥을** 반영할 수 있도록 만들어 져야함

토큰 embedding과 Positional Encoding을 **Concatenation**



- 토큰 embedding 텐서에 Positional Encoding 텐서를 Concat함
- 토큰 embedding 값이 훼손되지는 않지만, 텐서내에 완전히 다른 정보인 embedding과 위치값을 **함께 가지게** 되면서 Attention Weight 기반의 학습 최적화가 어려워 지고, **추가적인** 메모리도 더 많이 필요

토큰 Embedding에 Positional Encoding 적용



정수 index값 기반의 Positional Encoding 사용 시 문제

- 행렬에서 정수 index 값 기반의 Position Encoding을 사용할 경우 위치 정보 자체를 제공하는 역할보다는 **대소를** 가지는 값의 영향력이 더 커지는 문제 발생.
- 정수 위치 index값을 문장내 토큰 개수 기반으로 0 ~ 1 사이로 정규화 하면, 문장내 **토큰 개수**에 따라서 동일 위치 인덱스 값이 변하게 되므로 위치 인덱스의 역할을 제대로 수행하기 어려움

Token Embedding(d=3)				
0.46, 0.83, 0.69	0.70, 0.94, 0.39	0.51, 0.79, 0.45		0.47, 0.39 0.81
e1	e2	e3	e...	e100
+				
정수 Index 기반 Position Encoding				
1	2	3		100
p1	p2	p3	p...	p100

Position Encoding의 값이 Embedding 값 보다 **너무 커서**, 값을 더하면 원 Embedding 값이 크게 왜곡됨

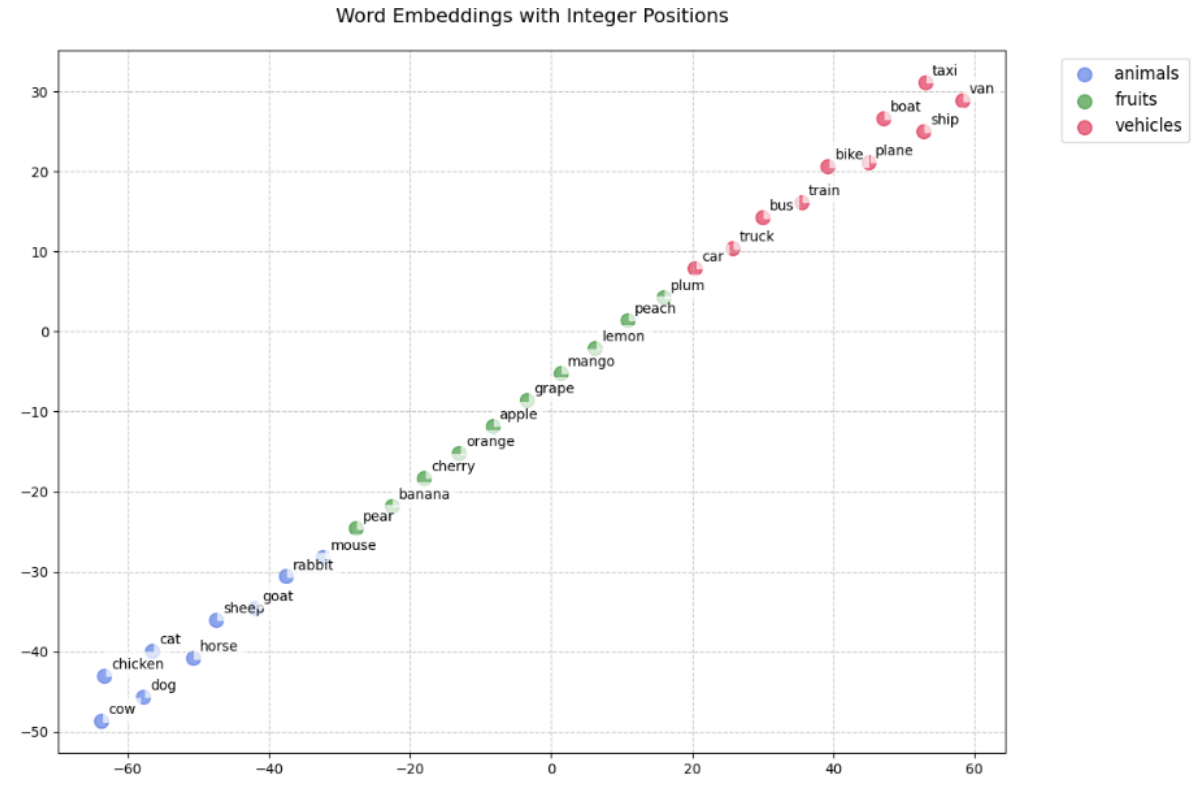
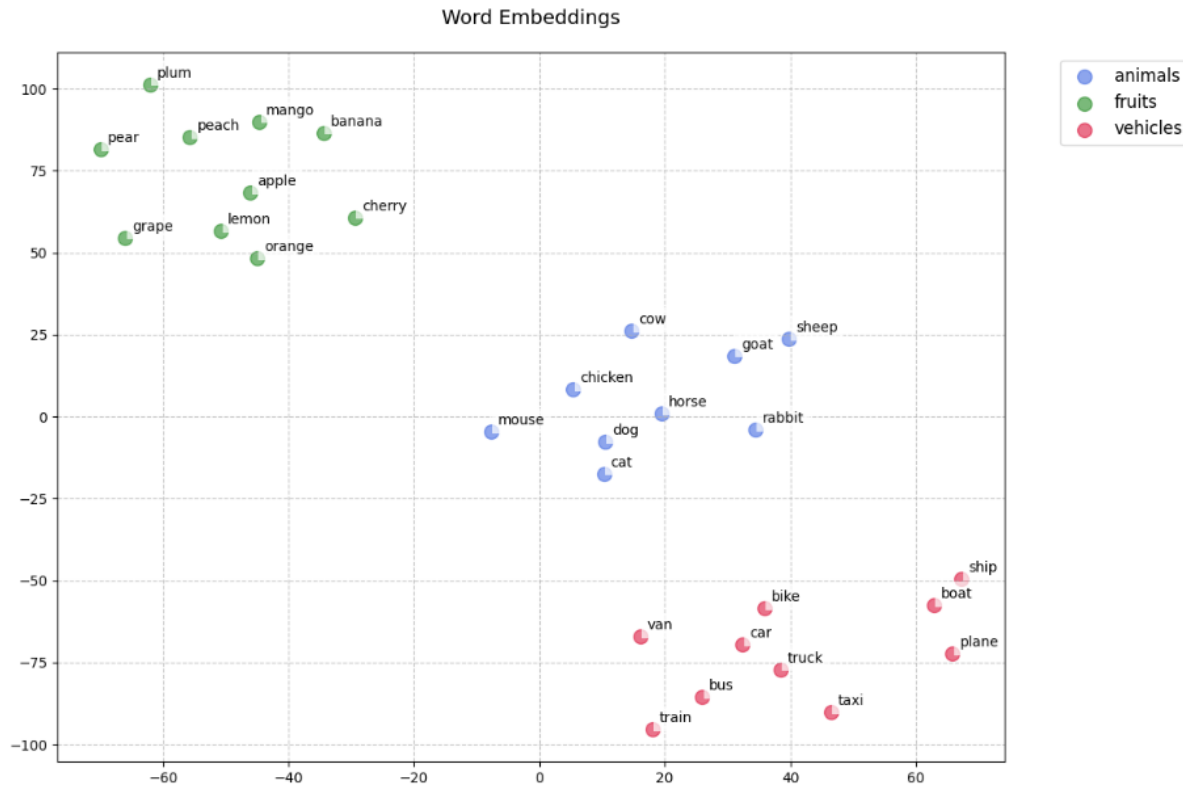
Token Embedding(d=3)				
0.46, 0.83, 0.69	0.70, 0.94, 0.39	0.51, 0.79, 0.45		0.47, 0.39 0.81
e1	e2	e3	e...	e100
+				
100개 위치 index를 0 ~ 1로 정규화 한 Position Encoding				
0	0.01	0.02		1
p1	p2	p3	p...	p100

만약 3개의 단어를 가지는 문장이라면, 아래와 같이 2번째, 3번째 index값이 이전 index값과 달라져서 위치 index로서의 역할을 올바르게 수행하기 어려우며, 또한 정규화 이전보다는 작지만, 여전히 대소값으로의 영향이 Embedding 값에 미치는 영향이 큼

0	0.5	1
p1	p2	p3

정수 index값 기반의 Positional Encoding 사용 시 문제

- 원 Embedding 값이 Positional Encoding 값 자체에 대한 영향력이 너무 커지게 됨.



좋은 Positional Encoding의 주요 조건

- 개별 위치를 고유한 값으로 나타낼 수 있어야 함
- 토큰 Embedding 값을 크게 변형 시키지 않도록 함
- 매우 긴 sequence라도 모델이 쉽게 학습하여 인지 할 수 있도록 특정 형태의(또는 고정된) **패턴**으로 위치 값 생성
- 위치간 상호 관계, 가령 특정 위치가 다른 위치에서 얼마나 떨어져 있는지를 나타낼 수 있는 방식 지원

이진수로 숫자 표현

0:	0	0	0	0
1:	0	0	0	1
2:	0	0	1	0
3:	0	0	1	1
4:	0	1	0	0
5:	0	1	0	1
6:	0	1	1	0
7:	0	1	1	1
8:	1	0	0	0
9:	1	0	0	1
10:	1	0	1	0
11:	1	0	1	1
12:	1	1	0	0
13:	1	1	0	1
14:	1	1	1	0
15:	1	1	1	1

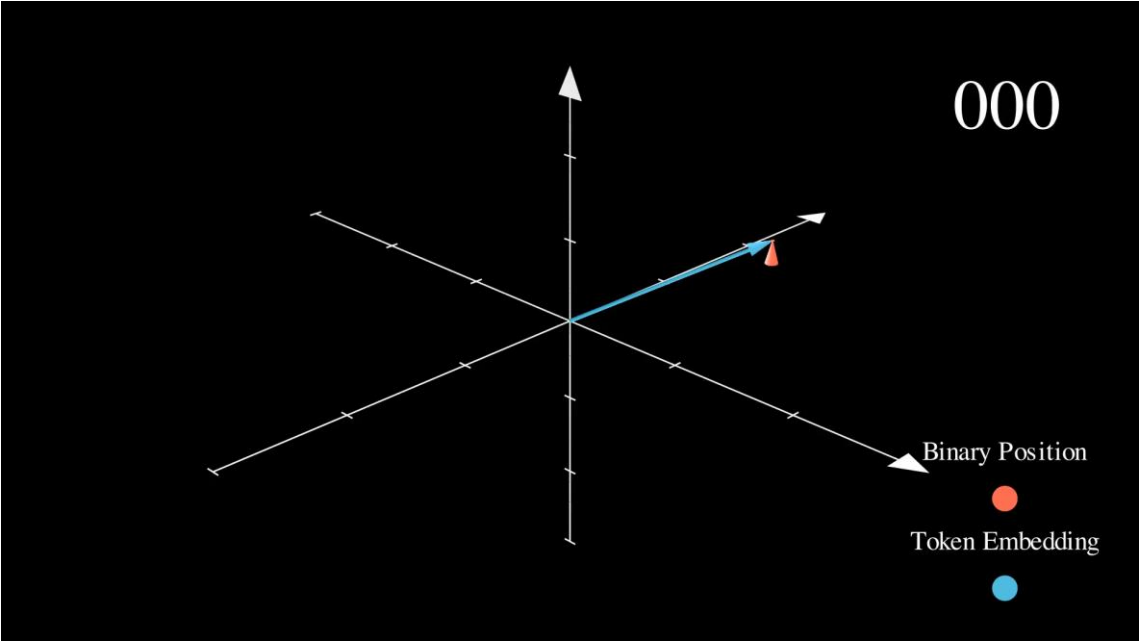
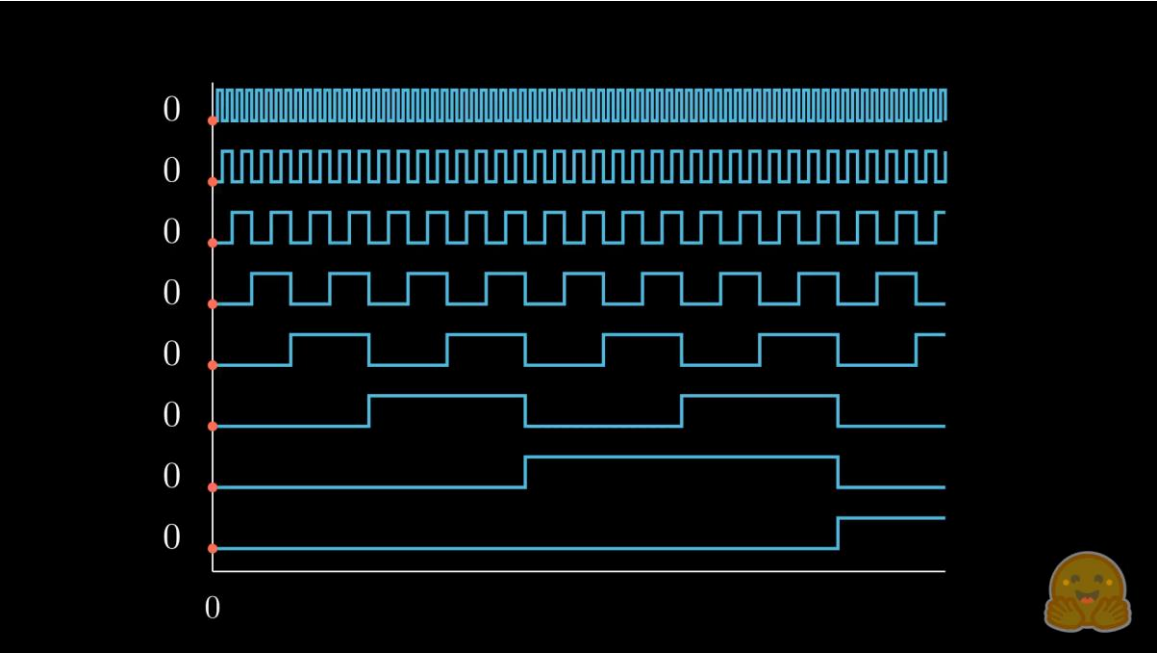
1부터 차례로 15까지 숫자를 이진수로 표현할 때 나타나는 패턴

- 맨 마지막 비트는 0과 1이 매번 반복됨(2회 주기로 0과 1이 됨)
- 맨 마지막에서 다음 왼쪽은 0이 연속 2회, 다음으로 1이 연속 2회로 매번 반복됨
- 맨 마지막에서 그 다음 왼쪽은 0이 연속 4회, 다음으로 1이 연속 4회로 매번 반복됨
- 맨 왼쪽은 0 연속 8회, 다음으로 1이 연속 8회로 매번 반복됨

특정 패턴의 조합으로 고유한 숫자를 표현할 수 있음

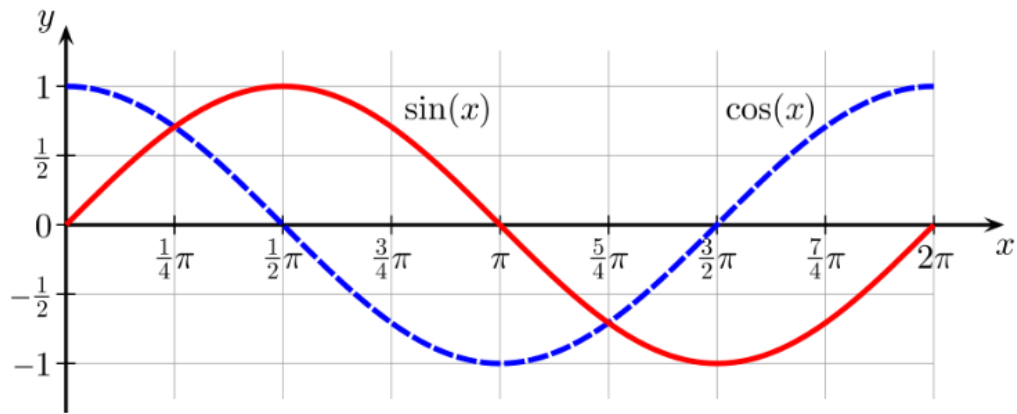
Binary Encoding 방식과 문제점

<https://huggingface.co/blog/designing-positional-encoding>



Sinusoidal(정현파) 소개

삼각함수의 사인(sine) 또는 코사인(cosine) 그래프와 같이 주기적으로 반복되는 부드럽고 규칙적인 파형을 의미



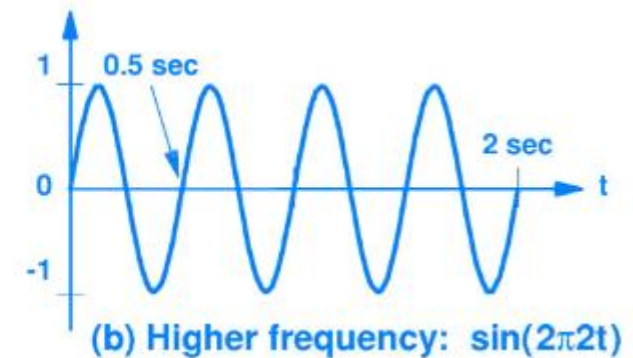
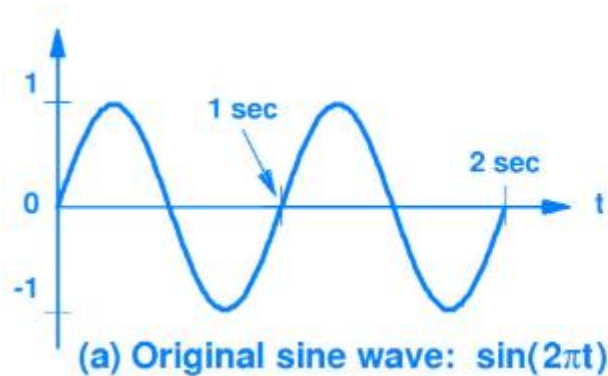
$$y(t) = A \cdot \sin(\omega t + \phi) \quad y(t) = A \cdot \cos(\omega t + \phi)$$

$A \rightarrow$ 진폭(파형의 높이)

$\omega \rightarrow$ 각주파수(얼마나 빠르게 진동하는가?)

$t \rightarrow$ 시간 또는 위치

$\phi \rightarrow$ 시작 위치 좌우 이동



Sinusoidal Positional Encoding 이해

Sinusoidal Positional Encoding 공식

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

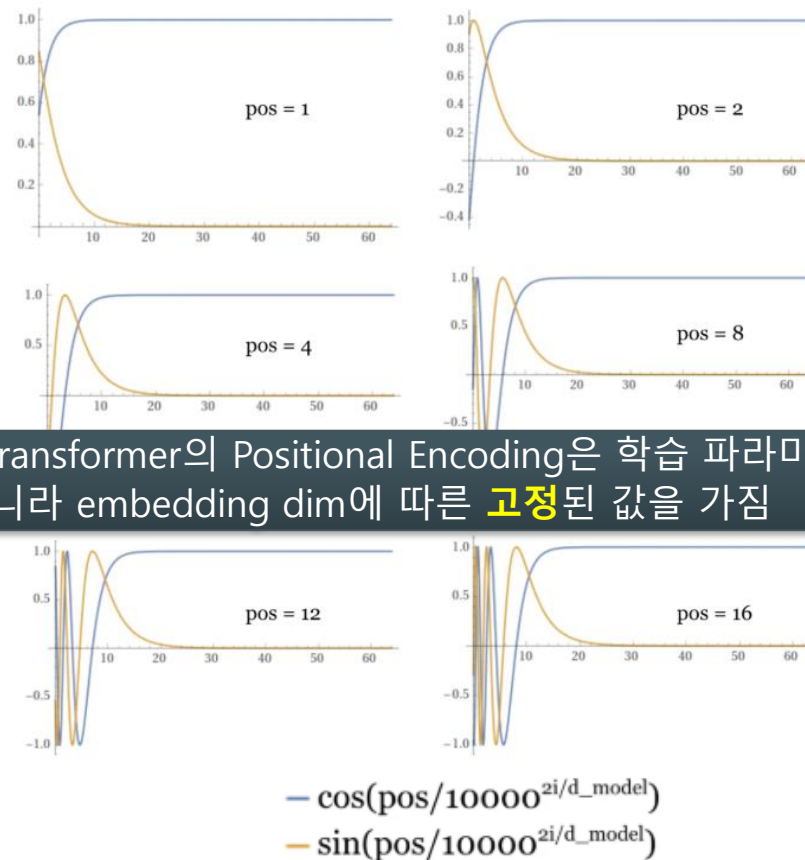
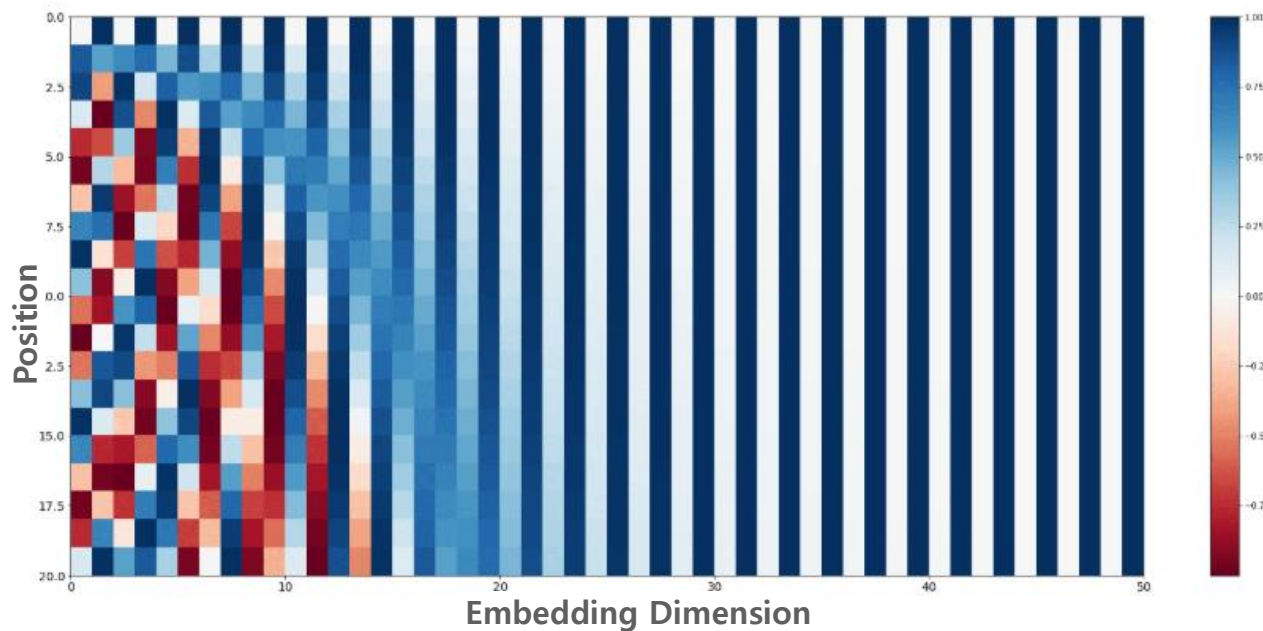
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

문장내 token 위치

$$pos \in [0, 1, 2, \dots, L - 1]$$

Embedding dimension

$$i \in [0, 1, 2, \dots, d_{model} - 1]$$



Original Transformer의 Positional Encoding은 학습 파라미터가 아니라 embedding dim에 따른 **고정**된 값을 가짐

$$\begin{aligned} \text{angle} &= pos \times \frac{1}{10000^{2i/d_{model}}} \\ &= pos \times \omega_i \end{aligned}$$

- 작은 embedding dimension 값 i 에는 주파수가 커서 파형이 **빨리 변하고**, i 값이 커질수록 주파수가 더 작아져서 파형이 **느리게 변함**
- 각 position 별로 패턴이 있으며, position이 근처에 있는 값은 서로 비슷한 패턴 값을 가지며
- position이 커질수록 각도가 증가하고 sine/cosine 파가 주기를 따라 이동
- Sine과 Cosine이 함께 사용되어 position의 상대 위치(회전 기반)를 용이하게 표현

Sinusoidal Positional Encoding 값 예시

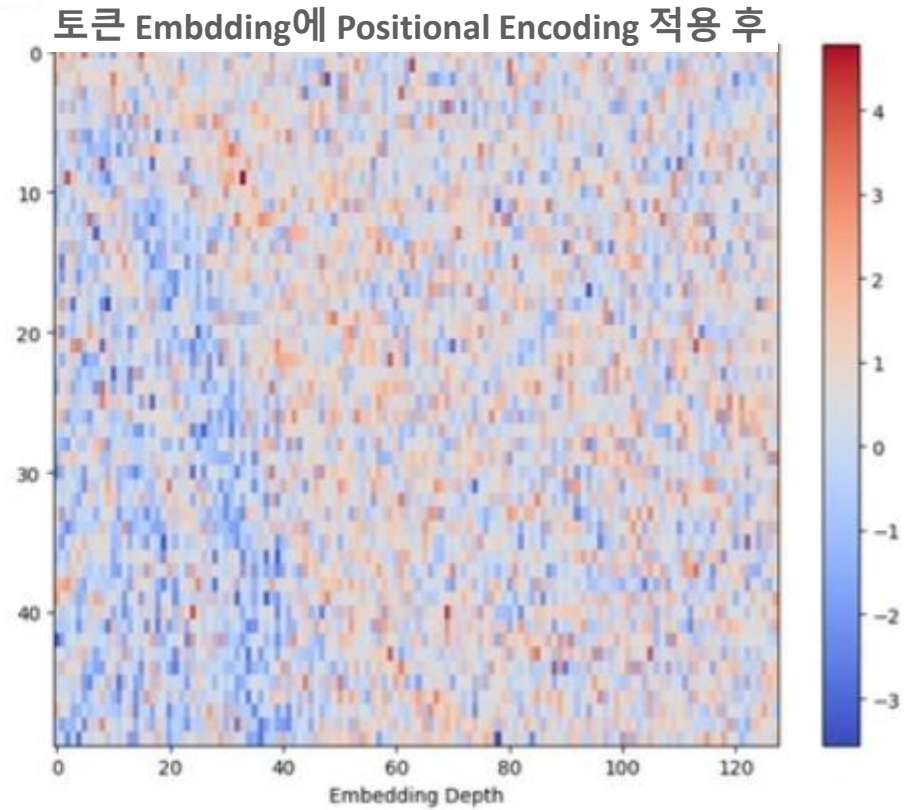
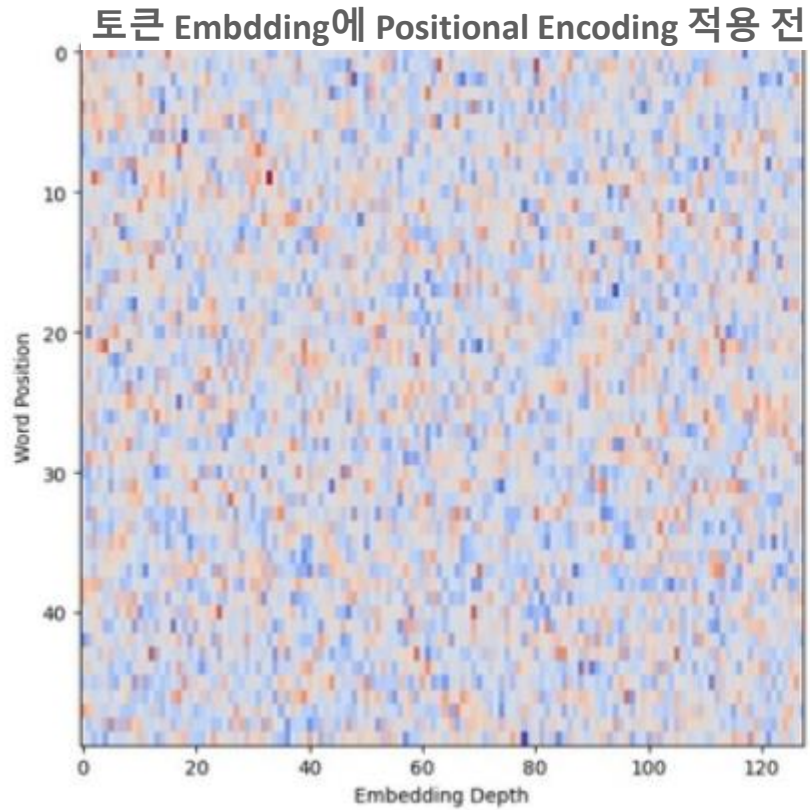
$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

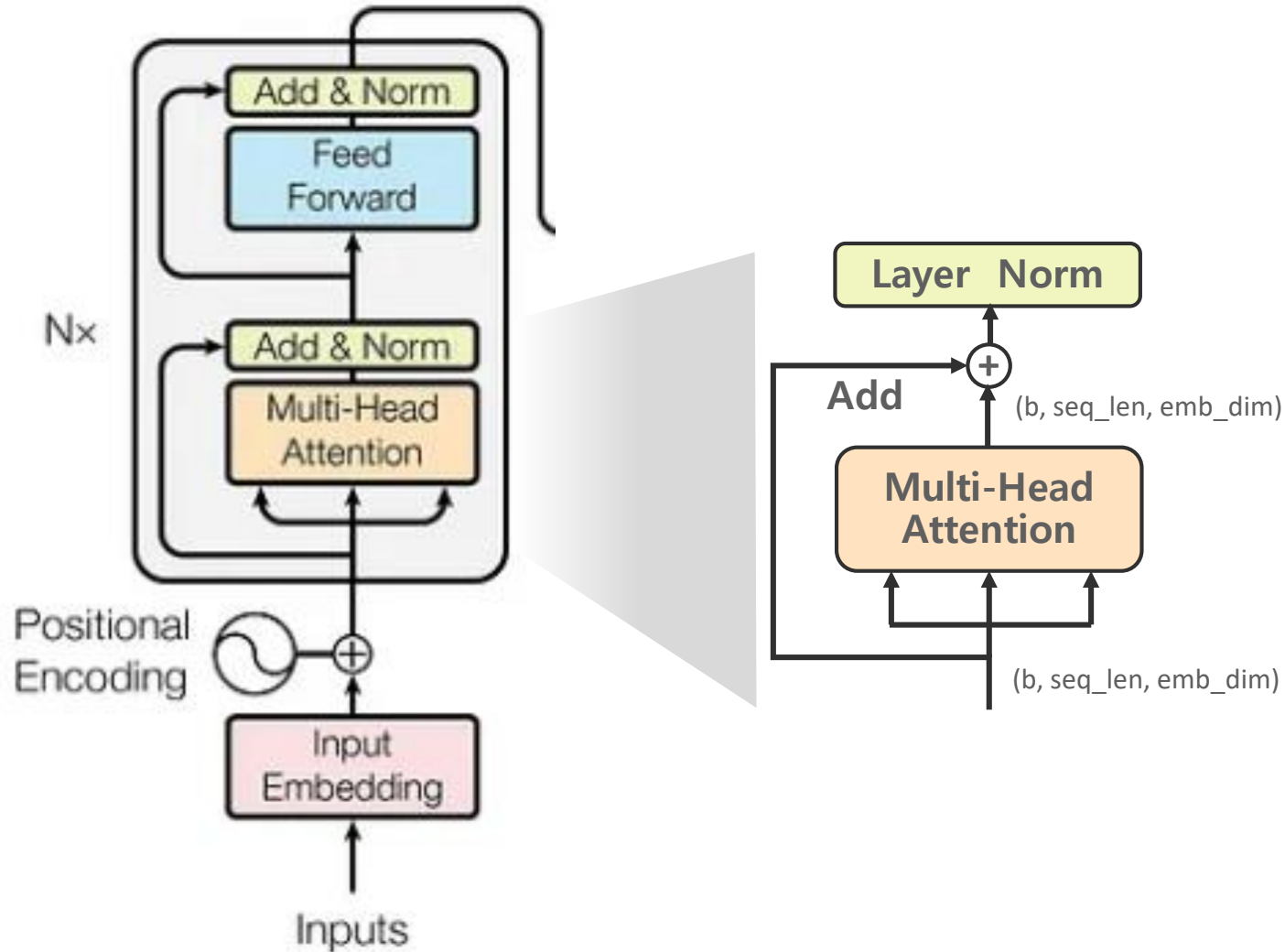
Position은 0부터 11까지, Embedding Dimension은 0 부터 15까지

	dim_0	dim_1	dim_2	dim_3	dim_4	dim_5	dim_6	dim_7	dim_8	dim_9	dim_10	dim_11	dim_12	dim_13	dim_14	dim_15
pos_0	0.00	1.00	0.00	1.00	0.00	1.00	0.00	1.00	0.00	1.00	0.00	1.0	0.00	1.0	0.0	1.0
pos_1	0.84	0.54	0.31	0.95	0.10	1.00	0.03	1.00	0.01	1.00	0.00	1.0	0.00	1.0	0.0	1.0
pos_2	0.91	-0.42	0.59	0.81	0.20	0.98	0.06	1.00	0.02	1.00	0.01	1.0	0.00	1.0	0.0	1.0
pos_3	0.14	-0.99	0.81	0.58	0.30	0.96	0.09	1.00	0.03	1.00	0.01	1.0	0.00	1.0	0.0	1.0
pos_4	-0.76	-0.65	0.95	0.30	0.39	0.92	0.13	0.99	0.04	1.00	0.01	1.0	0.00	1.0	0.0	1.0
pos_5	-0.96	0.28	1.00	-0.01	0.48	0.88	0.16	0.99	0.05	1.00	0.02	1.0	0.00	1.0	0.0	1.0
pos_6	-0.28	0.96	0.95	-0.32	0.56	0.83	0.19	0.98	0.06	1.00	0.02	1.0	0.01	1.0	0.0	1.0
pos_7	0.66	0.75	0.80	-0.60	0.64	0.76	0.22	0.98	0.07	1.00	0.02	1.0	0.01	1.0	0.0	1.0
pos_8	0.99	-0.15	0.57	-0.82	0.72	0.70	0.25	0.97	0.08	1.00	0.03	1.0	0.01	1.0	0.0	1.0
pos_9	0.41	-0.91	0.29	-0.96	0.78	0.62	0.28	0.96	0.09	1.00	0.03	1.0	0.01	1.0	0.0	1.0
pos_10	-0.54	-0.84	-0.02	-1.00	0.84	0.54	0.31	0.95	0.10	1.00	0.03	1.0	0.01	1.0	0.0	1.0
pos_11	-1.00	0.00	-0.33	-0.94	0.89	0.45	0.34	0.94	0.11	0.99	0.03	1.0	0.01	1.0	0.0	1.0

토큰 Embedding + Positional Encoding 적용

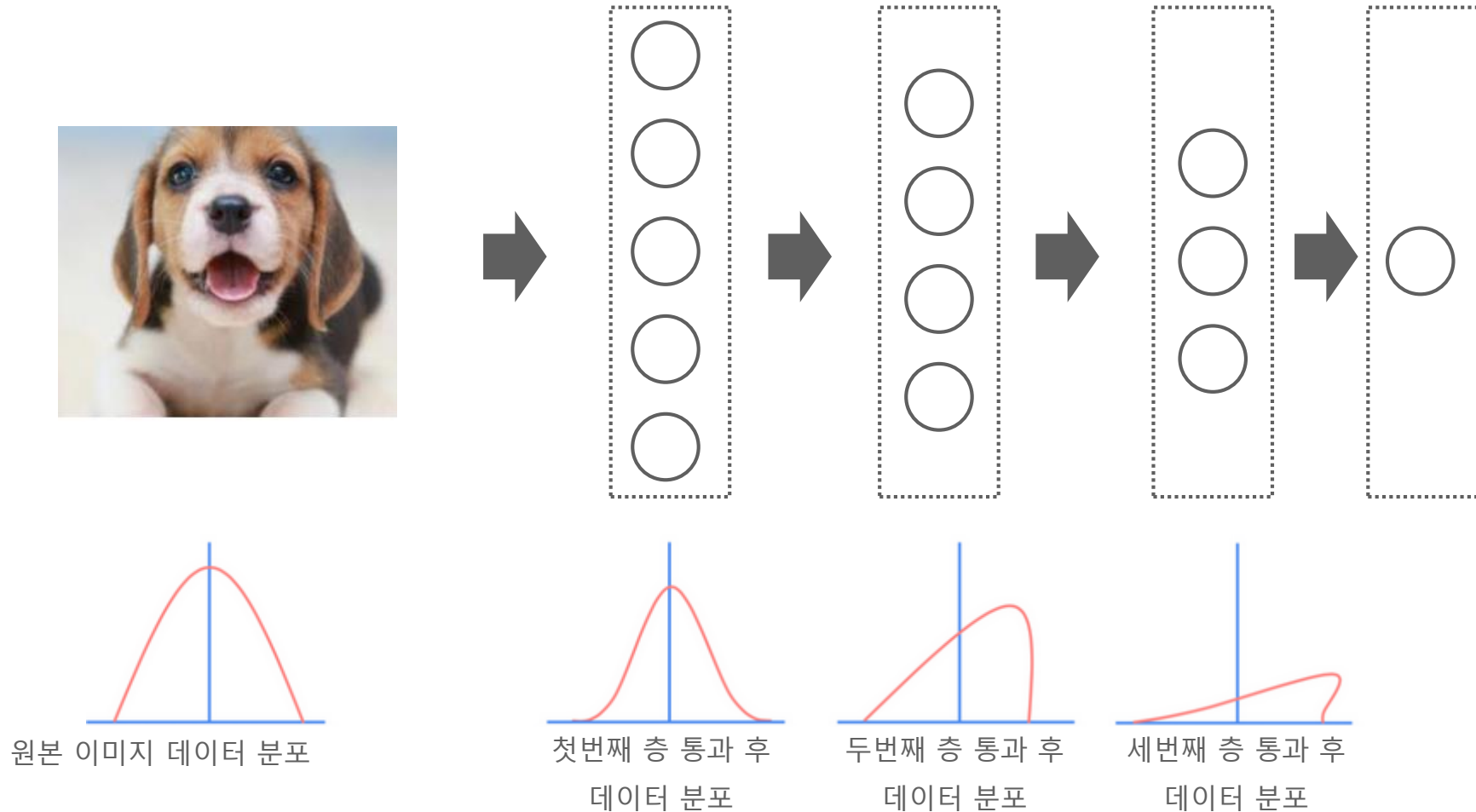


Residual Connection과 Layer Normalization



Batch Normalization의 필요성 - Internal Covariate Shift

신경망 내부의 각 층(Layer)를 통과 할 때마다 입력 데이터의 분포가 조금씩 변경되는 현상.



Batch Normalization 개요

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

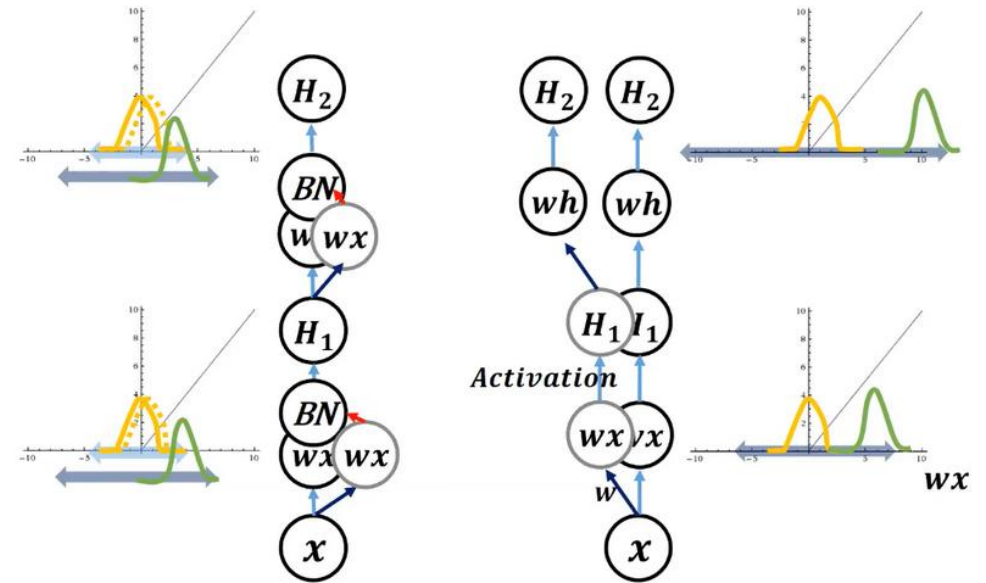
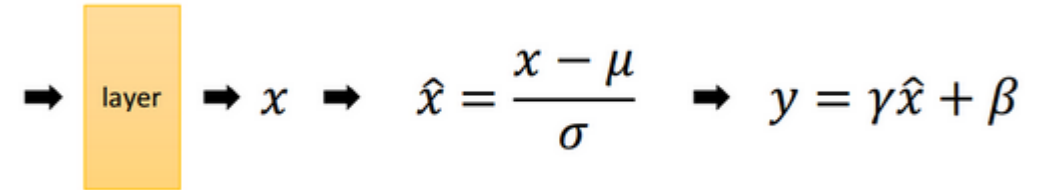
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

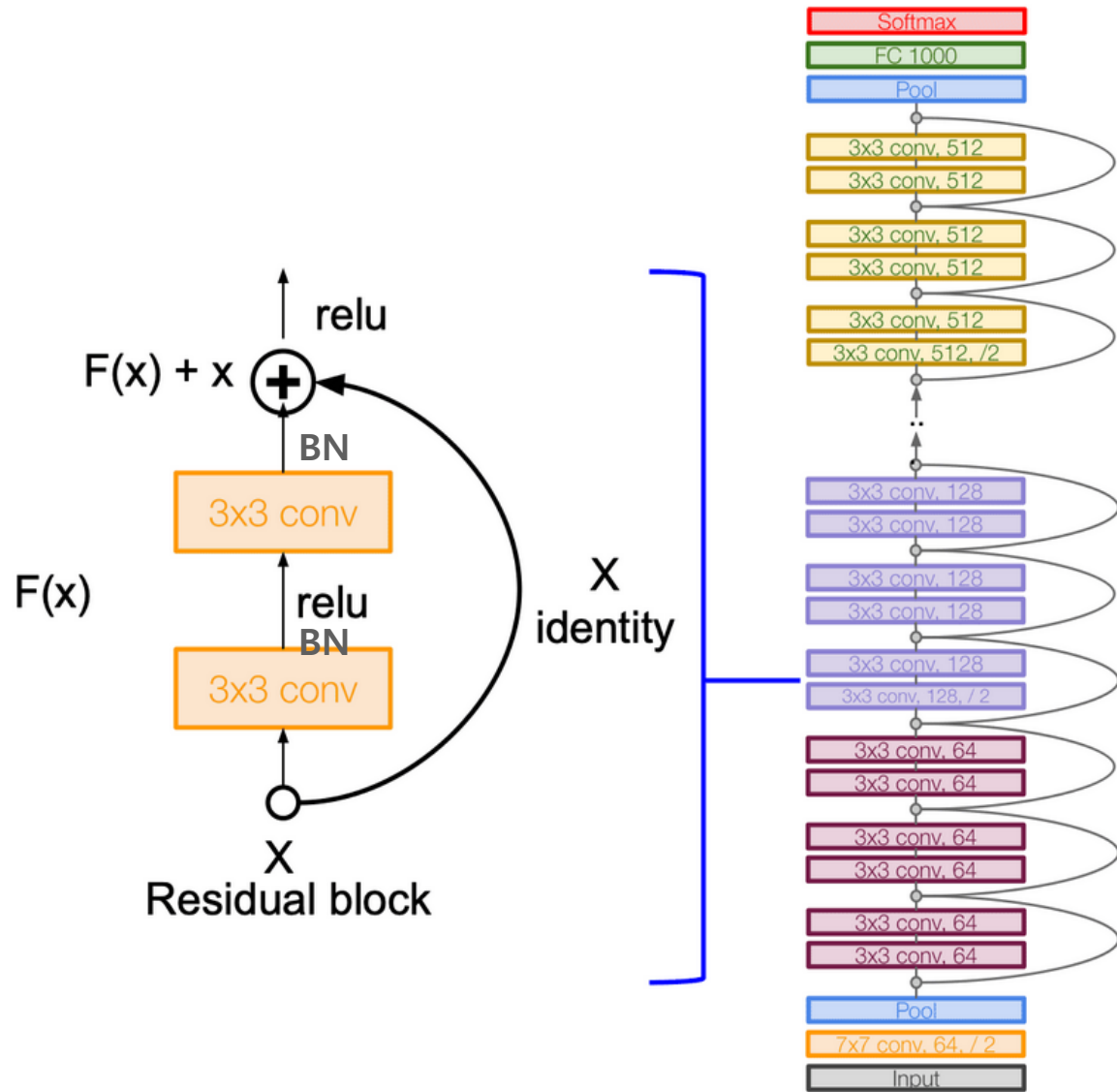
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

γ : 스케일링(Scaling) 파라미터

β : Shift 파라미터



CNN에서 Batch Normalization과 Residual Connection 구성

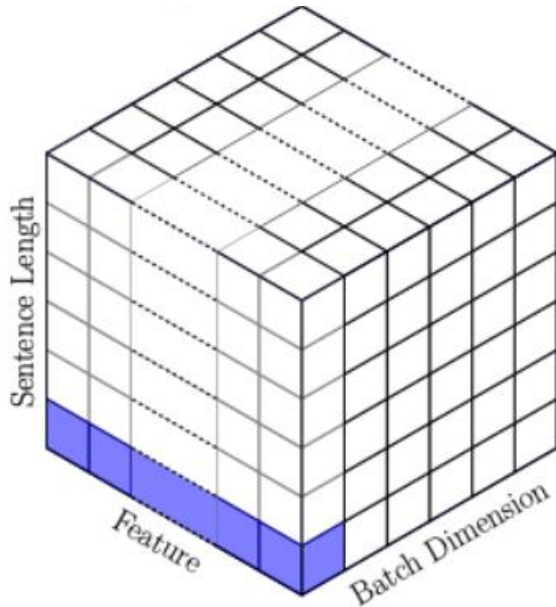


- CNN Network 구성 시 Batch Normalization은 Conv 연산 다음에 적용. 이후에 Activation 적용
- Conv $\rightarrow$ BN $\rightarrow$ Activation 으로 연속된 연결과 Residual Connection은 ResNet 이후에 CNN 모델의 주요 구성요소가 됨

Batch Normalization과 Layer Normalization

- Batch Normalization은 이미지 데이터 기반의 모델에서 성능향상이 검증됨(BatchNorm2d 적용)
- 이미지(Batch를 포함한 4차원) 데이터와는 다른, 언어 데이터(Batch를 포함한 3차원)의 특성을 감안한 정규화 방식 필요

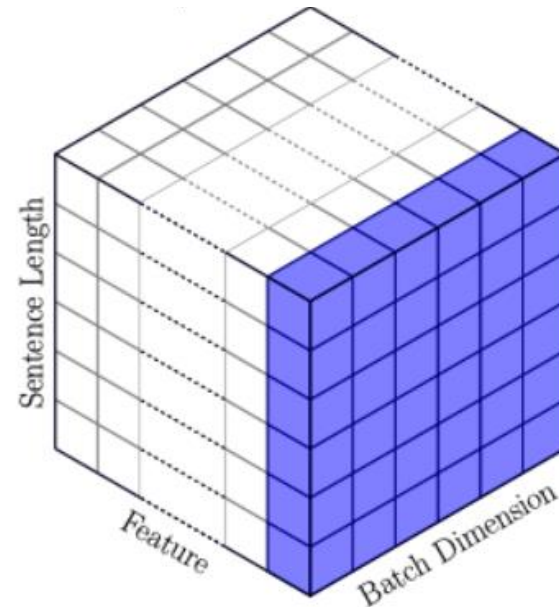
Layer Normalization



입력 Tensor가 3D 일 경우
(batch, sequence, feature)

Layer Normalization은 **개별 토큰 레벨**에 기반한 평균과 분산을 적용하여 개별 feature 값을 정규화

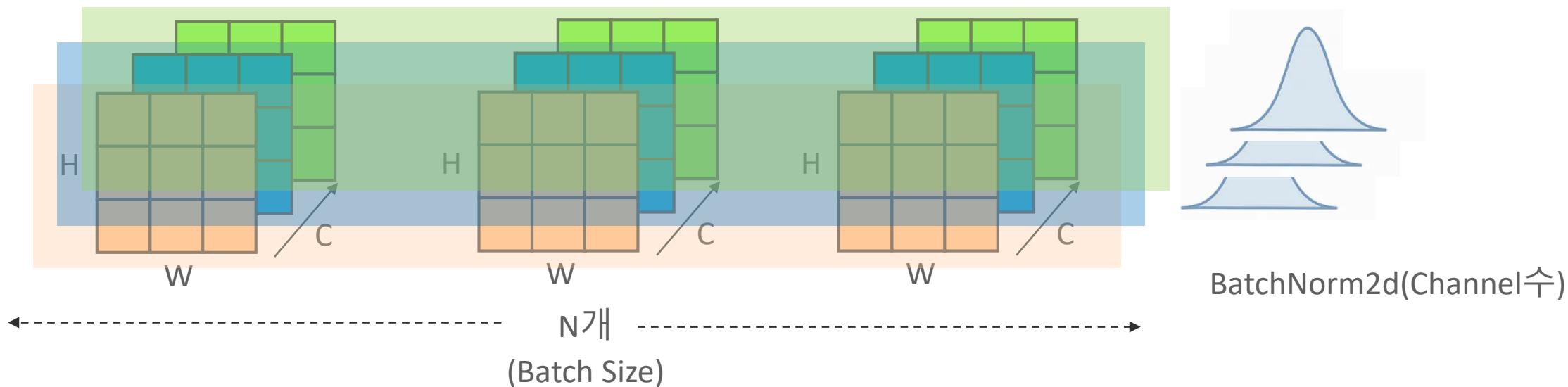
Batch Normalization(BatchNorm1d)



Batch + Sequence 걸쳐서 Feature의 개별 dim 레벨에 기반한 평균과 분산을 적용하여 개별 feature 값을 정규화

4차원 데이터에 Batch Normalization(BatchNorm2d) 적용

4차원 데이터(N, C, H, W)의 BatchNorm2d 적용 (개별 채널 별로 전체 평균, 분산을 구해서 정규화 적용)



평균과
분산 구하기

$$\mu_c = \frac{1}{N \times H \times W} \sum_{n=1}^N \sum_{h=1}^H \sum_{w=1}^W x_{n,c,h,w}$$

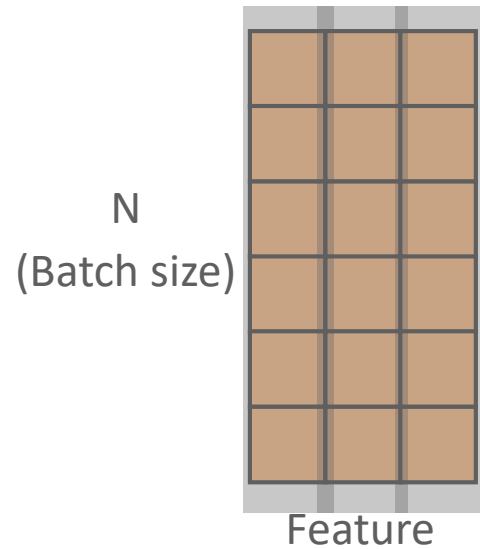
$$\sigma_c^2 = \frac{1}{N \times H \times W} \sum_{n=1}^N \sum_{h=1}^H \sum_{w=1}^W (x_{n,c,h,w} - \mu_c)^2$$

개별 값에
정규화 적용

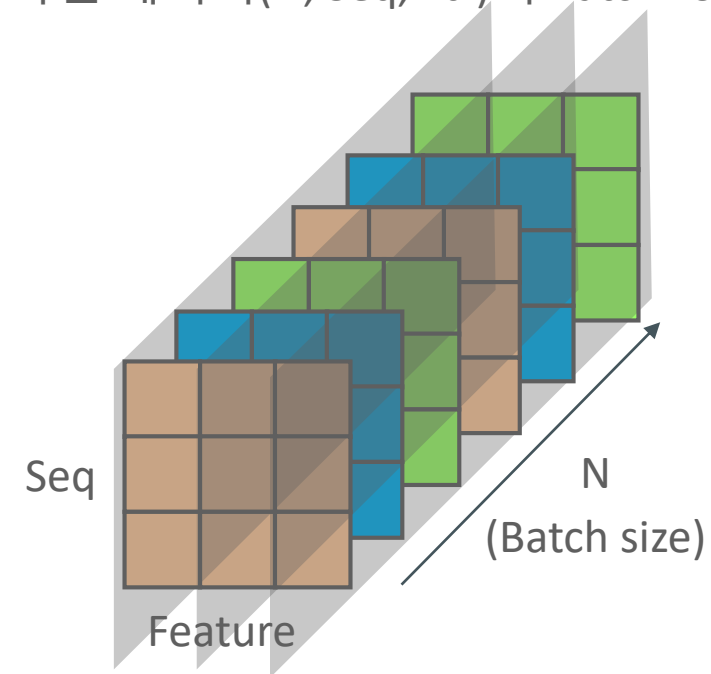
$$\hat{x}_{n,c,h,w} = \frac{x_{n,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}$$

2, 3차원 데이터에 Batch Normalization(BatchNorm1d) 적용

2차원 데이터(N, Ftr)의 BatchNorm1d 적용

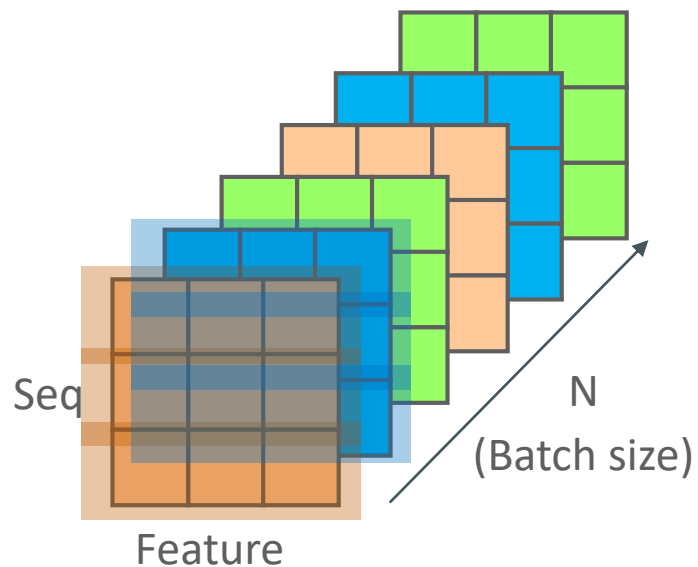


3차원 데이터(N, Seq, Ftr)의 BatchNorm1d 적용



3차원 NLP 데이터에 Layer Normalization 적용

3차원 NLP 데이터(N, Seq, Ftr)에 LayerNorm 적용



평균과 분산을 개별 token에 걸쳐서 (axis=-1) 구하여, 이를 기반으로 정규화 적용

평균과 분산 구하기

$$\mu_{i,j} = \frac{1}{D} \sum_{d=1}^D x_{i,j,d}$$

$$\sigma_{i,j}^2 = \frac{1}{D} \sum_{d=1}^D (x_{i,j,d} - \mu_{i,j})^2$$

D는 embed 크기,
i, j는 batch를 감안한 token index,
d는 embed dim index

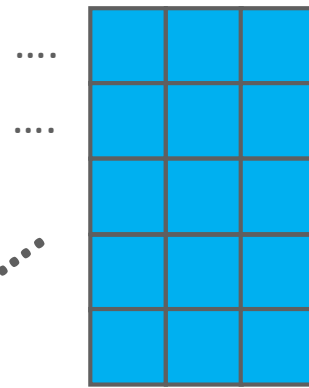
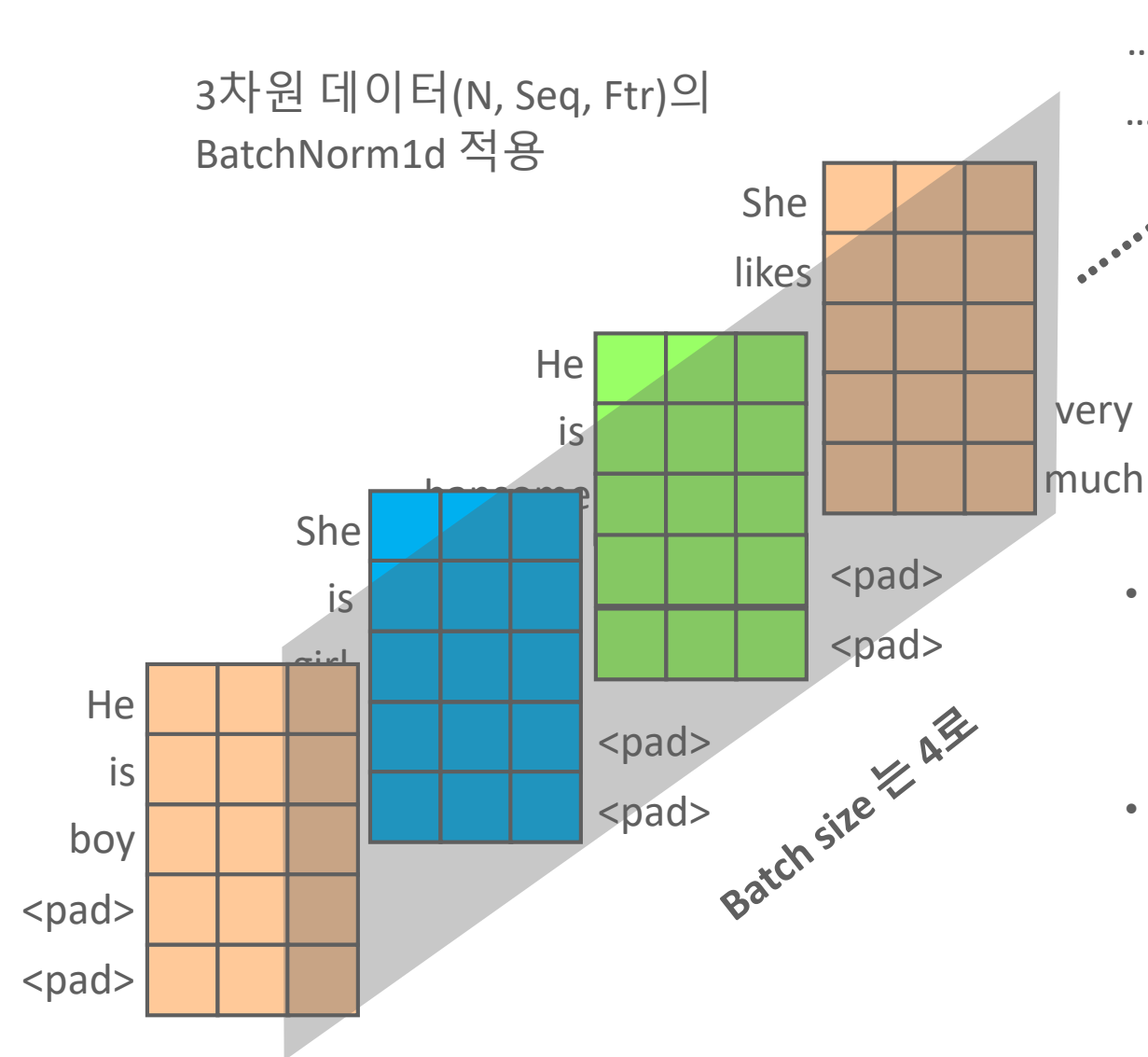
개별 값에 정규화 적용

$$\hat{x}_{i,j,d} = \frac{x_{i,j,d} - \mu_{i,j}}{\sqrt{\sigma_{i,j}^2 + \epsilon}}$$

개별 Token에 고유한 평균과 분산

언어 데이터에 Batch Normalization을 적용 시 이슈

3차원 데이터(N, Seq, Ftr)의
BatchNorm1d 적용

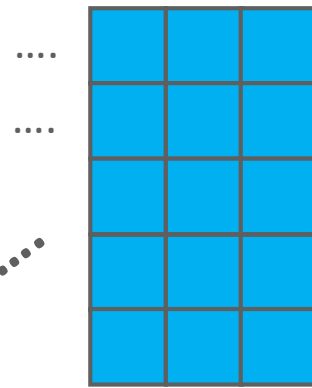
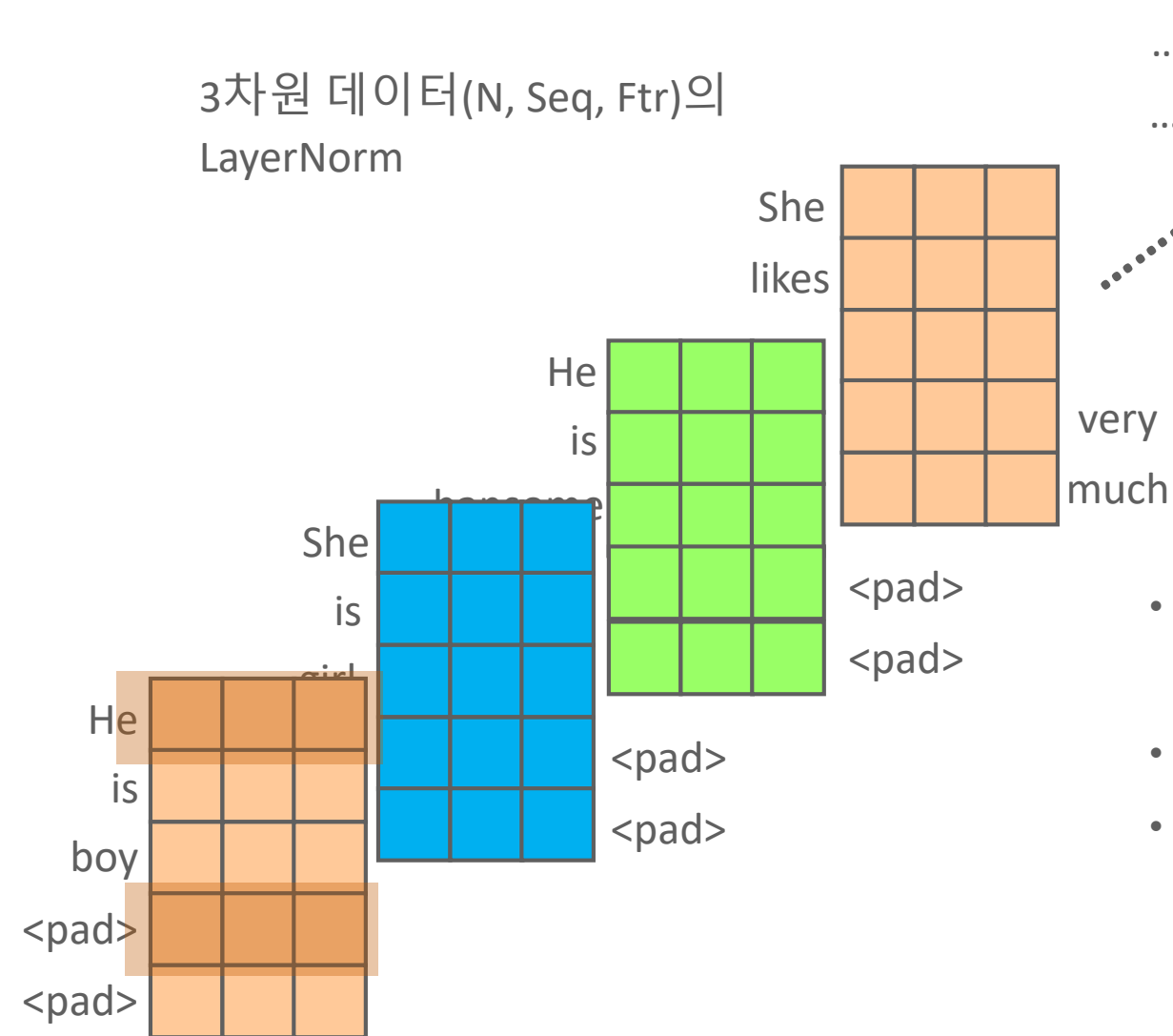


["He is (a) boy",
"She is (a) girl",
"He is handsome",
"She likes him very much"
.....
.....]

- 개별 문장은 서로 다른 크기로 구성되며 작은 문장의 경우 <pad> 값으로 token이 채워지는 개수가 많아짐. 동적으로 변하는 <pad> 개수를 포함한 평균/분산 값의 의미적 훼손
- Transformer는 메모리를 많이 소모하여 Batch Size를 일반적으로 작게 설정해야 함. 매우 많은 문장들에 대해서 작은 Batch Size 기반의 Sampling된 평균과 분산이 전체 데이터를 대표하기에 적절하지 않을 수 있음

언어 데이터에 Layer Normalization 적용

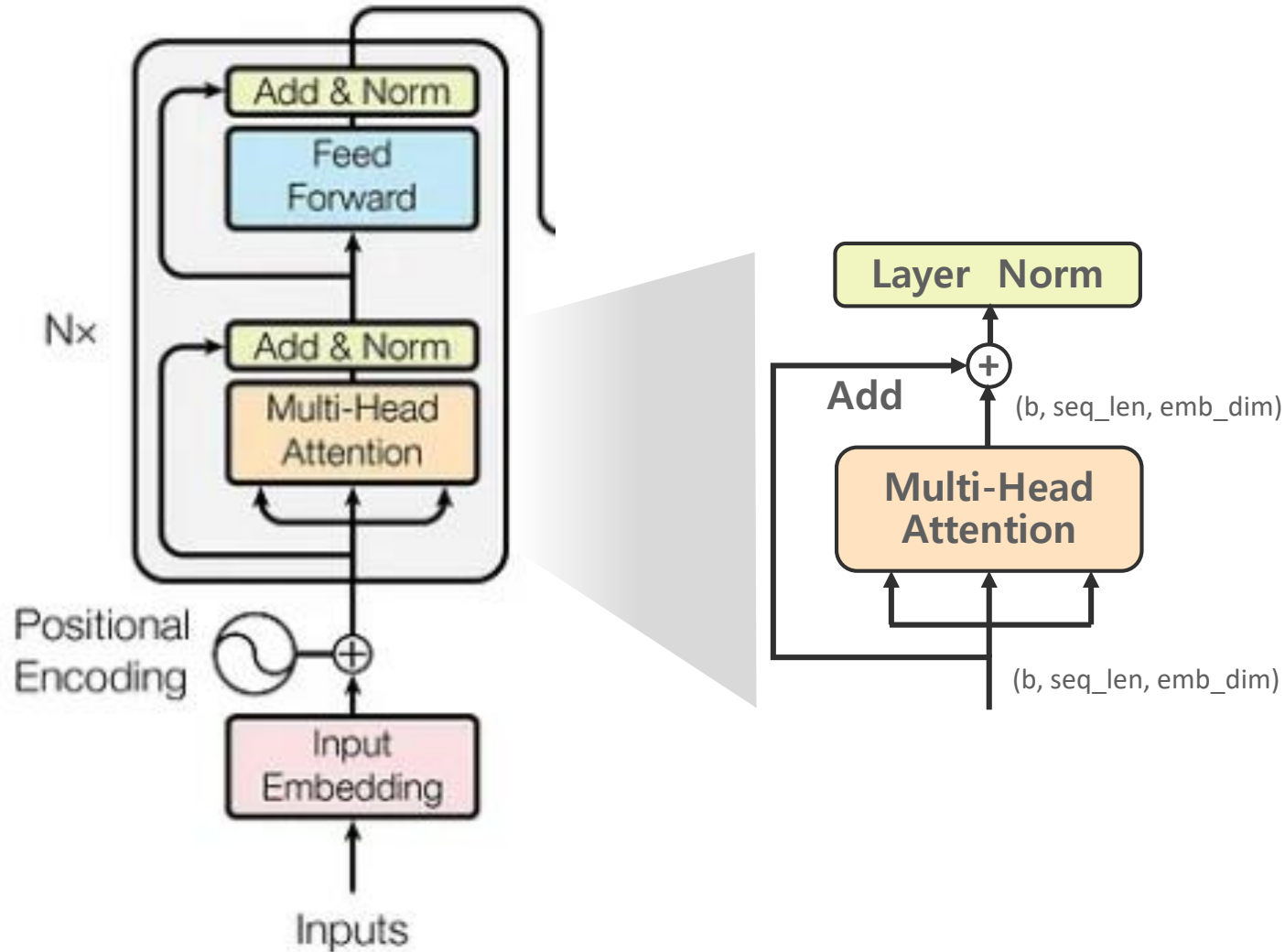
3차원 데이터(N, Seq, Ftr)의
LayerNorm



["He is (a) boy",
"She is (a) girl",
"He is handsome",
"She likes him very much"
.....
.....]

- Batch Size에 관계없이 개별 Token 레벨에서 구한 평균/분산을 기반으로 정규화 수행
- <pad>의 경우도 <pad> 토큰의 평균/분산 기반으로 정규화 수행
- Transformer 모델에서 보다 안정적으로 성능향상을 가져옴

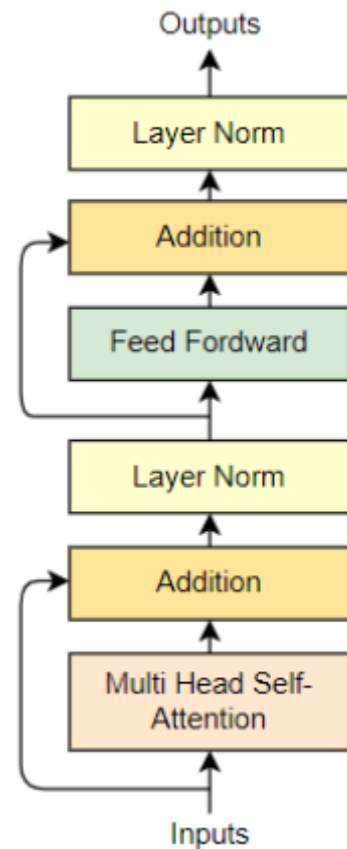
Residual Connection과 Layer Normalization



Residual Connection 및 Layer Normalization 적용

Post-Norm(Normalization)

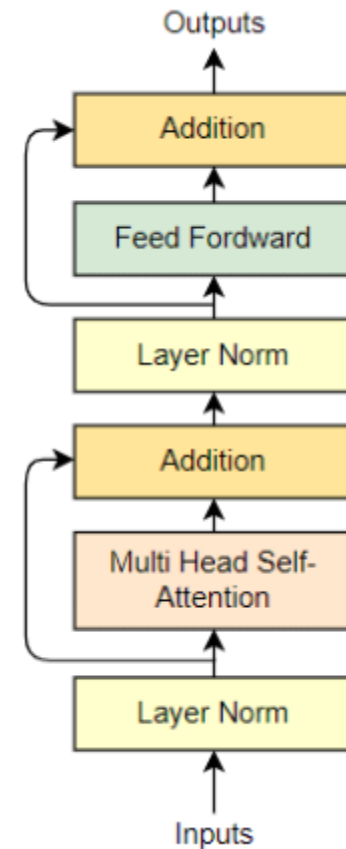
Add & Norm
Attention/Feed Forward **이후에**
residual connection 및 Layer
Norm 적용



Original Transformer, BERT

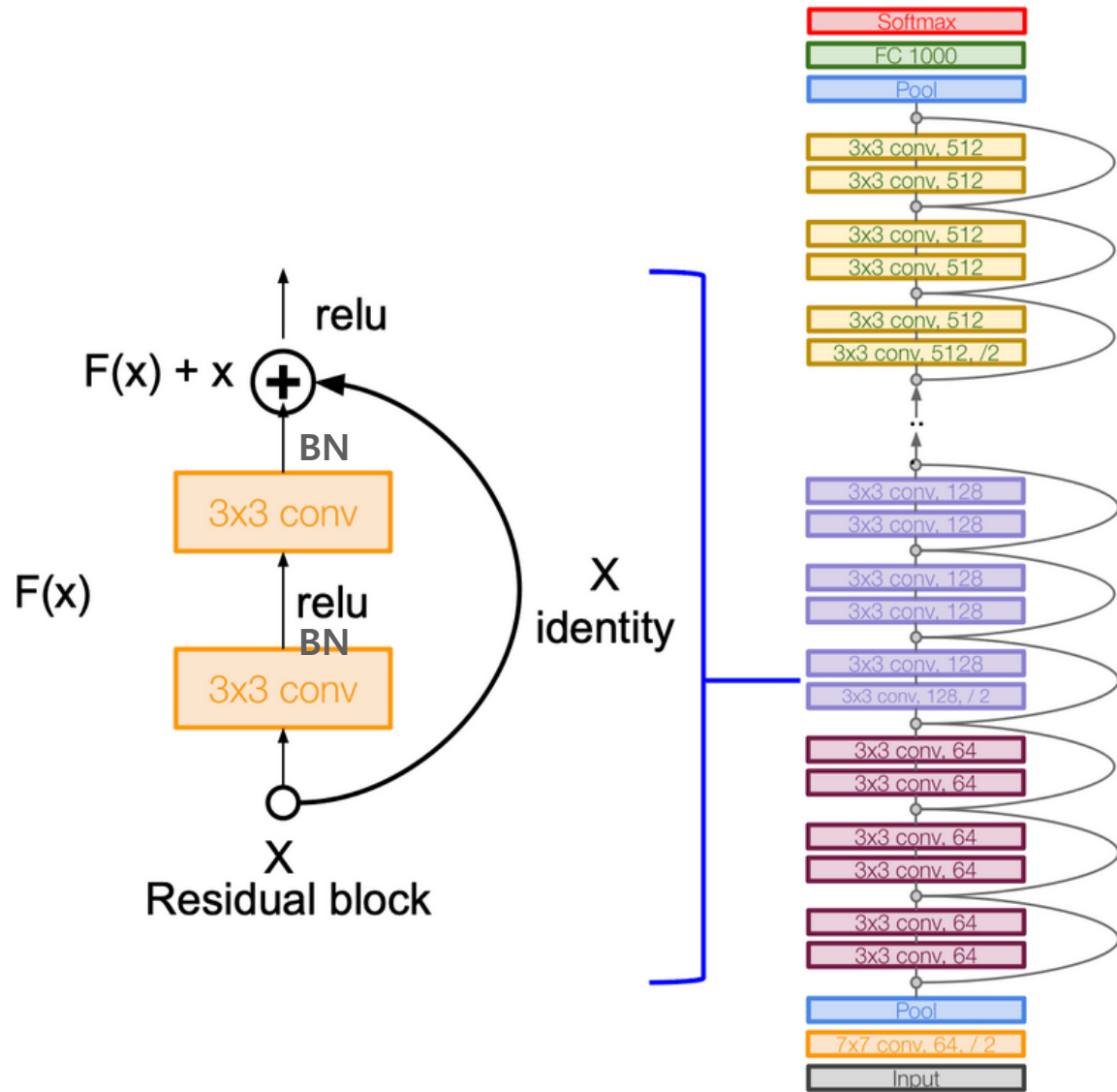
Pre-Norm(Normalization)

Norm & Add
Attention/Feed Forward **이전에**
Layer Norm을 적용 한 후
residual connection



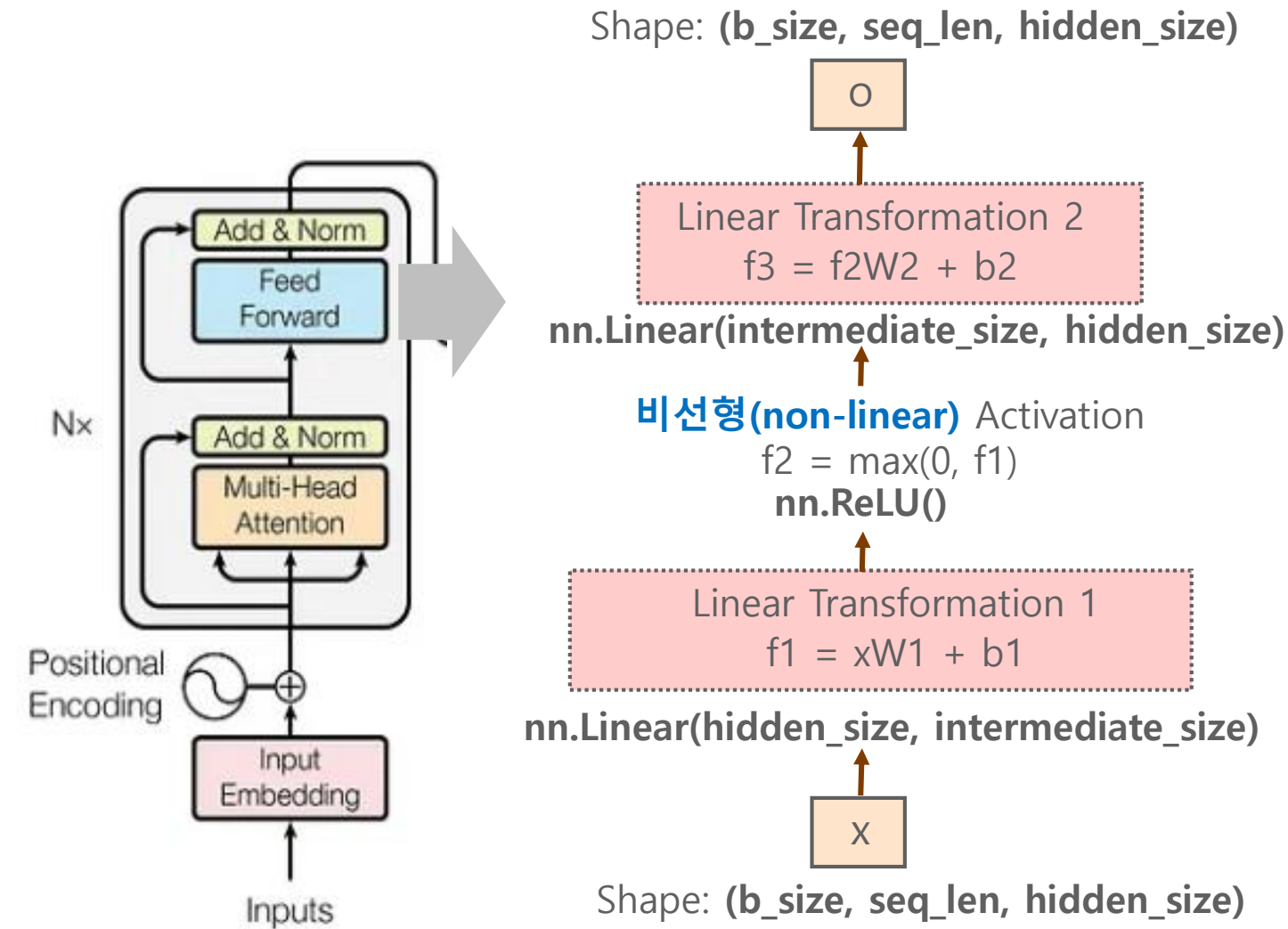
ViT, GPT등 LLM

CNN에서 Batch Normalization과 Residual Connection 구성



- CNN Network 구성 시 Batch Normalization은 Conv 연산 다음에 적용. 이후에 Activation 적용
- Conv $\rightarrow$ BN $\rightarrow$ Activation 으로 연속된 연결과 Residual Connection은 ResNet 이후에 CNN 모델의 주요 구성요소가 됨

Point wise Feed Forward 개요

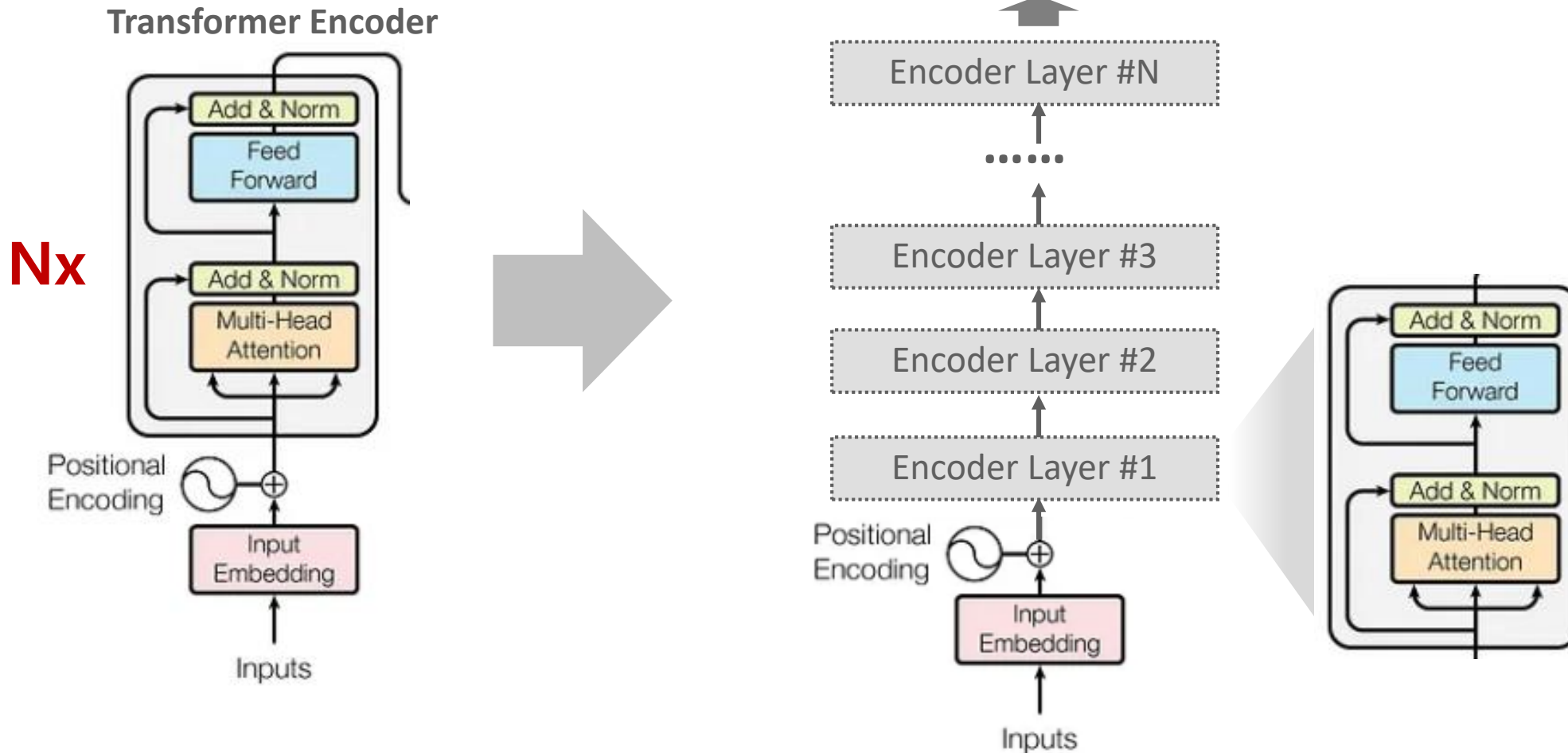


$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- ✓ Self Attention 내에서는 **비선형 변환이** (거의) 존재하지 않으며 Query, Key, Value의 선형 변환만이 존재.
- ✓ Softmax 함수는 비선형 함수이지만, 0 ~ 1사이 값 값 **변환 정규화** 용도로 사용
- ✓ Self Attention은 (나와) 타 토큰 간의 관계에 집중하여 결과 출력하며, 이렇게 출력된 값을 **개별 토큰 단위로 비선형성을** 적용하여 학습할 수 있도록 함
- ✓ FFN은 **2개의 선형 변환 Layer들과 그 사이에 Activation 함수로 구성**. 개별 선형 변환 Layer들은 출력 features 수가 다르게 구성되며, intermediate_size가 hidden_size보다 크게 구성(보통 4배)

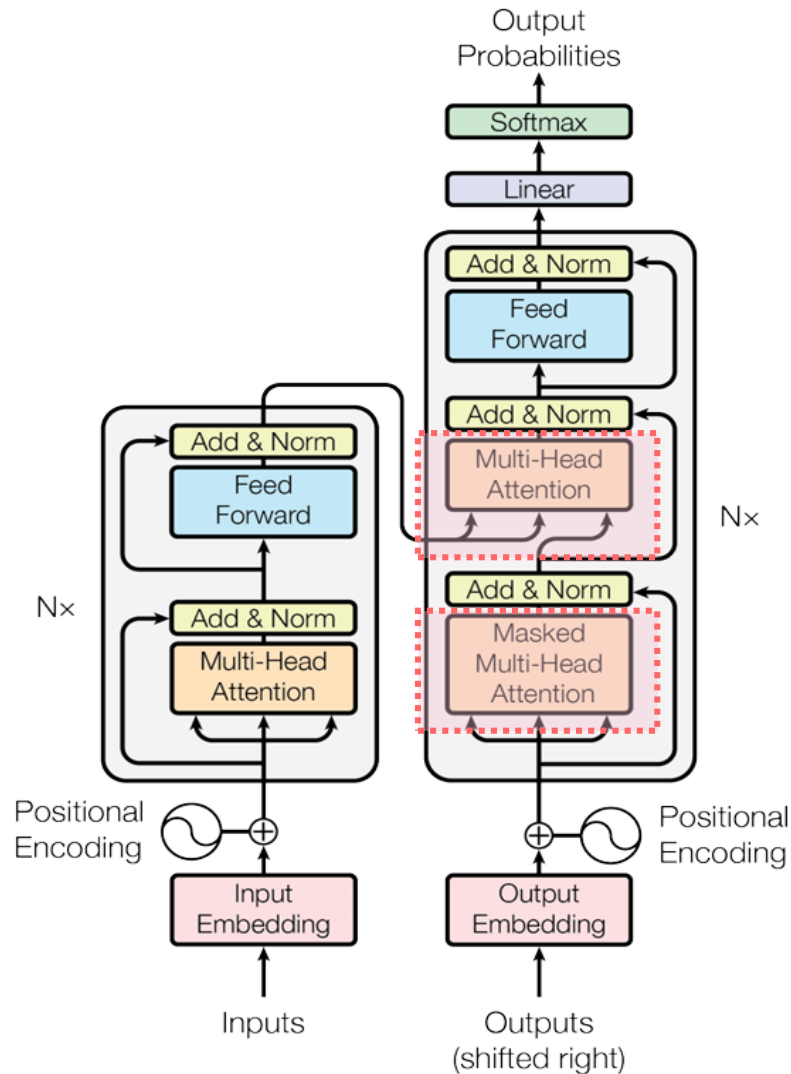
Transformer Encoder와 Encoder Layer

- Transformer **Encoder** 구조는 Multi-Head Attention -> Add & Norm -> Feed Forward -> Add & Norm으로 구성된 **Encoder Layer(Block)**들을 연이은 Stack 형태로 쌓아 올린 구조



Transformer – Decoder

Decoder 개요

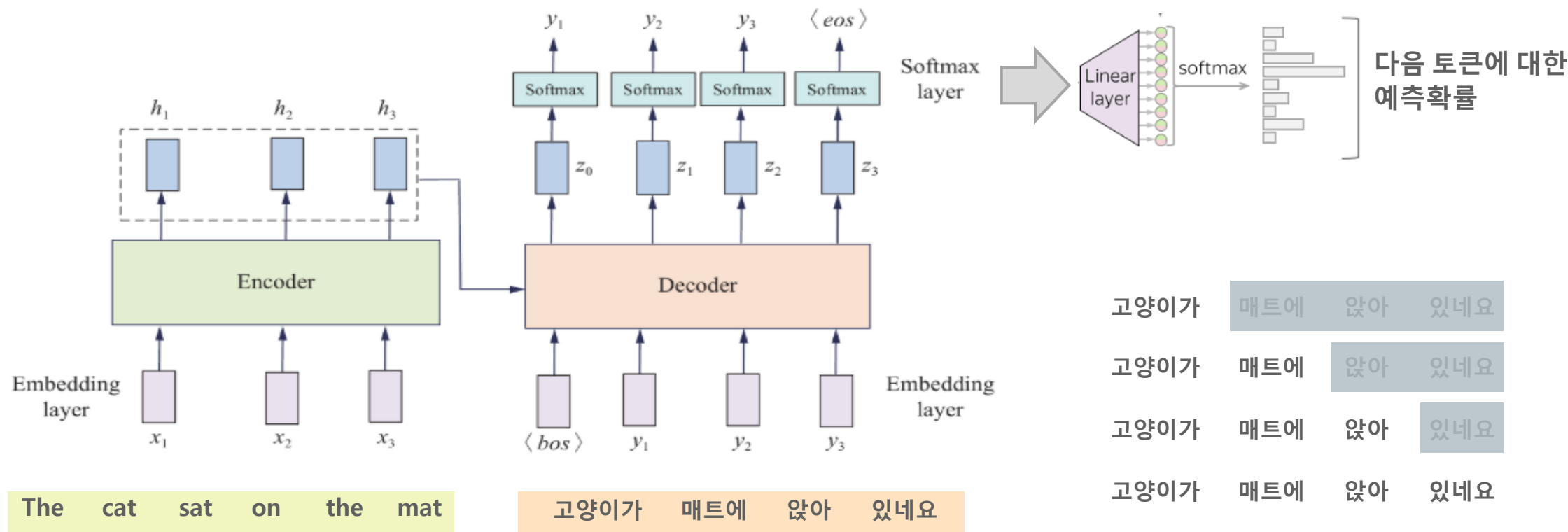


Seq2Seq 기반의 Transformer Decoder는 Encoder와 비슷한 구조로 되어 있지만, 2개의 주요한 차이가 있음

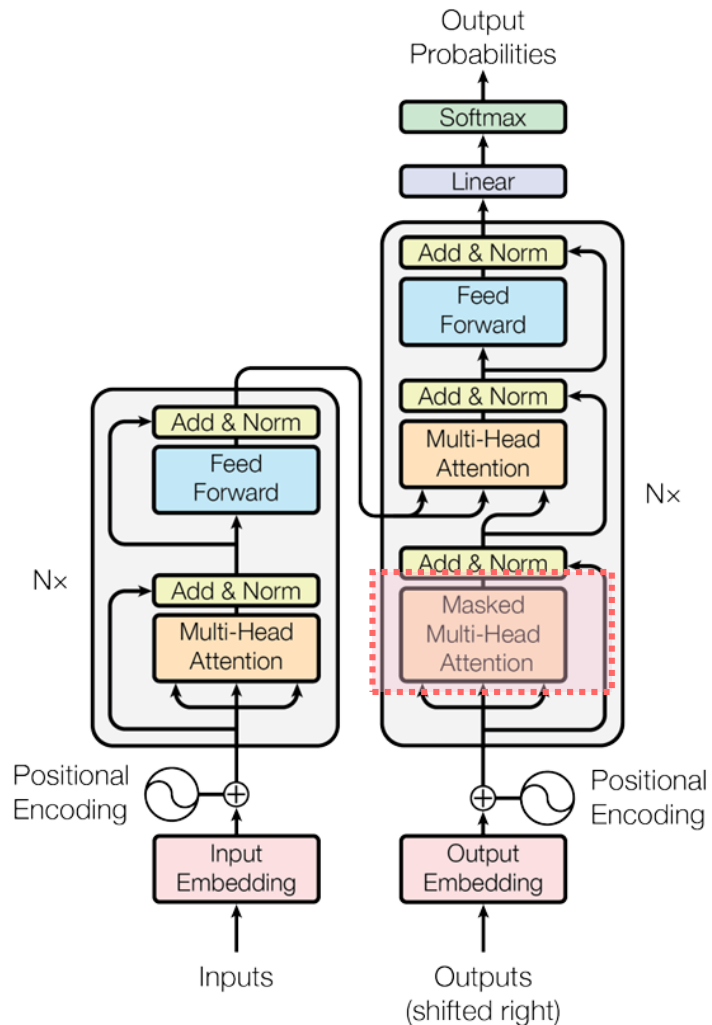
- Decoder의 Self Attention은 **Causal(Masked) Self Attention**임.
- Decoder는 Encoder의 최종 Context Vector를 Key와 Value로 Decoder의 Query를 기반으로 하는 **Cross Attention**

Encoder – Decoder 모델에서 Decoder의 주요 역할

Encoder-Decoder 모델에서 Decoder 의 주요 역할은 첫 Token부터 현재까지의 Token들을 기반으로 **다음에 올 수 있는 Token을 예측하는 것임**. 이를 위해 Encoder의 Context Vector와 Decoder 자체의 입력 Token들을 사용하여 모델을 학습 시킴.



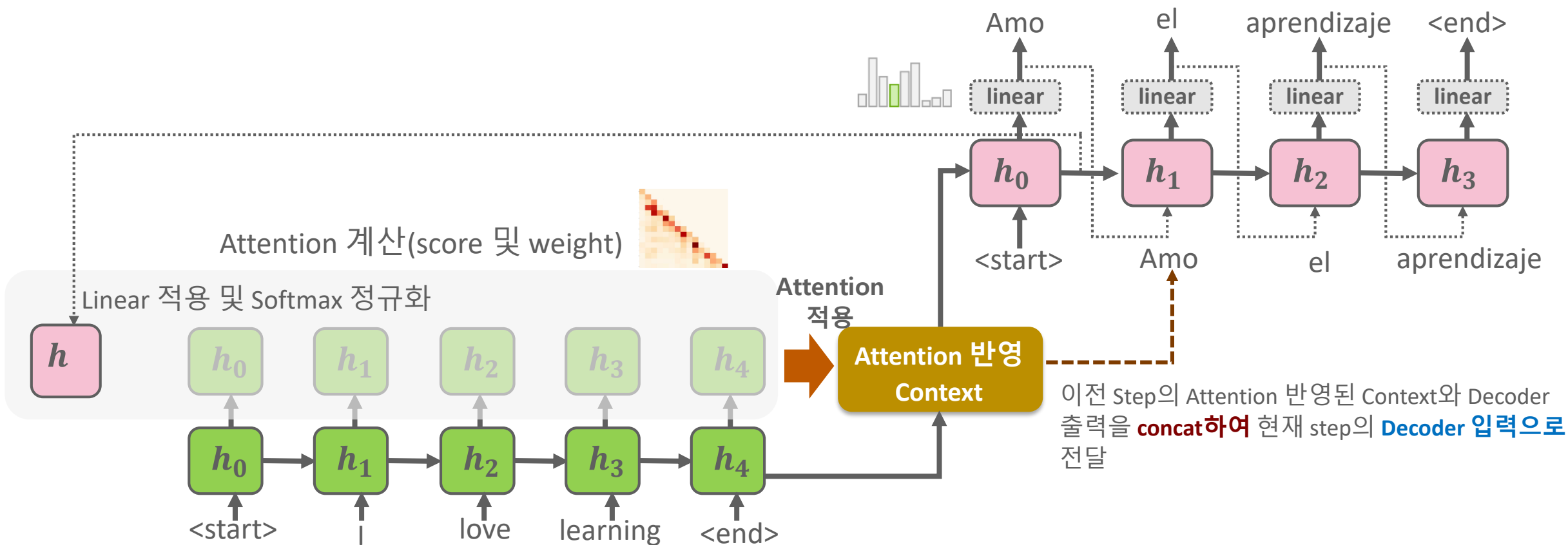
Decoder(Self Attention) 에서 Causal Attention의 필요성



- Decoder의 주요 역할은 첫 Token부터 현재까지의 Token들을 기반으로 **다음 Token을 예측하는 것임**
- 이를 위해 Decoder Model은 자신이 예측한 다음 Token과 실제 다음 Token(학습 데이터의 Target 값) 차이에 대해 Loss 함수를 적용하면서 Model 학습을 수행.
- 하지만 Encoder의 Self Attention과 같이 Decoder에서도 문장내 모든 Token들에 대해서 Self Attention을 적용하게 되면 Model에서 **Loss 적용을 하기 전에 미리 다음 Token에 대한 정보를 알게 되면서** 모델 성능이 떨어지게 됨(미리 다음 Token에 대한 힌트를 얻게 되어서, 모델 학습이 제대로 되지 않음)
- Transformer Decoder는 입력되는 (병렬 계산을 유지하면서도) 문장 Token들의 순서에 따라서 자신의 현재 기준 Token 이후의 Token들은 Attention Weights를 계산하지 않도록 Causal Attention을 적용함.

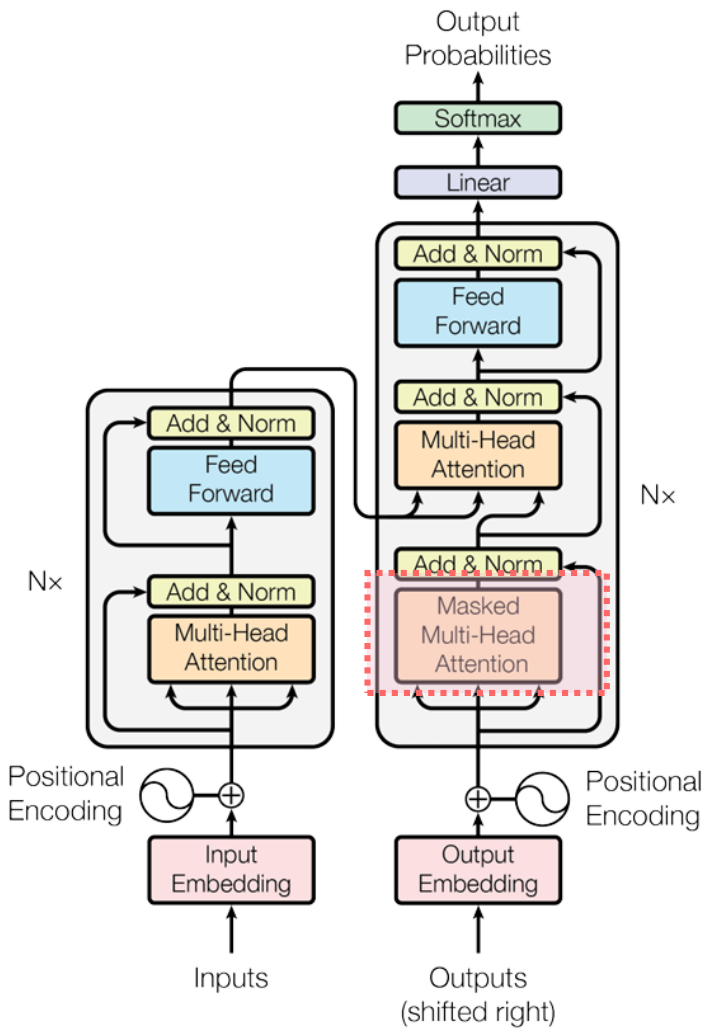
Seq2Seq with Attention 모델 개요 – RNN 기반

- **특정 시점의** Decoder Hidden State 별로 Encoder의 **모든 Hidden states들과** 얼마나 관련되어 있는 지 Attention을 계산
- 이를 위해 Decoder의 Hidden State가 생성 될 때마다 Encoder의 모든 Hidden states와 결합 후 Linear 적용 및 Softmax 정규화하여 Attention Score 및 Attention Weight 생성하며 **Context Vector는 이 Attention Weight를 반영하여 Dynamic 하게 재 생성됨**
- 새롭게 Attention Weight를 반영한 Context Vector는 Decoder 자체의 embedding 입력과 **결합**하여 다음 시점의 Decoder 입력이 됨.



Decoder 의 Causal Attention(Self Attention + Causal Masking)

Self Attention Scores에 Causal Masking 적용 하여 Attention Weight 계산



Attention Scores
(q_seq_len, k_seq_len) Shape

Key

Query

그 고양이가 저쪽 매트에 앉아 있네요

그	1.0	1.0	0.5	0.0	1.0	1.0
고양이가	1.0	2.0	1.5	1.0	1.0	1.5
저쪽	0.5	1.5	1.5	1.5	0.5	1.5
매트에	0.0	1.0	1.5	0.0	1.0	1.5
앉아	1.0	1.0	0.5	0.0	1.0	1.0
있네요	1.0	1.5	1.5	1.5	1.0	2.2

"Don't peek into the future"

Decoder에서는 학습 시 미래 Token 을 미리 들여다 보서는 안됨.

- 그 고양이가 저쪽 매트에 앉아 있네요
- 그 고양이가 저쪽 매트에 앉아 있네요
- 그 고양이가 저쪽 매트에 앉아 있네요
- 그 고양이가 저쪽 매트에 앉아 있네요
- 그 고양이가 저쪽 매트에 앉아 있네요
- 그 고양이가 저쪽 매트에 앉아 있네요

Decoder Self Attention에 Causal Masking 적용하기

Attention Scores는
(q_seq_len, k_seq_len) Shape

Key

	그	고양이가	저쪽	매트에	앉아	있네요
그	1.0	1.0	0.5	0.0	1.0	1.0
고양이가	1.0	2.0	1.5	1.0	1.0	1.5
저쪽	0.5	1.5	1.5	1.5	0.5	1.5
매트에	0.0	1.0	1.5	0.0	1.0	1.5
앉아	1.0	1.0	0.5	0.0	1.0	1.0
있네요	1.0	1.5	1.5	1.5	1.0	2.2

Causal mask
(q_seq_len, k_seq_len) Shape

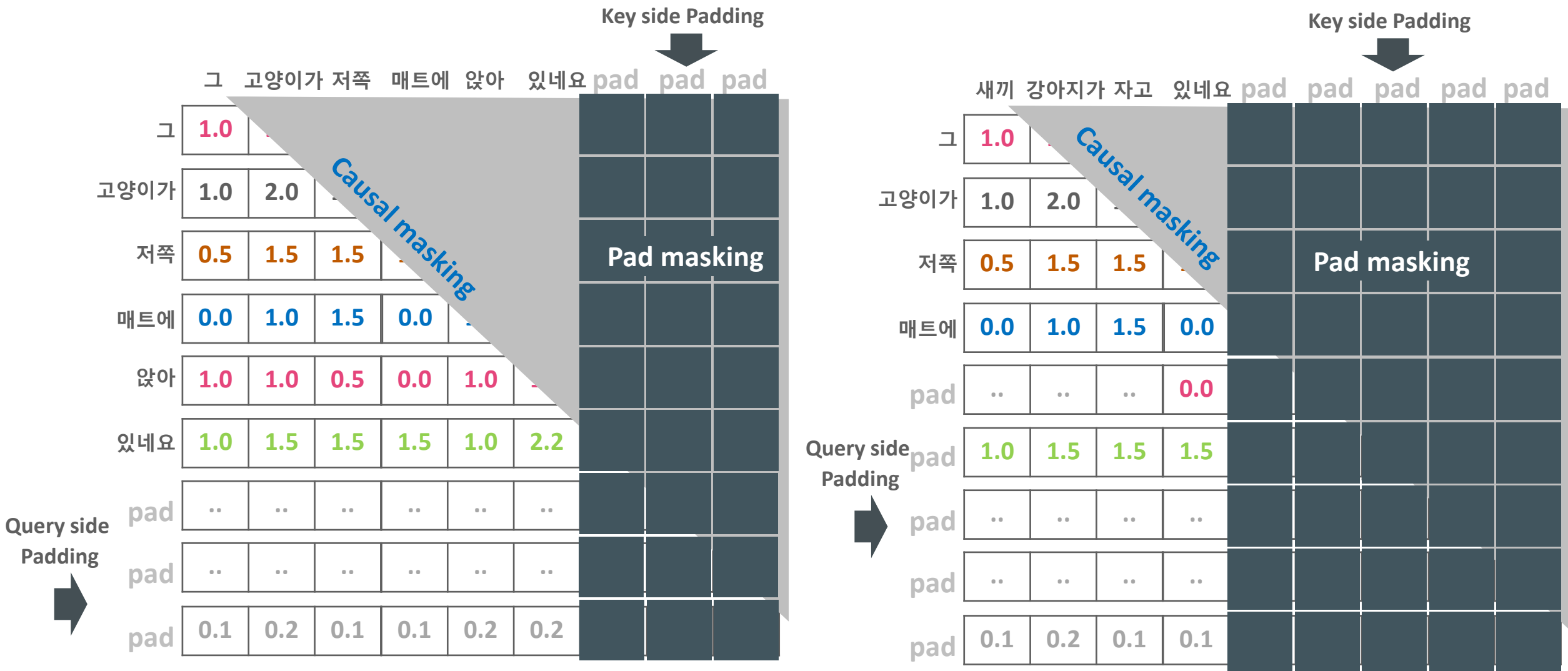
1	0	0	0	0	0
1	1	0	0	0	0
1	1	1	0	0	0
1	1	1	1	0	0
1	1	1	1	1	0
1	1	1	1	1	1

Causal Attention Scores는
(q_seq_len, k_seq_len) Shape

Key

	그	고양이가	저쪽	매트에	앉아	있네요
그	1.0	-inf	-inf	-inf	-inf	-inf
고양이가	1.0	2.0	-inf	-inf	-inf	-inf
저쪽	0.5	1.5	1.5	-inf	-inf	-inf
매트에	0.0	1.0	1.5	0.0	-inf	-inf
앉아	1.0	1.0	0.5	0.0	1.0	-inf
있네요	1.0	1.5	1.5	1.5	1.0	2.2

Pad Masking + Causal Masking 결합하여 Masking 적용 - 01



Pad Masking + Causal Masking 결합하여 Masking 적용 - 02

Causal Masking

1	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0
1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	0	0
1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1

Bitwise AND 연산

&

Pad Masking

1	1	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---

Pad Masking + Causal Masking 결합하여 Masking 적용 - 02

Pad Masking

2개의 문장(batch_size=2)들에 대한 Masking 정보

	1	1	1	1	0	0	0	0	0
1	1	1	1	1	1	0	0	0	

(Batch_size, Seq_len) 2차원 tensor를 (Batch_size, 1, 1, seq_len) 4차원으로 변환

Bitwise AND 연산

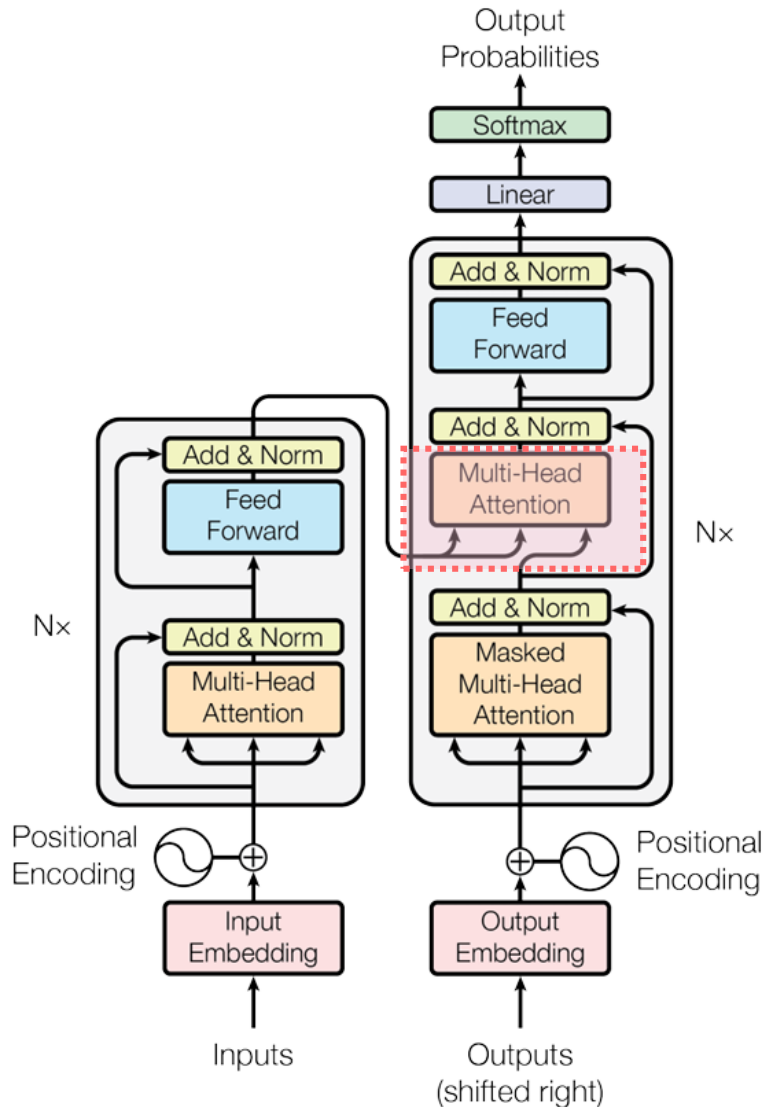


Causal Masking

1	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0
1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	0	0
1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1

(seq_len, seq_len) 2차원 tensor를 (1, 1, seq_len, seq_len) 4차원으로 변환

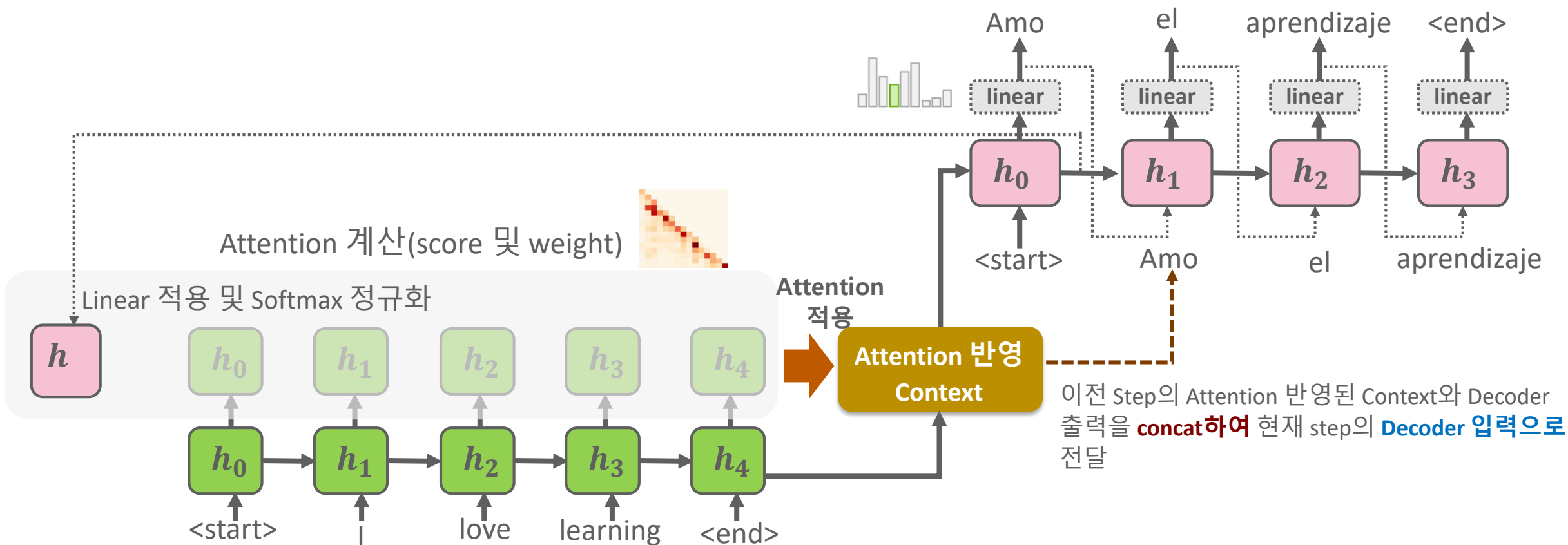
Decoder의 Cross Attention 개요



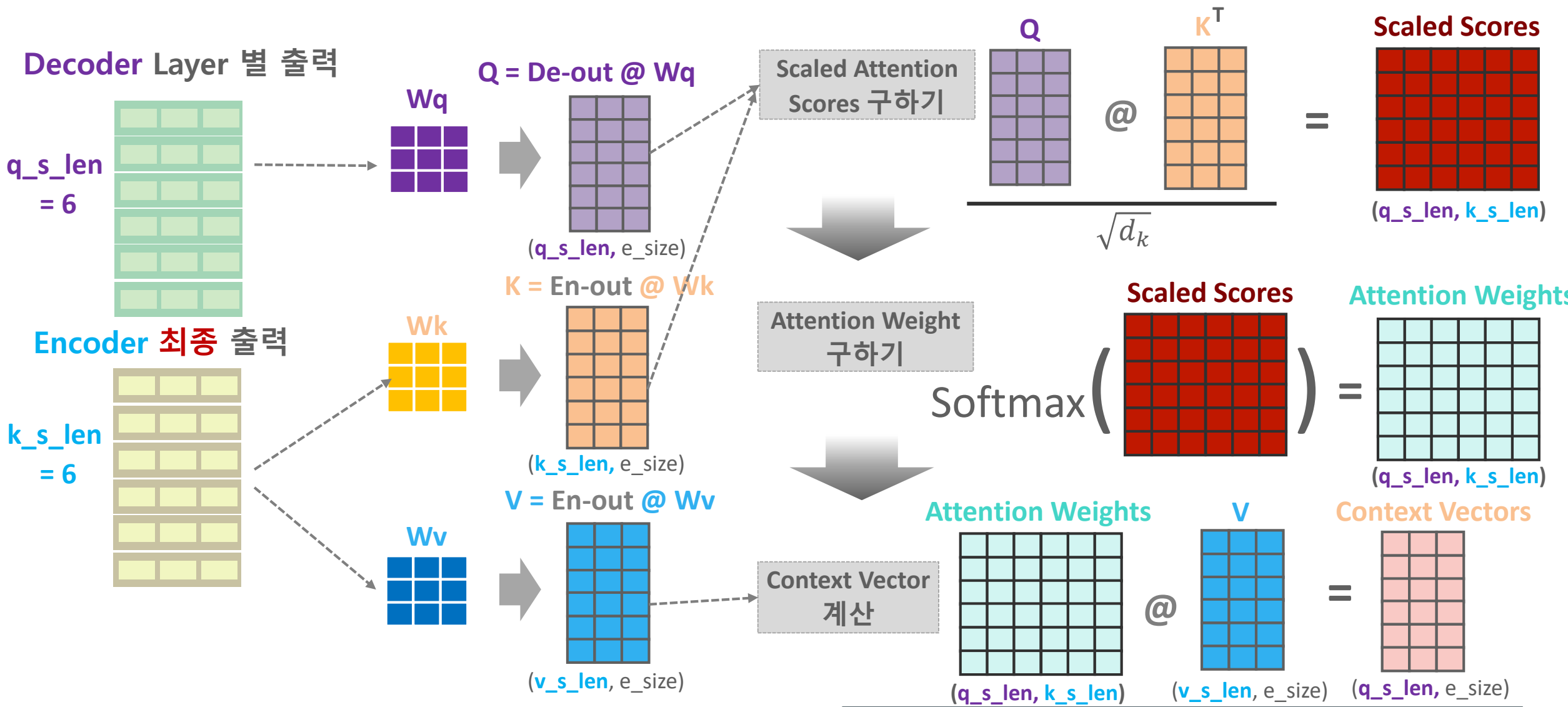
- **Self-attention**은 Query, Key, Value 가 모두 **동일한** Sequence 데이터를 기반으로 생성하지만, **Cross-attention**은 Decoder의 Query와 , Encoder의 Key와 Value를 기반으로 Attention을 생성
- **Cross Attention**은 Encoder-Decoder(BART, T5) 구조에서만 사용. Encoder 전용의 모델(BERT)나 Decoder 전용의 모델(GPT등의 LLM)에서는 사용되지 **않음**

Seq2Seq with Attention 모델 개요 – RNN 기반

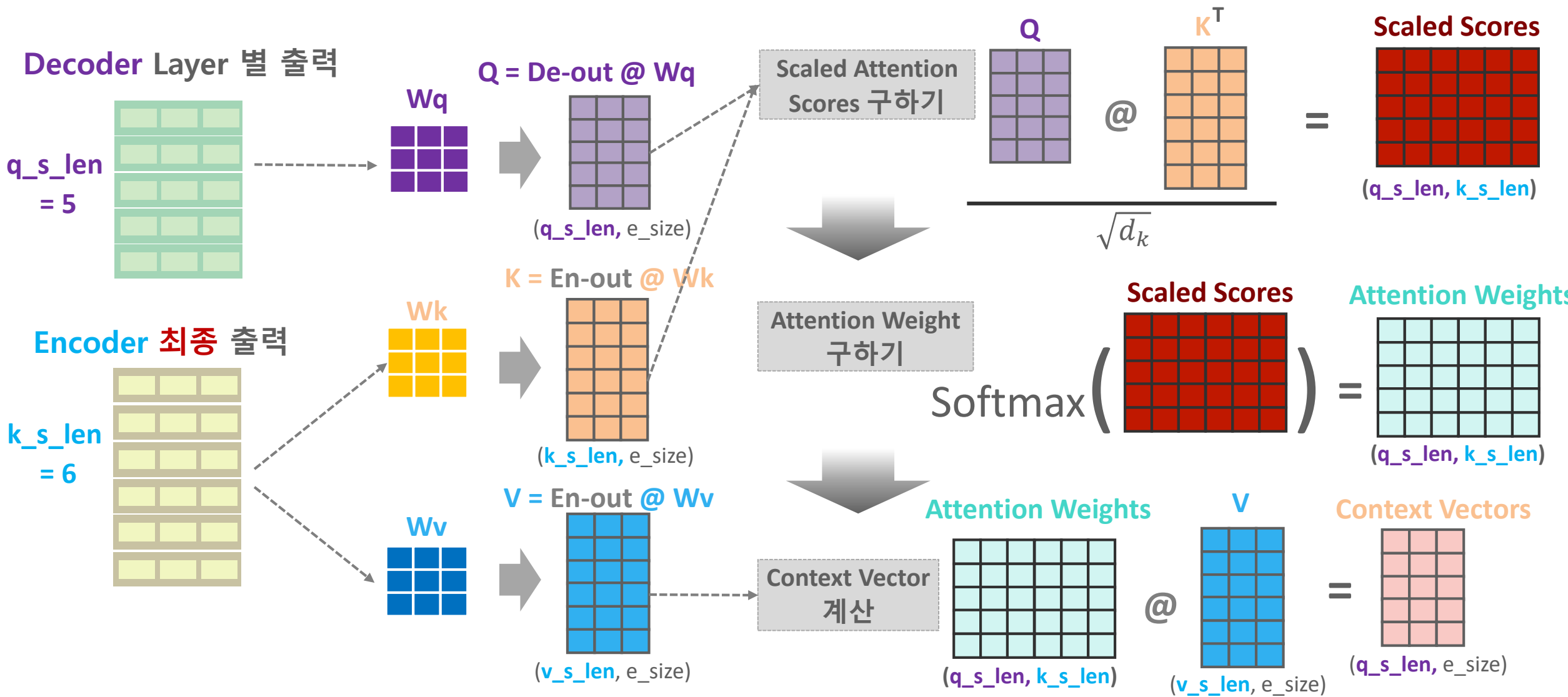
- **특정 시점의** Decoder Hidden State 별로 Encoder의 **모든 Hidden states들과** 얼마나 관련되어 있는 지 Attention을 계산
- 이를 위해 Decoder의 Hidden State가 생성 될 때마다 Encoder의 모든 Hidden states와 결합 후 Linear 적용 및 Softmax 정규화하여 Attention Score 및 Attention Weight 생성하며 **Context Vector는 이 Attention Weight를 반영하여 Dynamic 하게 재 생성됨**
- 새롭게 Attention Weight를 반영한 Context Vector는 Decoder 자체의 embedding 입력과 **결합**하여 다음 시점의 Decoder 입력이 됨.



Cross Attention 적용 – Query 와 Key의 Seq 길이가 같은 경우



Cross Attention 적용 – Query 와 Key의 Seq 길이가 다른 경우



Cross Attention에서 masking 적용하기

Cross Attention은 Query와 Key의 Length가 다를 수 있음. 이 경우 Attention score는 정방행렬이 되지 않음

		Key						Key side Padding		
		The	cat	sat	on	the	mat	pad	pad	pad
Query	고양이가	1.0	1.0	0.5	0.0	1.0	1.0	-inf	-inf	-inf
	매트에	1.0	2.0	1.5	1.0	1.0	1.5	-inf	-inf	-inf
	앉아	0.5	1.5	1.5	1.5	0.5	1.5	-inf	-inf	-inf
	있네요	0.0	1.0	1.5	0.0	1.0	1.5	-inf	-inf	-inf
	pad	..	..	..	..	..	..	-inf	-inf	-inf
	pad	..	..	..	..	..	..	-inf	-inf	-inf

Cross Attention Score Shape:
(batch_size, num_heads, query_seq_len, key_seq_len)

- Cross Attention의 Padding Key 쪽(Column 쪽)의 Padding만 적용. Query 쪽의 Padding은 별도로 하지 않는 이유는 Query에서 추가된 Padding은 train 시 loss에서 아예 제외되는 등 모델 학습시에 사용되지 않기 때문임
- Decoder의 Cross Attention에서는 Padding Mask만 적용하며 Causal Mask는 적용할 필요 없음

Encoder와 Decoder Attention에서 Masking 적용 방식

구분	Attention score shape (Batch size와 Heads 개수 제외)	Key padding mask	Causal maks
Encoder Self Attention	Query와 Key의 Shape 동일 (Encoder Query 길이, Encoder Key 길이)	Encoder Key 기반	없음
Decoder Causal Attention	Query와 Key의 Shape 동일 (Decoder Query 길이, Decoder Key 길이)	Decoder Key 기반	Causal
Decoder Cross Attention	Query와 Key의 Shape가 동일하지 않을 수 있음 (Decoder Query 길이, Encoder Key 길이)	Encoder Key 기반	없음

Hugging Face Dataset 다루기

HuggingFace Datasets



Datasets

- 간단한 명령어로 다운로드 가능한 수십만개의 공용 데이터 세트

```
raw_datasets = load_dataset("rajpurkar/squad")
```

- 다양한 데이터 전처리 기능 지원

다운로드 된 다양한 데이터 세트들을 쉽게 Preprocessing 할 수 있는 다양한 기능 지원.

```
processed_dataset = raw_dataset.map(process_example)
```

- 딥러닝 모델의 Training Process에 통합되어 모델 학습과 평가를 편리하게 수행 할 수 있도록 지원

HuggingFace Dataset 개요



Based on



- Huggingface Dataset Hub에서 `load_dataset()` 함수로 다운로드된 Dataset은 **Apache Arrow 기반 format**을 가짐.
- Apache Arrow는 대량 데이터의 RAM Memory 제약을 개선한 Memory mapped file 구조와 대량 데이터를 컬럼 기반으로 빠르게 접근 할 수 있는 Columnar 기반 포맷 특성을 가짐.
- Dataset의 메타 정보를 통해 주요 feature명, 컬럼명을 조회할 수 있고, indexing을 통해 특정 영역의 데이터를 접근할 수 있음
- Dataset내의 데이터를 변환할 수 있는 **map() 등의 메소드를 제공하며** NLP에서는 데이터를 **미리 토큰화 변환 작업을** 수행하여 모델에 token 데이터를 입력하여 학습하는 시간을 절감 시킴

Apache Arrow 개요

Column 기반의 저장 방식 및
Memory mapped 파일 기반

	session_id	timestamp	source_ip
Row 1	1331246660	3/8/2012 2:44PM	99.155.155.225
Row 2	1331246351	3/8/2012 2:38PM	65.87.165.114
Row 3	1331244570	3/8/2012 2:09PM	71.10.106.181
Row 4	1331261196	3/8/2012 6:46PM	76.102.156.138

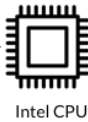
Traditional Memory Buffer

Row 1	1331246660	3/8/2012 2:44PM	99.155.155.225
Row 2	1331246351	3/8/2012 2:38PM	65.87.165.114
Row 3	1331244570	3/8/2012 2:09PM	71.10.106.181
Row 4	1331261196	3/8/2012 6:46PM	76.102.156.138

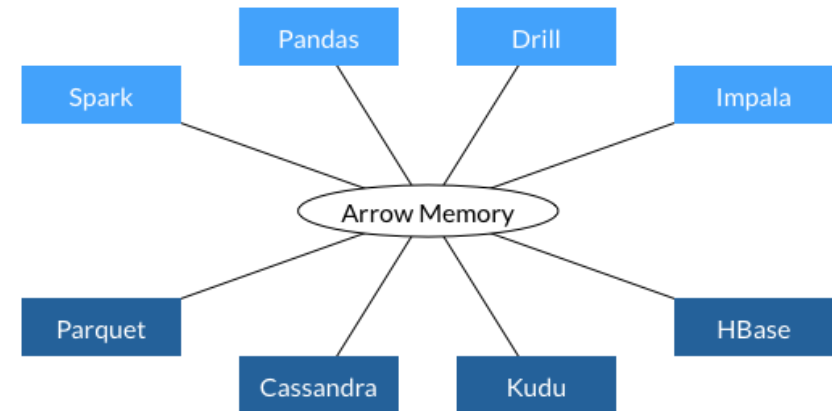
Arrow Memory Buffer

session_id	1331246660
session_id	1331246351
session_id	1331244570
session_id	1331261196
timestamp	3/8/2012 2:44PM
timestamp	3/8/2012 2:38PM
timestamp	3/8/2012 2:09PM
timestamp	3/8/2012 6:46PM
source_ip	99.155.155.225
source_ip	65.87.165.114
source_ip	71.10.106.181
source_ip	76.102.156.138

SELECT * FROM clickstream
WHERE session_id = 1331246351



다양한 데이터 처리 Engine의 통합 데이터 포맷 지향



Hugging Face Tokenizer와 DataCollator 다루기

Huggingface Tokenizer의 주요 프로세스



문장 분할에 따른 토큰화 유형

Types Of Tokenization

“Machine learning is fun”

"Machine",
"learning",
"is", "fun", "."

Word-Based

"ma",
"chine",
"learn", "ing"

Subword-Based

"M", "a", "c",
"h", "i", "n",
"e", "l", "e",
"a", "r", "n",
"i", "n", "g"

Character-Based

문장 분할에 따른 토큰화 유형

"Machine",
"learning",
"is", "fun", "."

Word 기반

- 간단하게 토큰화 작업을 수행
- 어근 변화 단어들을 별도로 토큰 관리 해야 하므로(happy, happier, happiest, happiness), (play, played, playing, plays)
매우 큰 단어 사전이 필요하게 됨(수십~수백만개 단어등록이 필요)
- 신조어등의 경우는 학습 단어 사전에서 처리하지 못해서 **OOV**(Out of vocabulary) 문제가 발생하기 쉬움

"ma",
"chine",
"learn", "ing"

Subword 기반

- 현대 NLP에서 **널리 사용되는** 방식으로 **자주 등장**하는 문자 조합을 하나의 단위로 묶고, **희귀 단어는** 작은 조각으로 쪼개어 처리하는 방식을 통해 단어 수는 줄이되, 언어의 모든 단어를 표현 가능하게 만듦
- 학습 단어 사전에 없는 단어도 Subword로 분할하여 **OOV 문제를 해결**. fashionless -> fashion + less
- Prefix/suffix등을 subword로 분할하여 어근 변화 단어들을 효과적으로 처리. happier -> happi + er
- **한국어**의 경우도 형태소 분석 엔진(Mecab등) **대신** subword 토큰화 방식을 주로 적용하게 됨.
- Subword가 **항상 직관적인** 형태로 분할되는 것은 아님

"M", "a", "c",
"h", "i", "n",
"e", "l", "e",
"a", "r", "n",
"i", "n", "g"

Character 기반

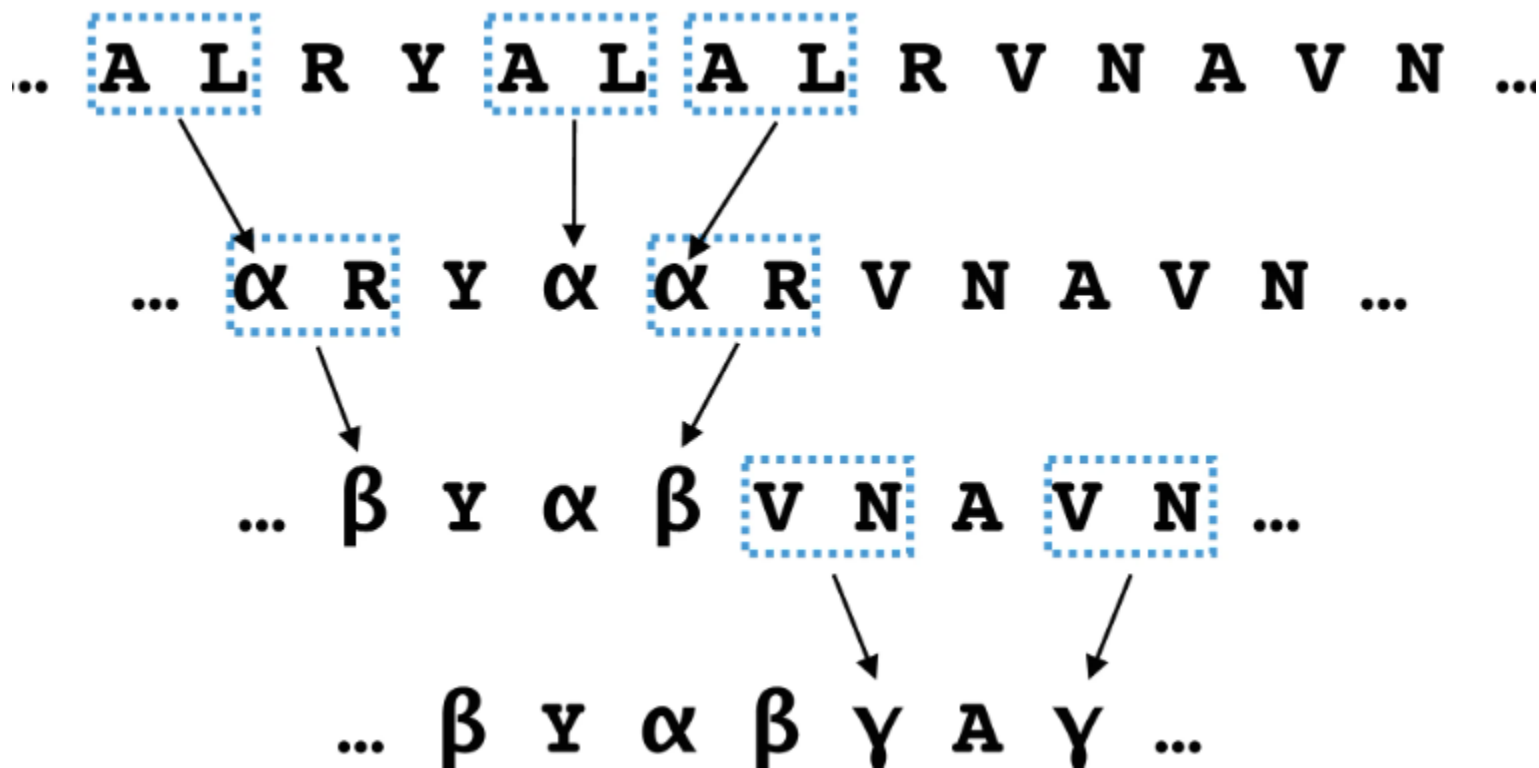
- 개별 문자별로 토큰화를 수행하므로 아주 작은 단어(문자) 사전만 필요하며 OOV 문제가 아예 발생하지 않음
- 단어가 문자레벨로 토큰화 되므로(happiness -> h, a, p, p, i, n, e, s, s) 하나의 문장은 **매우 많은** 토큰들로 구성되어야 함.
- 개별 문자는 **문맥적인 의미**를 반영하지 못함

주요 Subword 토큰화 방식

Subword 토큰화 방식	설명
BPE (Byte Pair Encoding)	말뭉치(Corpus)에서 반복적으로 가장 많이 사용된 연 이은 문자쌍(Pair) 를 선택하고 이를 병합하면서 병합 규칙과 단어 사전을 Update하면서 Subword 생성 예: GPT2,3 Tokenizer
WordPiece	BPE와 유사하지만, 가장 많은 Frequency가 아니라 최대 가능도(maximum likelihood) 를 기준으로 병합 예: BERT Tokenizer
SentencePiece	한국어, 중국어, 일본어등 공백으로 단어 경계를 명확히 나누기 어려운 경우 주로 사용. 문장을 공백 기준 단어로 쪼개지 않고 , 여러 개의 문장들을(또는 말뭉치)에서 확률 기반(Unigram적용) 또는 BPE 기반 병합으로 Subword 생성. 다국어(영어 및 비영어) 번역과 같은 많은 NLP 모델등에 사용됨. 예: T5, KoBERT, mBERT, MarianMT Tokenizer

BPE(Byte Pair Encoding) 개요

말뭉치(Corpus)에서 반복적으로 **가장 많이** 사용된 **연 이은 문자쌍**(Pair)를 선택하고 이를 병합하면서 **병합 규칙과 단어 사전을 Update**하면서 Subword 생성



BPE 토큰화 학습(Train) – 병합과 단어 사전 Update

말뭉치(Corpus)문장들을 아래와 같이 공백 기준으로 Word(단어)를 분할

“low lower newest widest” -> ‘low</w>’, ‘lower</w>’, ‘newest</w>’, ‘widest</w>’

최초에는 말뭉치의 문장내의 개별 Word들을 개별 문자열로 분할

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

이전 분할된 개별 문자열을 가장 높은 빈도의 문자쌍으로 병합

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

단어 사전 등록

l,
o,
w ,
</w>,
e,
r,
n,
s,
t,
d

연이은 문자쌍
(Pair)의 빈도 계산

문자쌍 빈도

lo: 2,
ow:2,
w</w>:1,
we:2,
er: 1,
r</w>:1,
ne:1,
ew:1,
es: 2,
st: 2,
t</w>:2,
wi: 1,
id: 1,
de:1,

가장 높은 빈도의 문자쌍 병합 규칙 등록

병합 규칙 등록

(‘l’, ‘o’)

동일한 빈도일 경우
등장 순서 또는 문자열 사전 순으로 선택

단어 사전 등록

l,
o,
w ,
</w>,
e,
r,
n,
s,
t,
d,
lo

병합 후 연이은 문자쌍(Pair)의 빈도 계산

문자쌍 빈도

low: 2,
w</w>: 1,
we: 1,
er: 1,
r</w>: 1,
ne: 1,
ew: 1,
es:2,
st:2,
t</w>:2,
wi: 1,
id:1,
de:1,

병합 규칙 등록

(‘l’, ‘o’)
(‘lo’, ‘w’)

단어 사전 등록

l,
o,
w ,
</w>,
e,
r,
n,
s,
t,
d,
lo,
low

연이은 문자쌍 빈도 계산 -> 가장 높은 빈도 문자쌍 병합 규칙 등록 및 단어 사전 수정 -> 가장 높은 빈도 문자쌍으로 병합을 반복적으로 수행

본 교재와 실습 자료는 다른 강의나 블로그에 활용 하시면 안됩니다.

BPE 토큰화 학습(Train) – 병합과 단어 사전 Update

‘low</w>’, ‘lower</w>’, ‘newest</w>’, ‘widest</w>’

가장 빈번한 연 이은 문자열 쌍으로 병합 수행

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

lo

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

low

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

es

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

est

‘l o w </w>’, ‘l o w e r </w>’, ‘n e w e s t </w>’, ‘w i d e s t </w>’

.....

병합 규칙 등록

(‘l’, ‘o’)

(‘l’, ‘o’)
(‘lo’, ‘w’)

(‘l’, ‘o’)
(‘lo’, ‘w’)
(‘e’, ‘s’)

(‘l’, ‘o’)
(‘lo’, ‘w’)
(‘e’, ‘s’)
(‘es’, ‘t’)

단어 사전 등록

l, o, w, </w>, e, r, n, s, t, d, lo

l, o, w, </w>, e, r, n, s, t, d, lo, low

l, o, w, </w>, e, r, n, s, t, d, lo, low, es

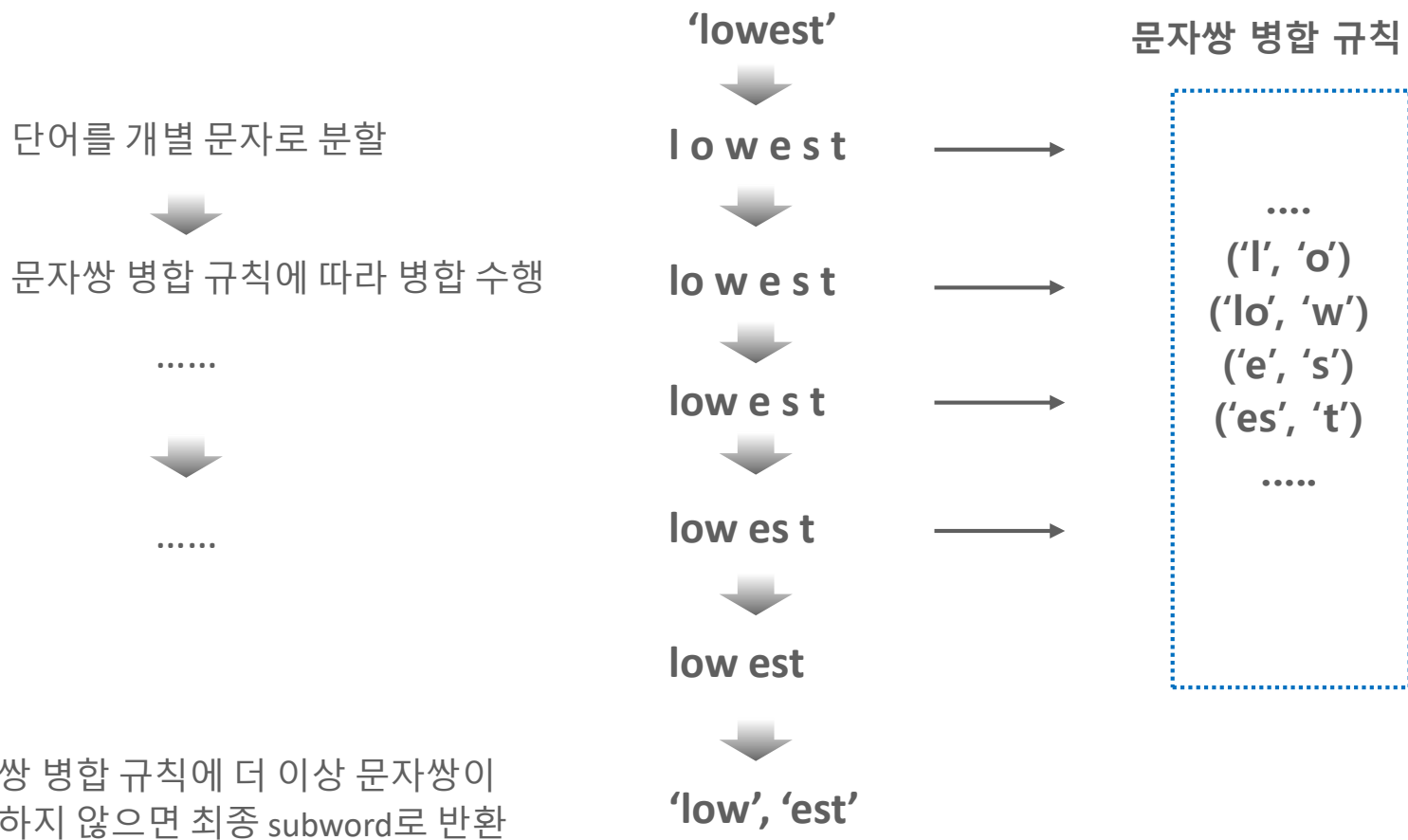
l, o, w, </w>, e, r, n, s, t, d, lo, low, es, est

지정된 단어 사전(Vocabulary)의 크기 만큼 (또는 지정된 merge 횟수 만큼) 반복 수행

- 반복적으로 연속으로 이어지는 문자들은 개별 subword로 분할 생성
- unhappy => un + happy, fashionless => fashion + less

BPE 토큰화 적용

말뭉치 데이터(학습 데이터)를 기반으로 병합과 단어사전 Update 적용하여 만들어진 **문자쌍 병합 규칙**(merge rule)을 기반으로 단어를 subword 적용



BPE Tokenizer 구현 개요

BPE Tokenizer 학습

1. 초기화

- 말뭉치(Corpus) 문장들을 (공백 기준) 단어로 분할 후 개별 단어를 개별 문자로 분할
- 모든 고유 문자로 초기 단어 사전 구성

2. 지정된 단어 사전의 크기만큼 반복해서 병합 수행

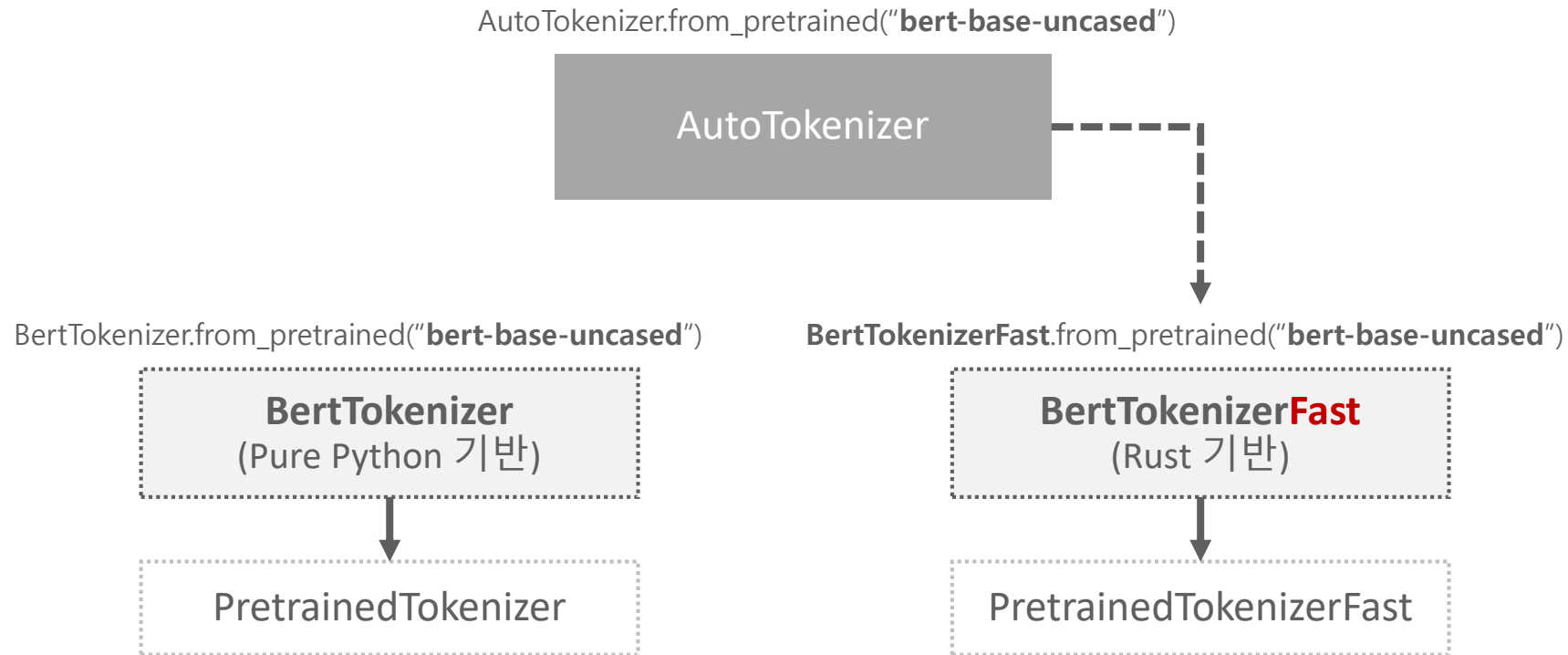
- 연이은 문자쌍을 추출한 뒤 가장 많은 횟수의 해당 문자쌍을 선택
- 가장 많이 반복되는 문자쌍을 병합 규칙에 등록하고 하나의 심볼로 병합하여 새로운 subword 토큰 생성.
- 병합된 새로운 subword 토큰을 단어 사전에 등록하고 이를 다시 하나의 심볼로 간주하고 다시 연이은 문자쌍을 추출

학습된 BPE Tokenizer의 병합 규칙을 이용하여 단어의 Subword 토큰화 적용

- 단어를 개별 문자로 분할한 뒤 연이은 문자쌍이 병합 규칙에 있으면 이들 문자쌍을 병합, 병합 규칙에 없으면 개별 문자를 그대로 유지하면서 Subword 토큰화 적용

Fast Tokenizer와 Auto Tokenizer

- HuggingFace는 (기존) Pure Python 기반 Tokenizer를 대체하고, 성능을 향상 시킨 **Rust** 기반 Tokenizer를 새롭게 제공
- HuggingFace가 제공하는 모든 Tokenizer가 다 Rust 기반은 아니지만, 자주 활용되는 BERT 계열 등의 Tokenizer는 Rust 기반으로 제공
- BertTokenizerFast와 같이 기존 Tokenizer 클래스명에 **Fast** 접미어를 붙인 새로운 클래스로 제공
- **AutoTokenizer**를 이용하면 Fast Tokenizer가 있는 Tokenizer의 경우 자동으로 Fast Tokenizer를 제공하며 Tokenizer 클래스명을 몰라도 model 명 (id)만 입력하면 해당되는 Tokenizer 객체를 반환



HuggingFace Dataset에 Tokenization 적용하기

- HuggingFace Dataset은 torch.utils.data.Dataset 과 호환되어 DataLoader에 바로 사용 가능
- 이미 Tokenization이 Dataset에 적용되었기 때문에 DataLoader에서 Batch Dataset을 Fetch시마다 Tokenization을 따로 적용할 필요 없어서 Train 시간 단축 및 로직 간소화

```
bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
def tokenize_fn(examples):  
    return bert_tokenizer(examples, padding="max_length",...)
```



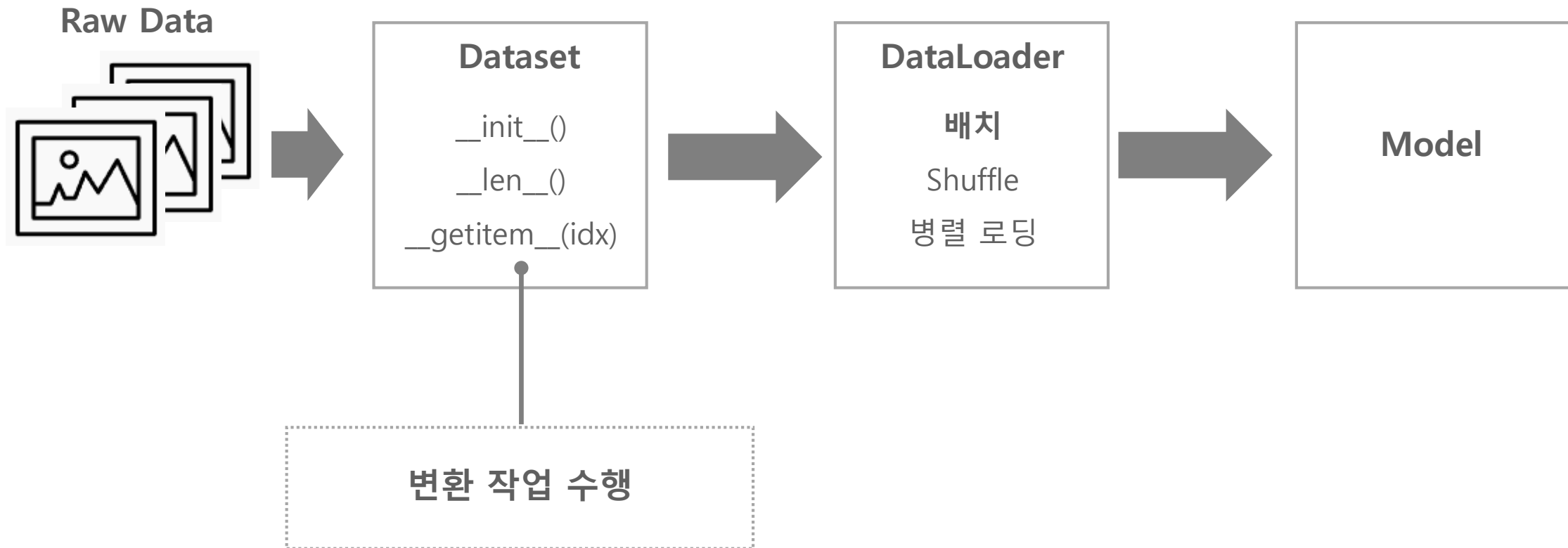
Text 기반 최초 Dataset



Tokenize 적용된 Dataset

torch.utils.data.Dataset와 DataLoader 개요

- DataSet은 원본 데이터를 접근하고 전처리/변환에 중점
- DataLoader는 DataSet 데이터를 Batch/Shuffle/병렬 로딩 등으로 모델에 효과적으로 전달하는데 중점.



torch.utils.data.Dataset vs 와 HuggingFace Dataset 사용 비교

Torch.utils.data.Dataset 사용 시

```
class CusDataset(torch.util.data.Dataset):  
    def __init__(self, , texts, labels, tokenizer):  
        내부 변수 초기화  
        self.tokenizer = tokenizer  
    def __len__(self):  
        전체 건수 반환  
        .....  
    def __getitem__(self, index):  
        데이터 접근 및 토큰화 적용  
        .....  
  
cus_dataset = Cus_Dataset(...)  
train_dataloader = DataLoader(dataset=cus_dataset,  
                               batch_size=8, shuffle=True)
```

HuggingFace Dataset 사용 시

```
raw_datasets = load_dataset("imdb")  
def tokenize_fn(examples):  
    return bert_tokenizer(examples['text'], padding="max_length"...)  
tokenized_datasets = raw_datasets.map(tokenize_fn,  
                                       batched=True, remove_columns=["text"])  
  
train_dataloader = DataLoader(dataset=tokenized_datasets["train"],  
                              batch_size=8, shuffle=True)
```

DataLoader에서 custom collate 함수 적용하기

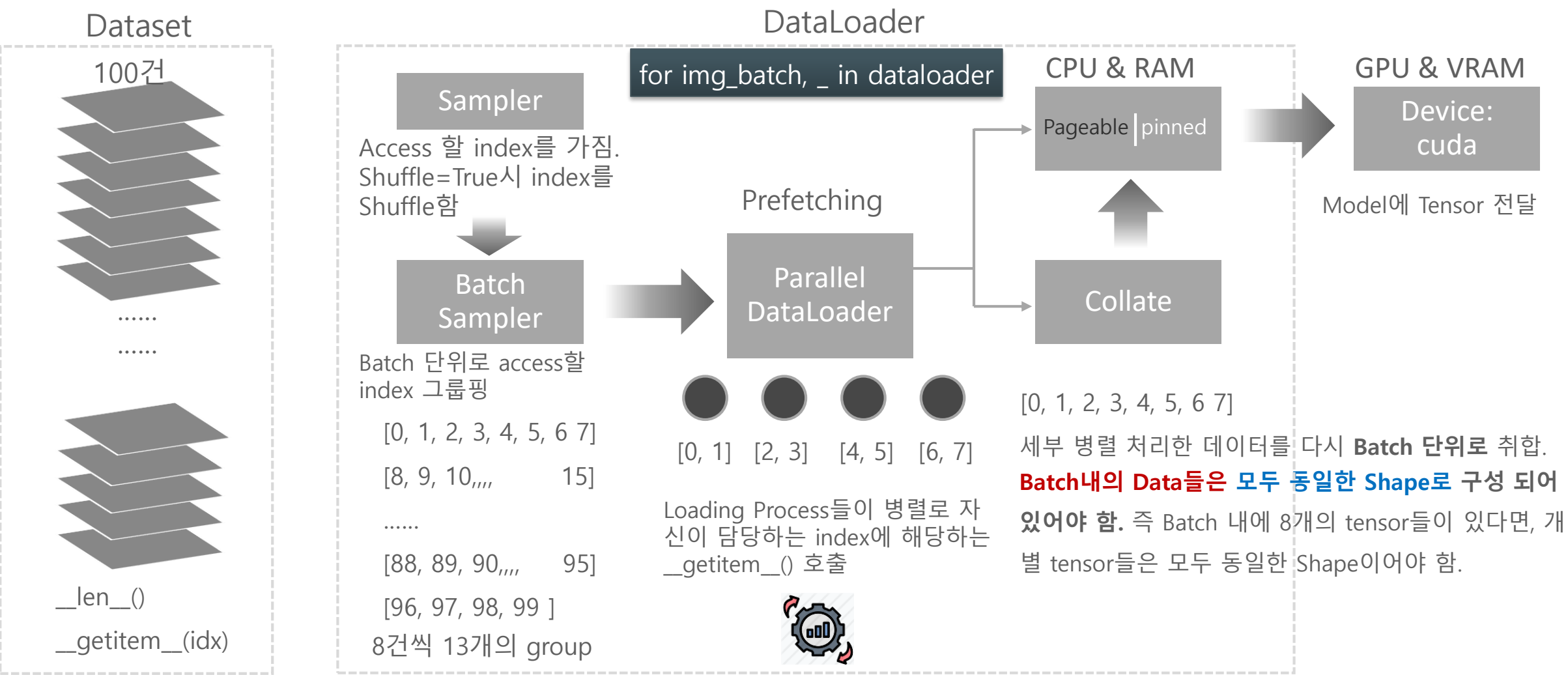
- DataLoader는 Dataset를 Sampler, Batch Sampler, Parallel 처리, 그리고 Collate 단계를 거쳐 batch 단위로 최종 출력
- DataLoader의 **Default Collator**는 최종 출력전에 Data를 **batch 단위로 묶어주는** 역할을 수행
- 만약 batch내의 Data들이 **서로 다른 shape**를 가지고 있다면 Default Collator는 이를 batch 단위로 묶을 수 없어서 오류 발생.
- 이 경우 **collate_fn** 인자에 개별 Data들을 batch 단위로 묶을 시 동일한 shape로 처리할 수 있는 **custom collate 함수**(또는 callable object)를 직접 생성해서 적용
- 보통 NLP에서 배치단위로 **동적 Padding**을 적용하거나, Post Processing 및 Shape 변경등을 수행 시 custom collate 함수 이용

```
train_dataloader = DataLoader(dataset=train_dataset, ....., collate_fn = custom_collate_fn)
```

```
def custom_collate_fn(batch):  
    pad_token_id = 0  
    .....  
    for sample in batch:  
        ids = sample  
        ....  
    return torch.tensor(input_ids)
```

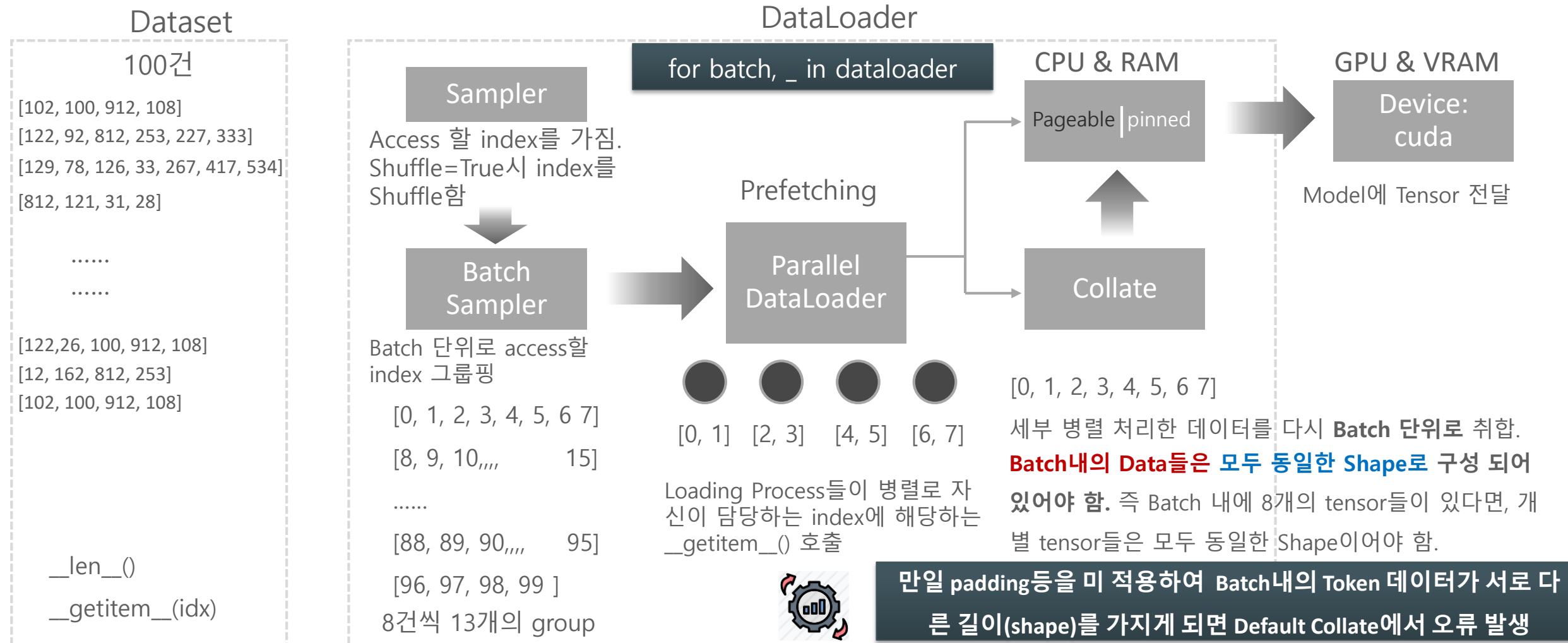
DataLoader 동작 메커니즘 – Image Classification

Image Dataset 총 데이터 건수 100건, batch_size = 8, num_workers=4, pin_memory = True시 Data Loader 동작 메커니즘



DataLoader 동작 메커니즘 - NLP

Token Dataset 총 데이터 건수 100건, batch_size = 8, num_workers=4, pin_memory = True시 Data Loader 동작 메커니즘

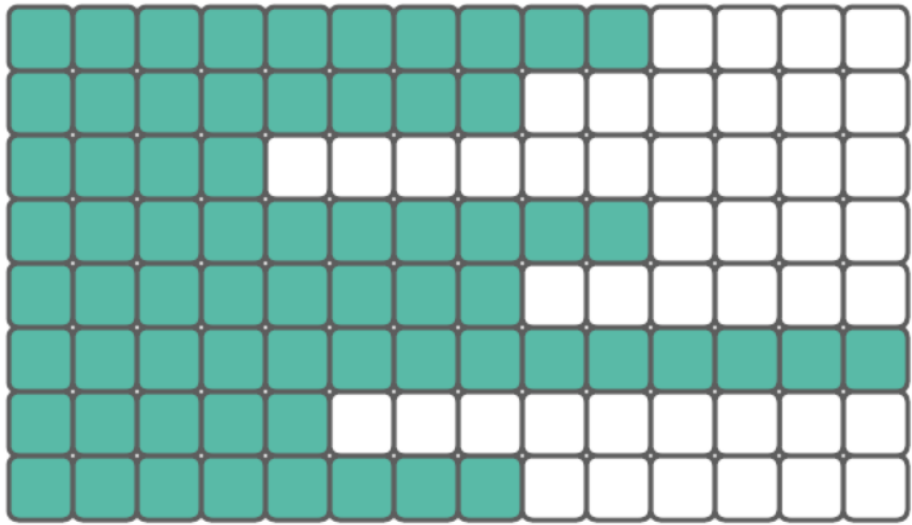


본 교재와 실습 자료는 다른 강의나 블로그에 활용 하시면 안됩니다.

NLP 데이터 세트의 배치 단위 동적 Padding 적용

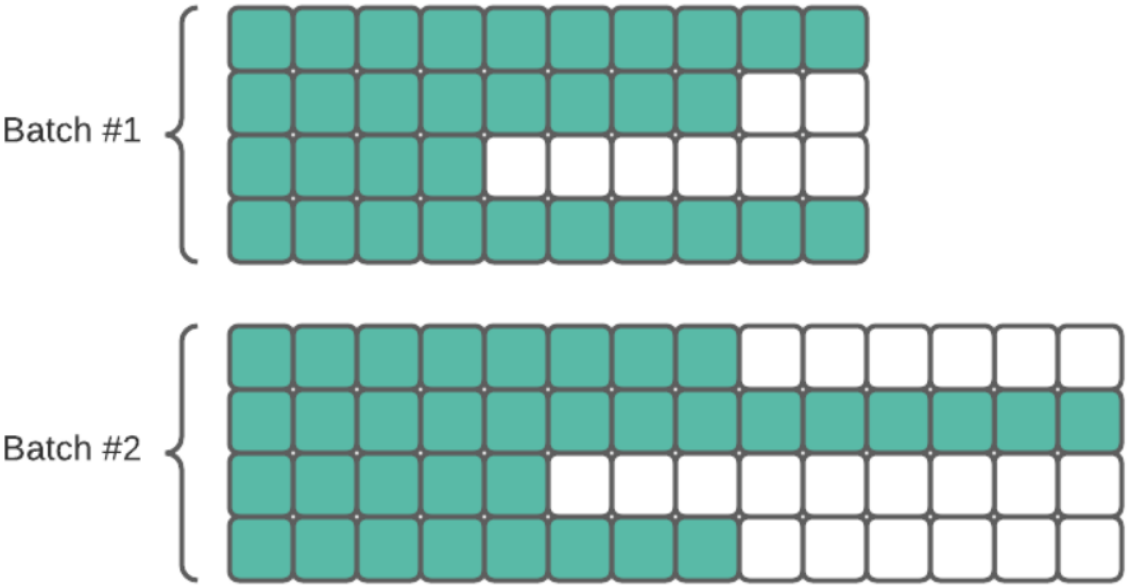
■ 문장내 실 Token
□ Pad Token

전체 데이터 세트의 문장들을
미리 동일한 크기로 Padding 적용



VS

배치 단위로 Fetch된 문장들 내에서 동일한
크기로 Padding 적용



- 개별 문장들의 길이가 큰 편차를 보이는 경우 과다 GPU 메모리 소모 및 모델 학습 시간이 길어질 수 있음

- 배치내에서 가장 긴 문장들을 기준으로 동적으로 Padding 적용
- 개별 문장들의 길이가 큰 편차를 보이는 경우라도 보다 효율적으로 Padding 적용하여 GPU 메모리 절약 및 모델 학습 시간 감소

HuggingFace의 Data Collator 클래스

- HuggingFace는 Collate Function을 **callable object** 가 구현된 **다양한 Class**들로 사전 제공함
- 이들 Data Collator 클래스들은 배치 단위의 동적 Padding 제공으로 메모리 절약과 학습 시간 단축을 가능하게 함
- 또한 다양한 NLP 영역에서 필요한 **추가적인 데이터 처리 기능** 제공
 - Seq2Seq에서 Encoder와 Decoder 입력 및 Target 값 처리
 - Masked Language Model에서 token masking
 - Question and Answer Model에서 start와 end Label 생성.
- 기존 HuggingFace Tokenizer와 **잘 통합되며**, 보다 간결한 모델 Training Code 작성 기여

```
collator = DataCollatorWithPadding(tokenizer=bert_tokenizer)
train_dataloader = DataLoader(dataset=train_dataset, ..... , collate_fn = collator)
```

주요 Data Collator 클래스 및 역할

DataCollator Class	주요 NLP Task	주요 수행 역할
DataCollatorWithPadding	Text 분류(Classification)	배치 단위 동적 padding
DataCollatorForLanguageModeling	BERT MLM 학습 또는 GPT등의 LLM 학습	MLM 또는 causal LM masking
DataCollatorForSeq2Seq	Translation, Summarization	Encoder, Decoder 의 입력과 Target 값 가공, Decoder 입력의 shift 값 적용(모델 제 공시에만)등
DataCollatorForTokenClassification	NER, POS tagging	Label 값 Padding 처리등
DataCollatorForQuestionAnswering	Question and Answer	start/end 위치 처리

BERT

(Bidirectional Encoder Representations from Transformer)

대량의 Text 데이터를 Transformer Encoder 기반의 **MLM과 NSP** 모델로 **Pretrained** 학습 시킨 뒤 이 Pretrained 모델을 다양한 NLP 영역에서 사용할 수 있도록 확장시킨 모델

BERT 개요

Transformer 모델 구조는 큰 변경이 없지만, **매우 많은 데이터 세트**를 이용하여 Language가 가지는 **일반적인 특성**(문법, 구조, 문맥등)을 **사전 학습** 한 뒤 **특정 도메인 영역**에 맞춰서 매우 적은 데이터 세트로 모델을 Fine Tuning하여 NLP 영역에서 높은 성능을 가진 모델을 구성함

Pre-Training

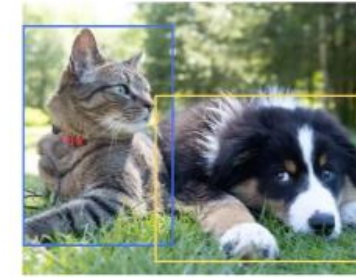
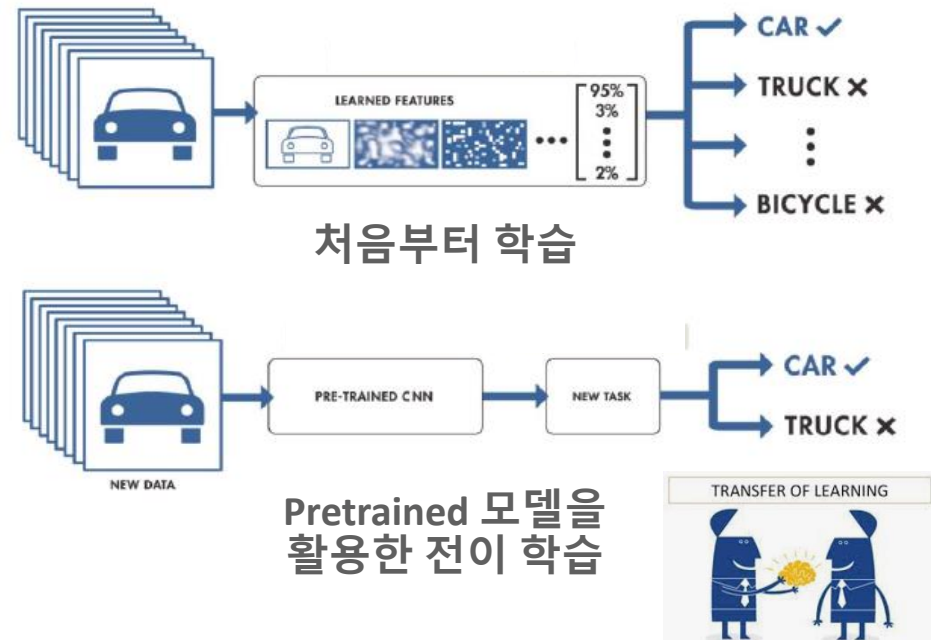
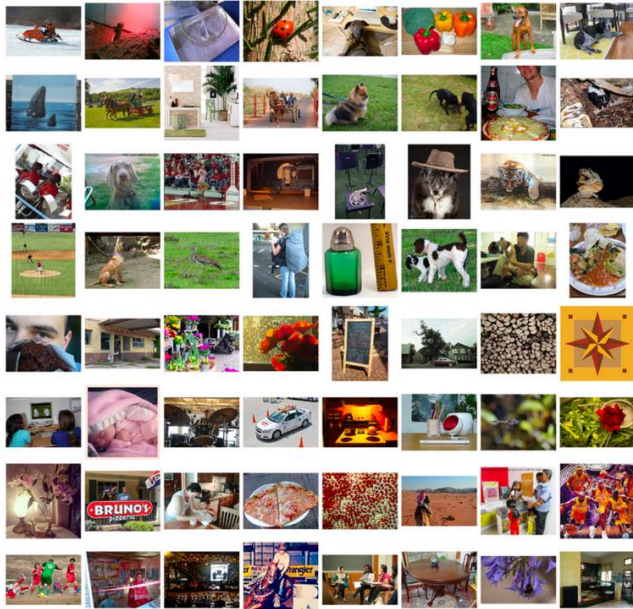


Fine Tuning

- 대용량의 데이터 세트를 기반으로 Language가 가지는 **일반적인 특성**(문법, 구조, 문맥등)을 사전 학습
MLM(Masked Language Model)과 **NSP**(Next Sentence Prediction) 적용하여 학습

- Pretrained 된 모델에 상대적으로 매우 적은 데이터 세트를 특정 도메인 영역에 맞춰서 학습 시킴
- 특정 도메인 영역에 따라서 Pretrained 모델 구조를 조금씩 변경하여 상세 모델 구성 및 학습 수행
- 텍스트 판별(스팸 판별, 감성 분석, 토픽 판별), 질문과 답변, 개체명 인식, 문장 요약 등의 도메인에 상세 모델 적용

이미지 영역에서의 Pretrained 모델과 다양한 활용



Object Detection



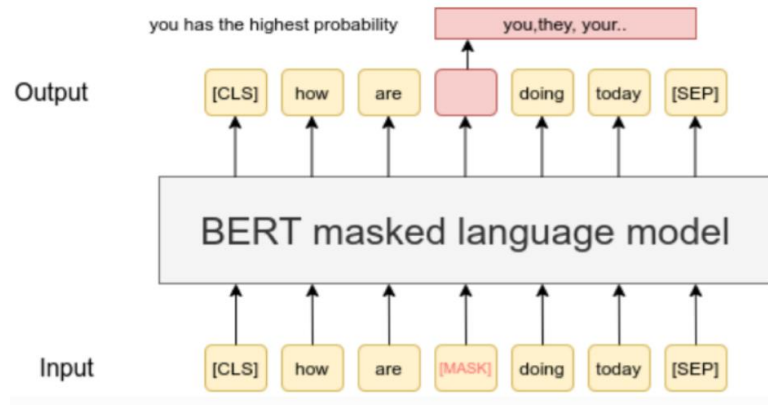
Image Segmentation

- ImageNet과 같은(백만개 이상의 이미지들, 1000 개의 class) **대량의 데이터 세트**에서 이미지 분류 학습을 수행한 **모델의 파라미터**를 다른 이미지의 분류나 Object Detection, Segmentation과 같은 다른 도메인 영역에서 재 활용함

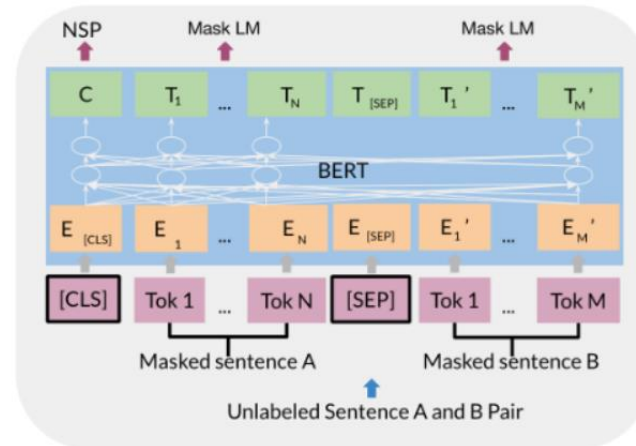
Pretrained MLM과 NSP 개요

- 11,000권의 출판되지 않은 책(800M 단어) 과 웹 상의 영어 Wikipeda(2300M 단어)를 Pretrained 학습 데이터로 사용
- Pretrained 모델은 Transformer Encoder 기반의 MLM과 NSP를 적용
- MLM은 입력 문장들의 전체 토큰 중 **일정 비율로 Random하게 Mask Token**으로 변환한 뒤 이들 Mask Token의 실제 토큰을 판별(Classification)하는 방식으로 모델을 학습
- NSP는 문장 A와 문장 B가 있을 때 문장 B가 문장 A의 **다음 문장인지의 여부**를 판별하는 방식으로 모델을 학습

MLM(Masked Language Model)



NSP(Next Sentence Prediction)



Book

800M 단어

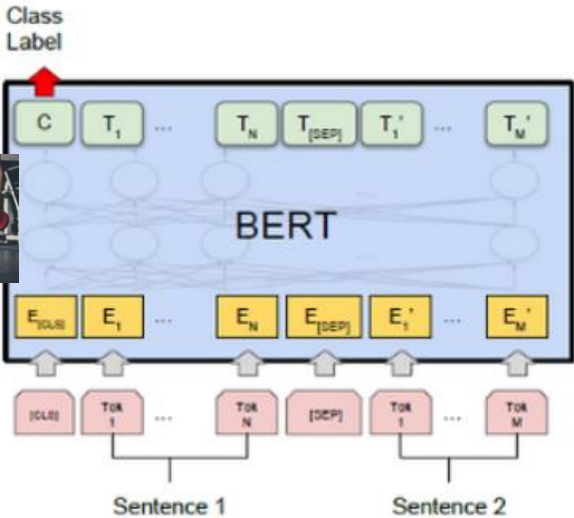


WIKIPEDIA
The Free Encyclopedia

2,300M 단어

BERT Pretrained 모델과 Down-stream task 모델

Pre-training Model

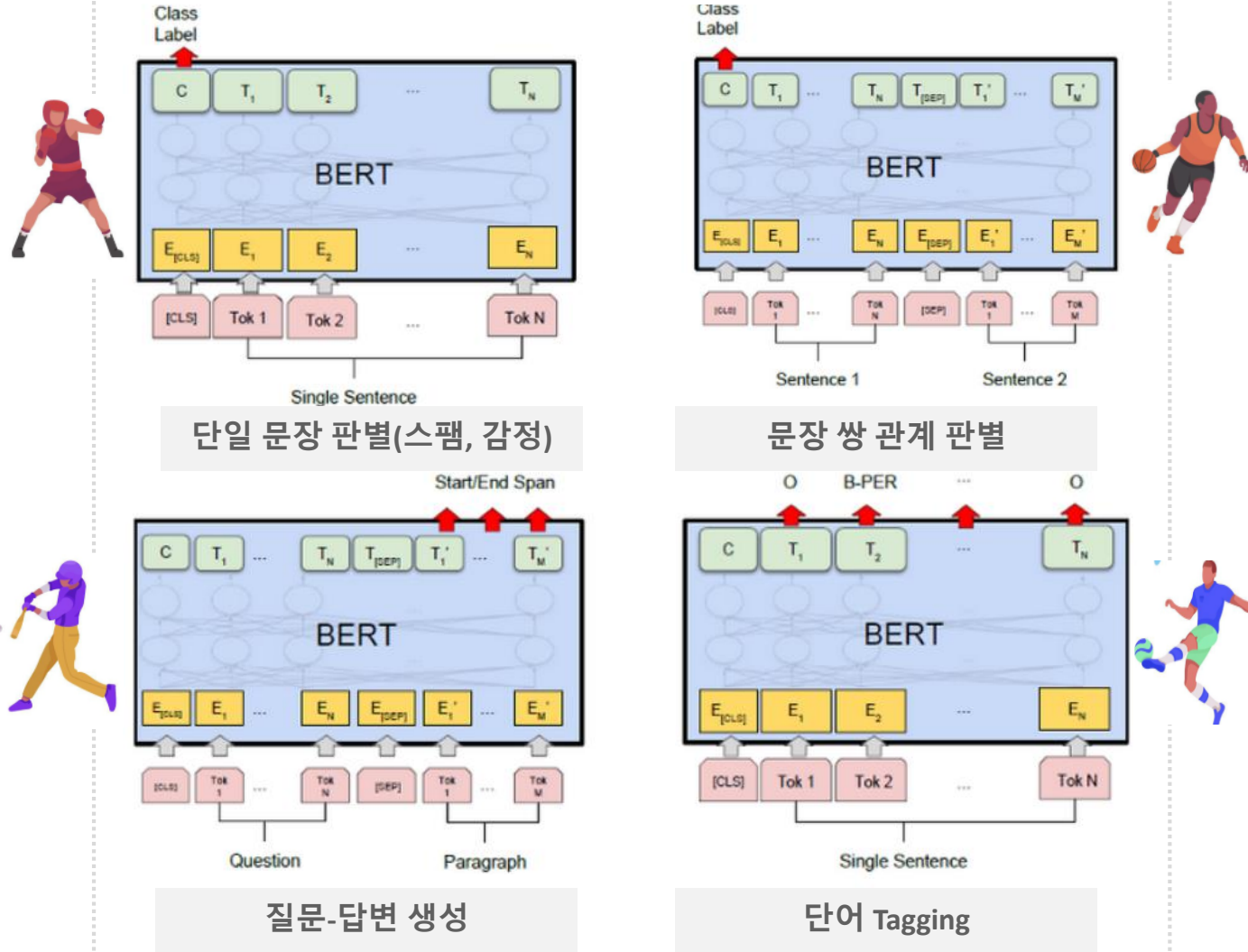


MLM + NSP

대용량 학습 데이터로 일반적인 Language 특성 (문법, 구문, 맥락)을 사전 이해

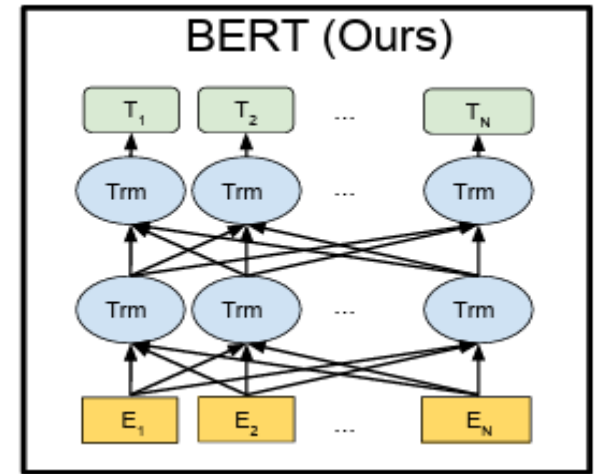
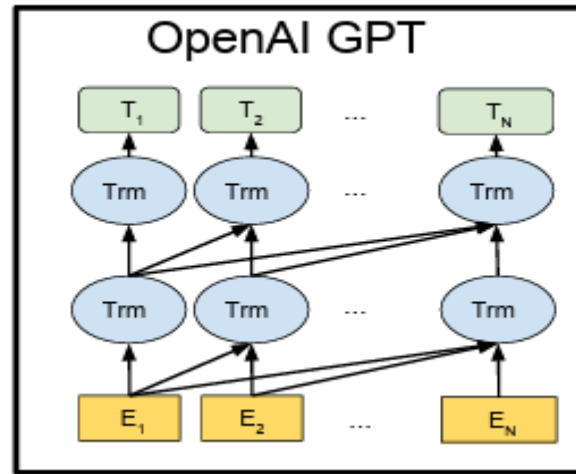
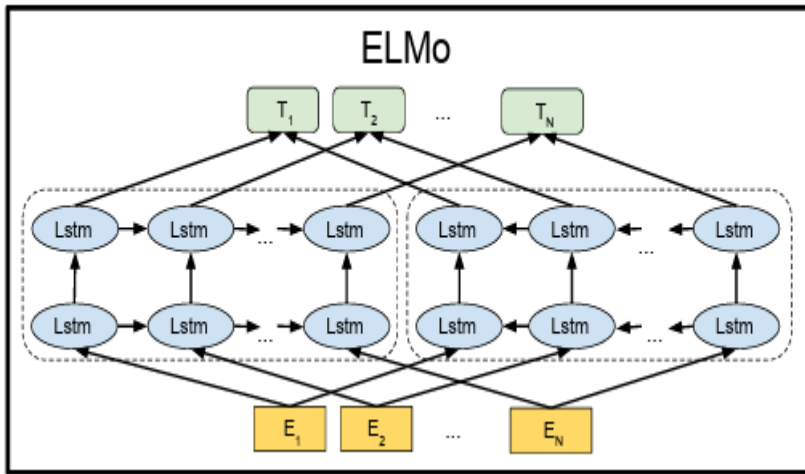


문제 영역에 따른 다양한 Fine Tuning 모델



BERT와 BERT 이전 Pretrained Model 비교

- BERT 이전, Pretrained 모델의 주요 기법은 feature-based와 fine-tuning
- Feature based 방식은 ELMo와 같이 주로 Pretrained된 embedding을 down-stream task(도메인 별) 모델에 제공
- BERT와 GPT 는 모두 embedding 부터 모델 까지 사전 학습되는 Fine tuning 방식이지만, GPT는 Decoder only 구조로 Causal Masking이 적용된 **단방향(Unidirectional)** 학습에 기반한 Pretrained 모델
- BERT는 Encoder-Only 구조로 Transformer Attention이 온전하게 활용된 **양방향(Bidirectional)** 학습에 기반한 Pretrained 모델 기법을 적용하여 타 기법대비 뛰어난 성능향상을 가져 옴



Decoder 의 Causal Masking

- GPT는 Decoder Only 구조이며 이전 까지 생성 된 Token들을 기반으로 다음 Token을 예측하는 데 초점(Auto Regressive)
- 이를 위해 Decoder의 Attention은 **Causal Masking**을 적용하기에 진정한 의미의 Bi-Directional한 Pretrained Model 에 적합하지 않다고 논문에서 언급

Decoder의 Causal Masking

		Key 그 고양이 저쪽 매트에 앉아 있네요					
Query	그	1.0	1.0	0.5	0.0	1.0	1.0
	고양이가	1.0	2.0	1.5	1.0	1.0	1.5
	저쪽	0.5	1.5	1.5	1.5	0.5	1.5
	매트에	0.0	1.0	1.5	0.0	1.0	1.5
	앉아	1.0	1.0	0.5	0.0	1.0	1.0
	있네요	1.0	1.5	1.5	1.5	1.0	2.2

“Don’t peek into the future”

Decoder에서는 학습 시 미래 Token 을 미리 들여다 보서는 안됨.

그	고양이가	저쪽	매트에	앉아	있네요
그	고양이가	저쪽	매트에	앉아	있네요
그	고양이가	저쪽	매트에	앉아	있네요
그	고양이가	저쪽	매트에	앉아	있네요
그	고양이가	저쪽	매트에	앉아	있네요
그	고양이가	저쪽	매트에	앉아	있네요

BERT와 타 모델 성능 비교

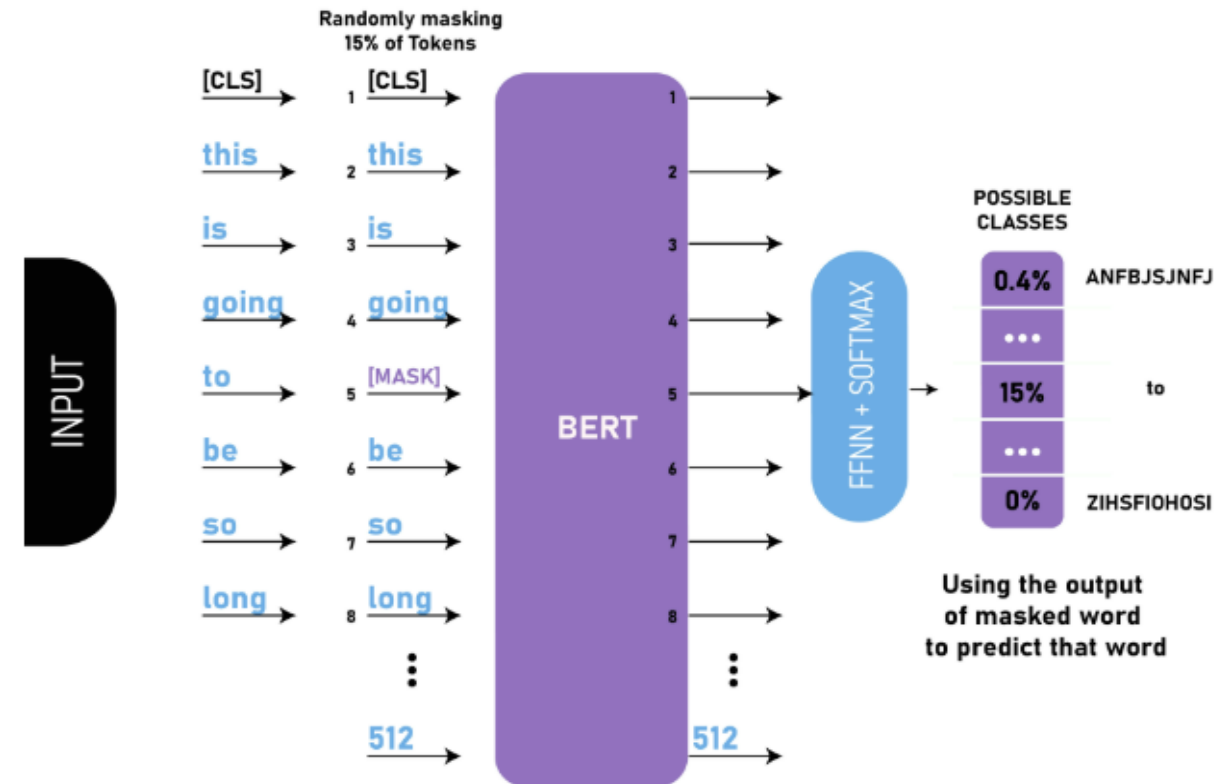
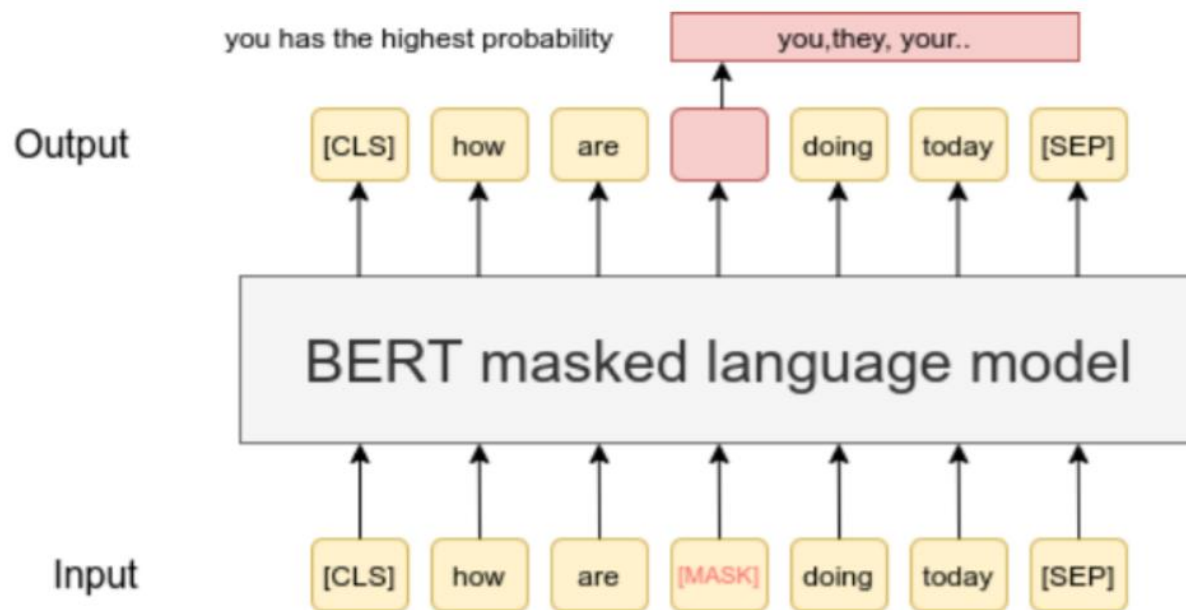
GLUE(General Language Understanding Evaluation) 비교

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

- MNLI (Multi-Genre Natural Language Inference) : 자연어 추론 (함의, 모순, 중립)
- QQP (Quora Question Pairs): 중복 질문 판별
- QNLI (Question Natural Language Inference, SQuAD 변형): 질의 응답.
- SST-2 (Stanford Sentiment Treebank): 감정 분류 (긍정 / 부정).
- CoLA (Corpus of Linguistic Acceptability): 문장이 문법적으로 옳은지 여부 판별
- STS-B (Semantic Textual Similarity Benchmark): 문장 의미 유사도 예측

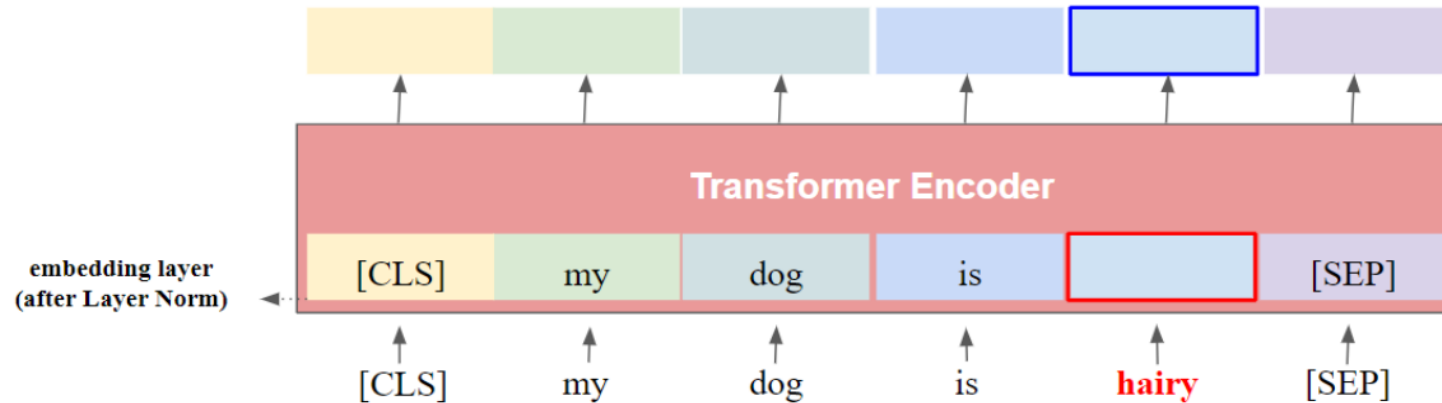
BERT의 Pretrained MLM(Masked Language Model) 개요

- MLM은 BERT가 **Bidirectional(양방향)** 문맥 표현을 사전 학습(Pretrain) 하기 위해 사용되는 주요 기법
- 입력 문장의 약 **15%의 Token**들을 [MASK] Token으로 치환하여 BERT Model에 입력하고, Label에 해당 Mask Token의 원래 Token을 예측할 수 있도록 모델을 학습 시키는 방법
- [MASK] Token이 실제 어떤 Token인지 예측하는 학습을 통해 모델이 **[MASK]를 둘러싼 문장의 양 방향으로** 최적의 Context를 학습하면서 추후에 Down-stream Task에서도 잘 활용될 수 있는 좋은 학습 파라미터가 Pretrained 됨

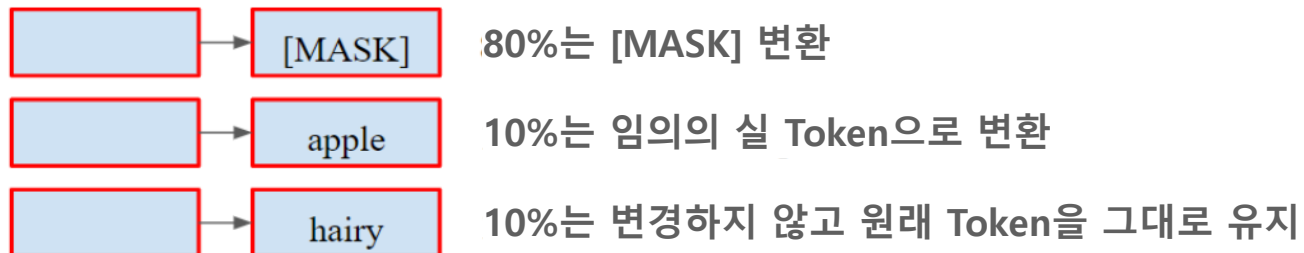


Pretrain-Finetune mismatch 개선을 위한 전략

- MLM Pretrained 시 MASK Token을 **잘 예측하는 방향으로만** 학습이 되면 실제 이를 활용하는 Down Stream Task 모델에서는 MASK Token이 입력되지 않기 때문에 오히려 성능이 저하되는 문제가 발생.
- 이를 개선하기 위해서 Pretrained MLM Model이 MASK Token에 집중해서 학습하는 방향성을 **조금 약화 시키기 위해**(Generalization) Mask 대상 토큰 중 일부는 **다른 실 Token 또는 원래 실 Token으로 대체**

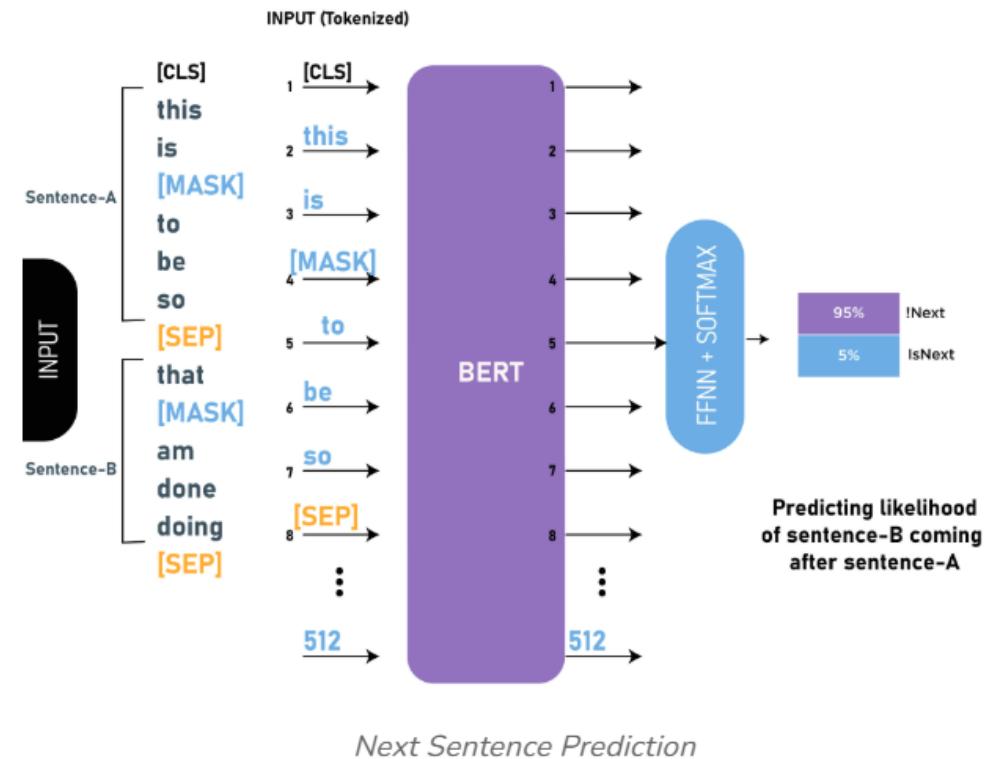
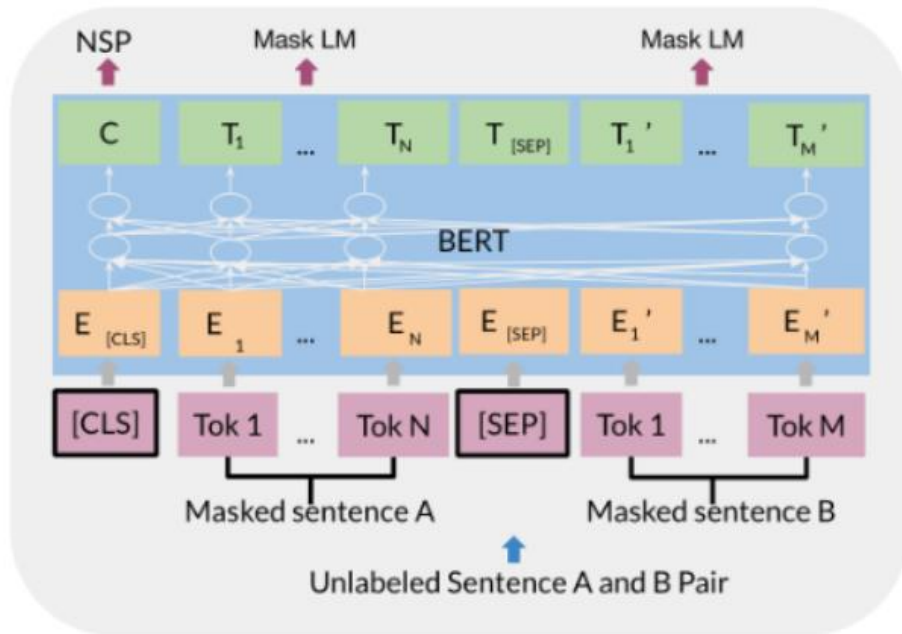


개별 문장에서 15%의 토큰들을 Random하게 [MASK] Token 변환 (e.g., **hairy**)



BERT의 Pretrained NSP(Next Sentence Prediction) 개요

- BERT는 **질의 응답(Question and Answer)**과 같은 문장 관계를 기반으로 한 NLP 세부 모델에 잘 활용되기 위해 **NSP를 MLM과 함께** Pretrain 적용
- NSP는 2개 문장 쌍으로 이뤄진 학습 데이터를 기반으로, 두번째 문장이 첫번째 문장 다음에 이어지는 문장인지를 학습하는 방식. 예를 들어 두개의 연이은 문장 두개가 A, B 순으로 있을 때 B 문장이 A문장 다음에 존재하는 문장인지 여부를 학습



BERT 모델과 Original Transformer Encoder와의 주요 차이

- BERT 모델은 Original Transformer Encoder와 거의 유사한 아키텍처를 가지지만 아래와 같은 주요 차이가 있음
 - ✓ Positional Embedding: 문장 Sequence의 Position 값을 위해 Sinusoidal을 사용하지 않고 **Learnable Embedding** 을 사용
 - ✓ Feed Forward 활성화 함수로 ReLU가 아닌 **GELU**를 사용
 - ✓ 보다 안정적인 [CLS] Token 표현과 학습을 위한 **Pooler Layer**

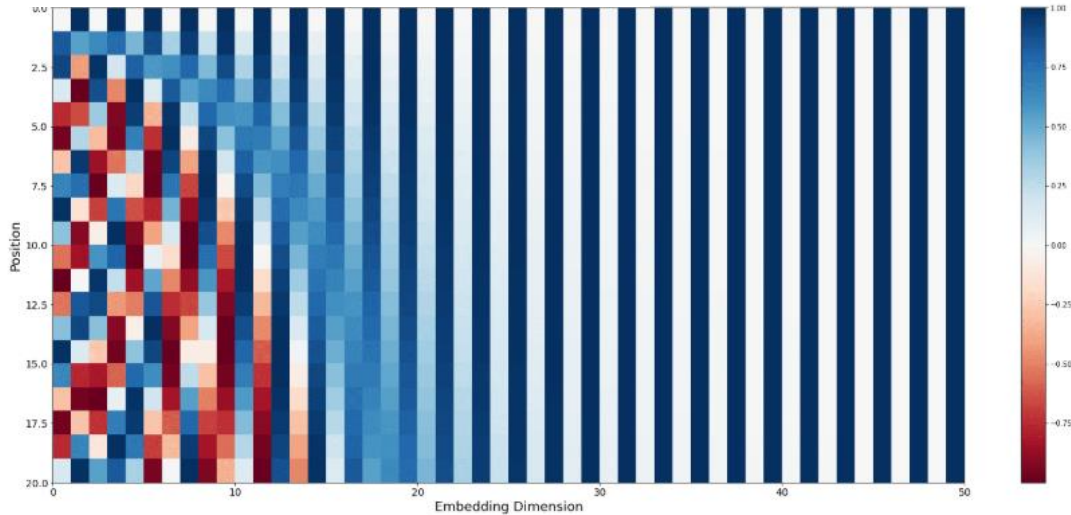
BERT의 (Learned) Positional Embedding

- Original Transformer에 사용된 Sin/cosine 기반의 Sinusoidal Positional Encoding과 다르게 문장내 Position 자체도 **학습**을 통해 얻을 수 있도록 **Embedding** 기반의 Positional Embedding을 적용
- 입력 문장의 0 ~ 문장길이 -1 까지의 토큰별 위치값을 embedding으로 학습하여 표현

Sinusoidal Positional Encoding

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



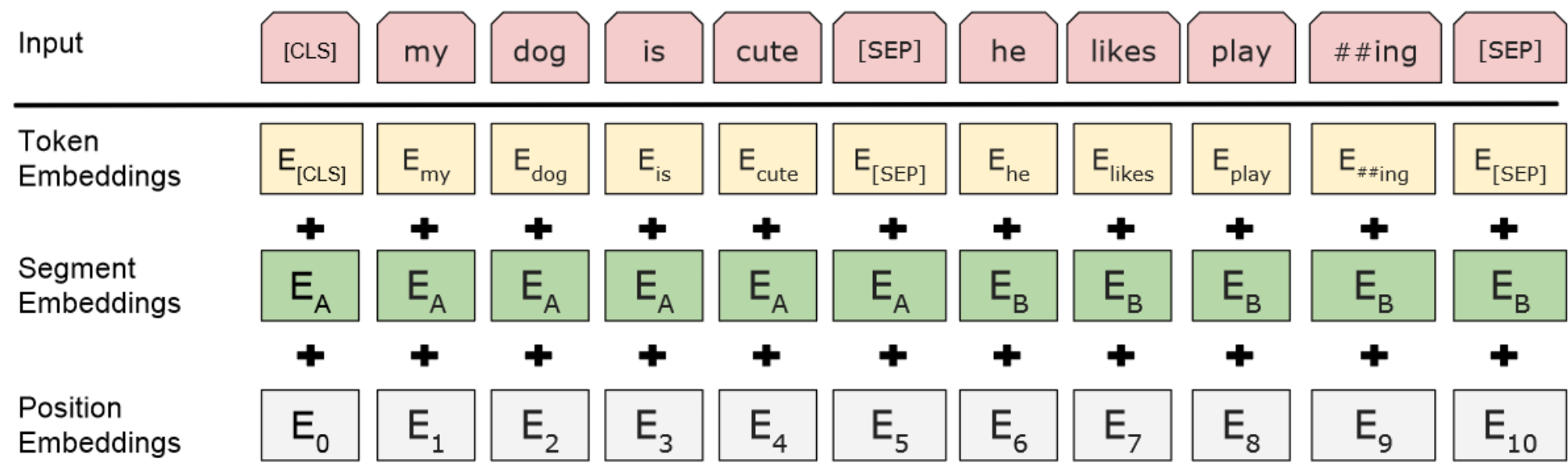
Positional Embedding

문장내의 Position 값을 Embedding 기반으로 **학습**

```
nn.Embedding(num_embeddings=max_len,  
              embedding_dim=hidden_size)
```

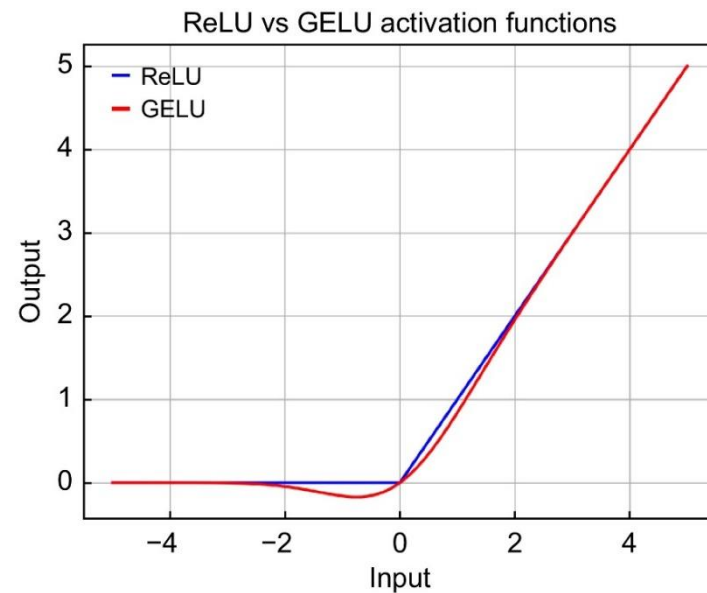
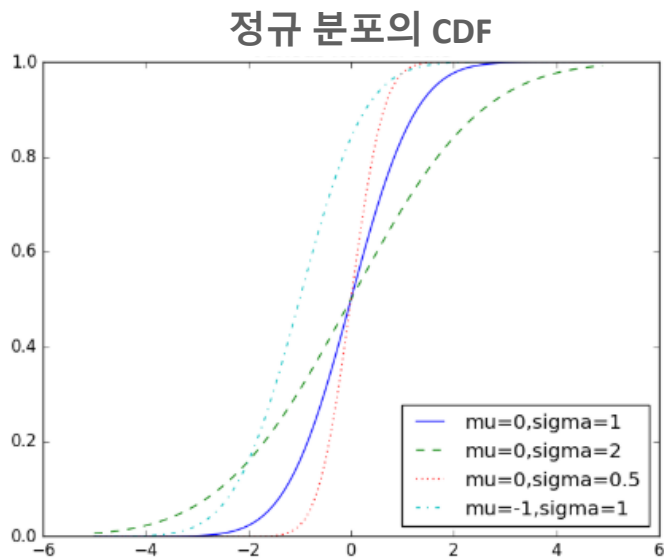

BERT의 입력 구성

- NSP pretrained나 Q&A등의 문장쌍 기반에서 토큰별 첫번째, 두번째 문장 여부(0 또는 1) 역시 segment embedding으로 학습
- Sequence(문장)의 토큰 Embedding과 Positional Embedding 그리고 문장 쌍의 첫번째, 두번째 문장 구분 Embedding을 모두 더해서 모델 입력



GELU(Gaussian Error Linear Unit) 활성화 함수

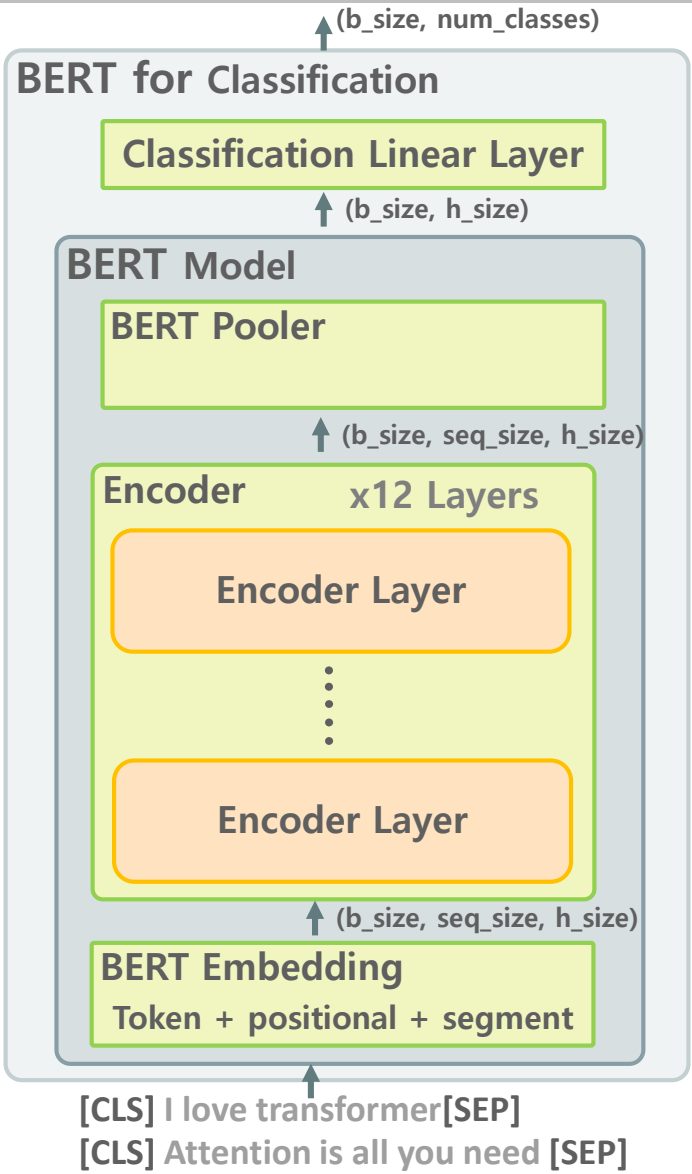
- 입력 값이 마이너스 ~ 0에서 플러스로 변할때 **급격히 미분 값이 변경되는** ReLU의 단점을 개선하기 위해 제시
- GELU는 입력값에 정규분포의 CDF를 곱해서 출력. **$GELU(x) = x \cdot \Phi(x)$**
- GELU는 입력이 +로 큰 값에서는 입력값과 유사하며, -로 큰 값에는 0에 가까움. 하지만 입력값이 0 근처에서 보다 Smooth한 값을 가짐.
- Transformer 계열의 모델에서는 ReLU보다 GELU를 선호.



$$\text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

$$\text{GELU}(x) = x \cdot \Phi(x)$$

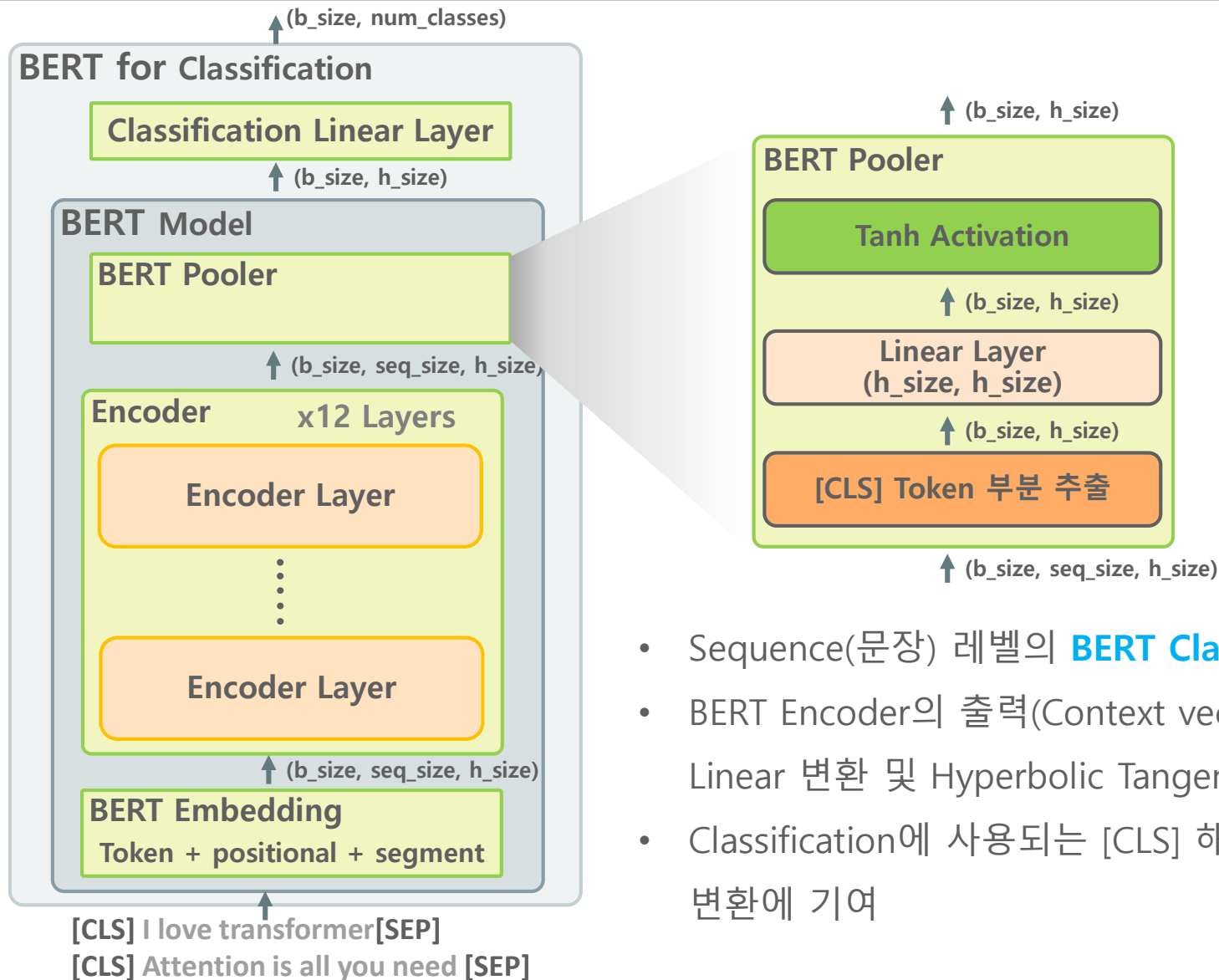
BERT 아키텍처 구조



BERT Base와 Large 차이

주요 Spec	BERT Base	BERT Large
Encoder Layer 수	12	24
Hidden size	768	1024
Attention multi heads 수	12	16
Feed forward intermediate 크기	3072	4096
학습 파라미터 수	~110M	~340M

BERT Pooler

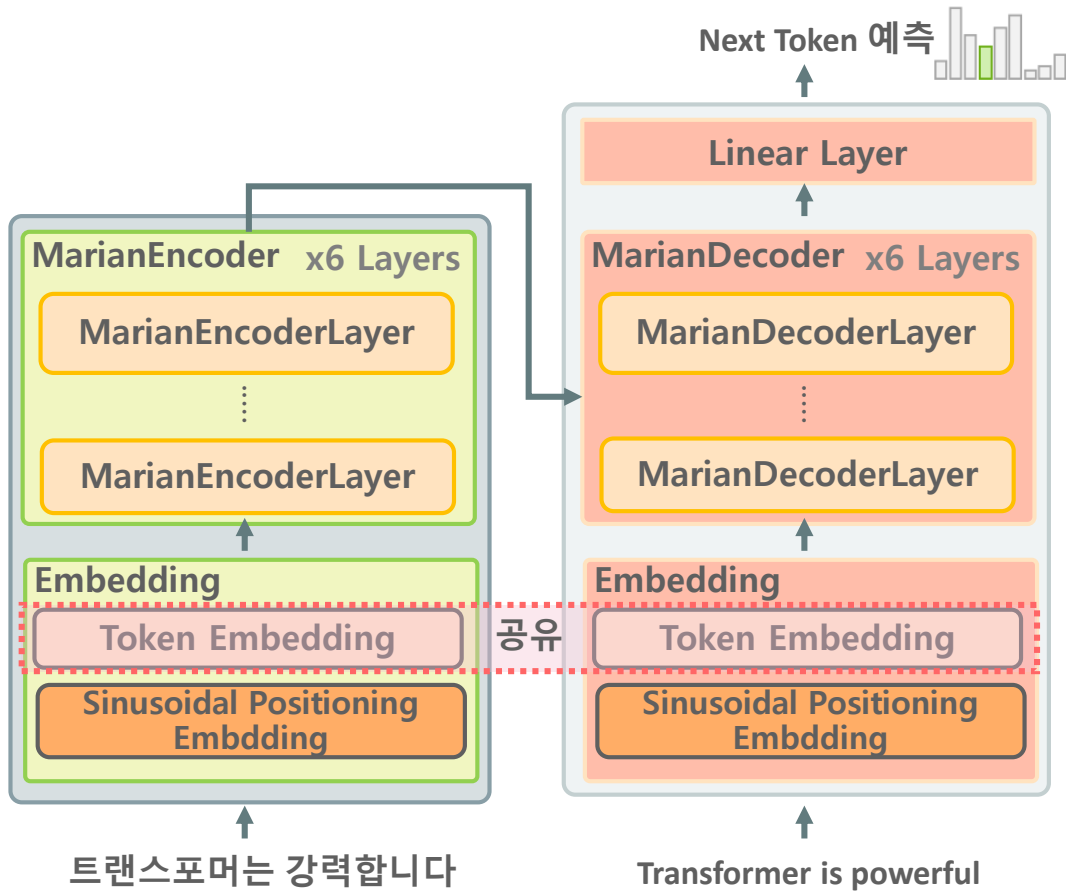


- Sequence(문장) 레벨의 **BERT Classification** 모델에서 주로 적용됨.
- BERT Encoder의 출력(Context vector)에서 **[CLS] Token에 해당하는 부분만** 추출 후 Linear 변환 및 Hyperbolic Tangent 활성화 함수 적용
- Classification에 사용되는 [CLS] 해당 되는 Context Vector 부분에 대한 보다 적절한 변환에 기여

Encoder-Decoder 번역 모델

MarianMT

MarianMT Model 소개



- Helsinki 대학 NLP 연구소의 Marian framework으로 만들어진 Machine Translation 모델을 기반으로 Hugging Face에서 Pytorch로 작성해 제공
- 모델 구조는 Original Transformer Encoder-Decoder와 **거의 유사하며**, Sinusoidal Positional Embedding 구조, Feed Forward에서 SiLU 활성화 함수를 사용
- Encoder-Decoder 기반의 번역 모델로서 **BART**와 유사한 특징을 가지지만, 더 적은 학습 파라미터로 구성
- Encoder와 Decoder에서 Token Embedding 공유, Decoder의 Context 벡터 출력을 vocabulary logits으로 변환하는 최종 Linear Layer의 weight가 Token Embedding의 weight를 반영하도록 설정

Marian Tokenizer의 특징

- Marian Tokenizer는 SentencePiece 기반으로 bos(beginning of sentence) 토큰을 가지지 않으며 eos(end of sentence) 토큰과 pad 토큰을 가짐
- eos 토큰은 </s> 이며 값은 0, pad 토큰은 <pad>이며 한글-영어 Marian Tokenizer의 경우 <pad>값은 65000 임
- bos 토큰이 없지만, 모델 학습 시에는 tokenized 된 sentence의 맨 앞에 bos 토큰을 Manual 생성해서 학습데이터로 입력
- MarianMT Model은 bos 토큰으로 pad 토큰을 사용함.

"Welcome to Transformer Course."



MarianTokenizer(한글-영어)



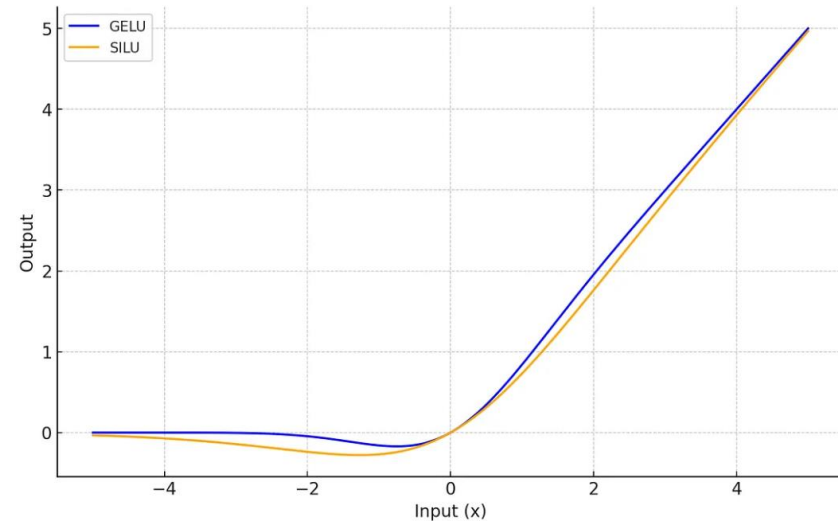
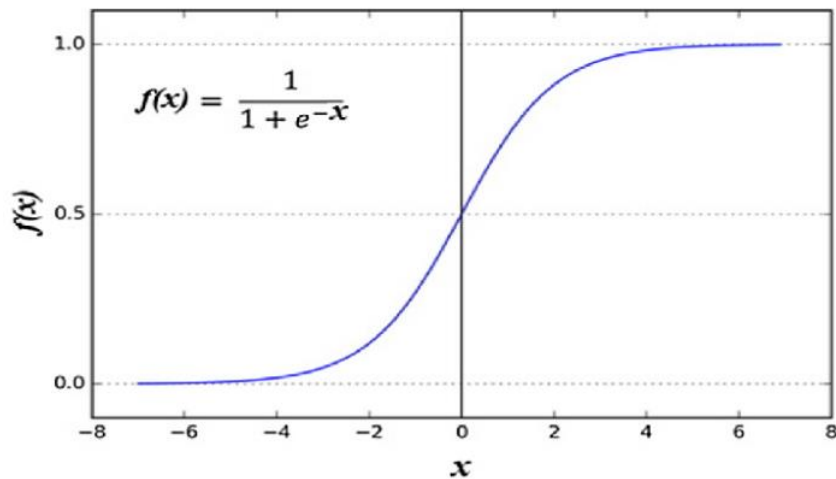
['_W', 'el', 'c', 'ome', '_to', '_T', 'ran', 's', 'f', 'or', 'mer', '_C', 'our', 'se', '.', '</s>', '<pad>', '<pad>', '<pad>', '<pad>']

[1278, 2242, 897, 10488, 5, 1220, 6221, 10, 1931, 992, 9437, 430, 5224, 4504, 2, 0, 65000, 65000, 65000, 65000]

SiLU 활성화 함수

- GELU와 유사하게 입력 값이 마이너스 ~ 0에서 플러스로 변할때 **급격히 미분 값이 변경되는** ReLU의 단점을 개선하기 위해 제시
- SiLU는 입력값에 Sigmoid 함수 출력값을 곱한 것임. **SiLU(x) = x · sigmoid(x)**
- GELU와 유사하게 SiLU는 입력이 +로 큰 값에서는 입력값과 유사하며, -로 큰 값에는 0에 가까움. 하지만 입력값이 0 근처에서 보다 Smooth한 값을 가짐.

Sigmoid



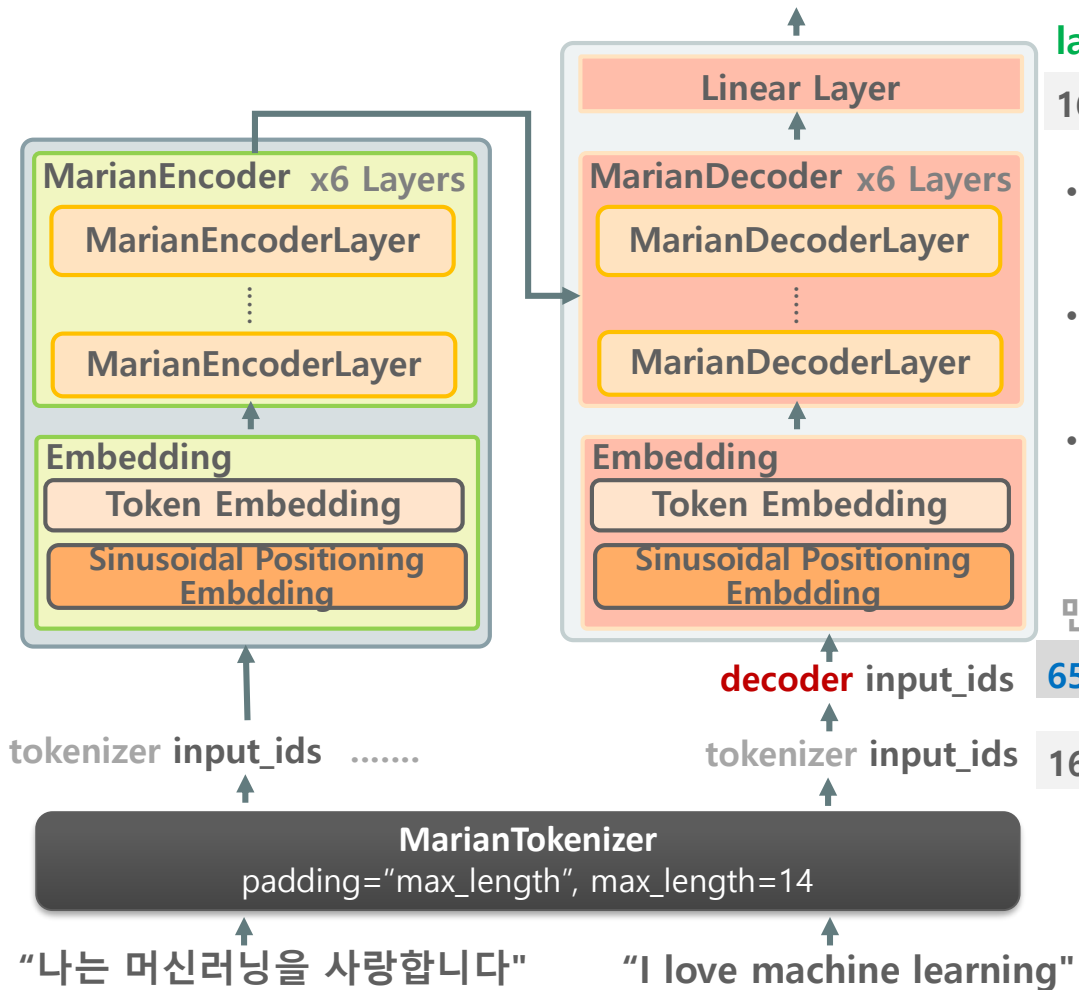
$$\text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

$$\text{GELU}(x) = x \cdot \Phi(x)$$

$$\text{SiLU}(x) = x \cdot \sigma(x).$$

Encoder-Decoder 모델 학습 시 Decoder 입력 토큰과 Labels

대부분의 NLP Encoder-Decoder(seq2seq) 모델은 **다음 토큰 예측**(Next Token Prediction)을 주요 기능으로 함



labels

16, 4874, 28848, 1495, 7543, 3645, 4874, 12470, 649, 130, 0, 65000, 65000, 65000

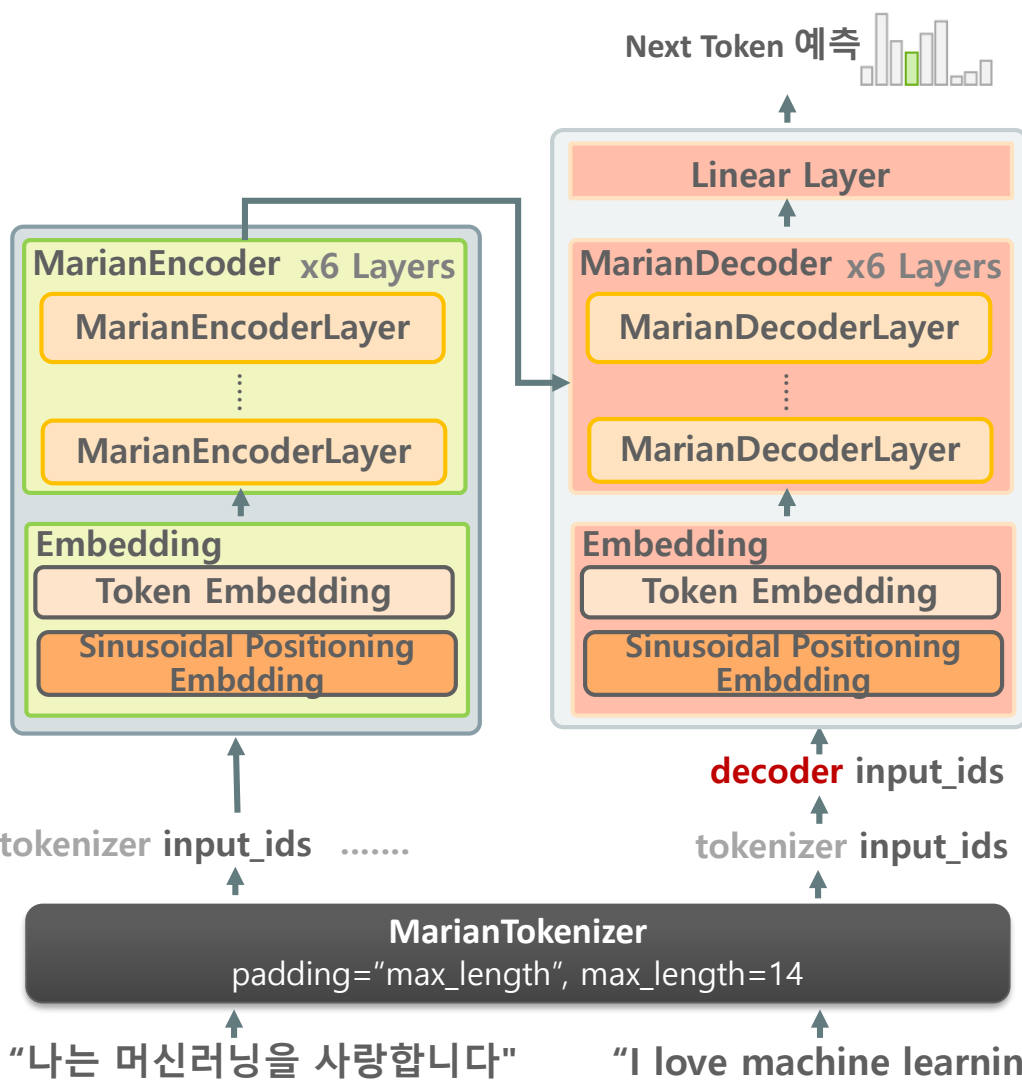
- bos를 적용하지 않는 tokenizer의 경우 tokenizer에서 생성하는 input_ids를 labels에 그대로 사용
- Decoder 모델(모듈)에 입력되는 decoder input_ids는 tokenizer의 bos 적용 여부와 관계없이 무조건 **첫번째 토큰이 bos 토큰이 와야함**(MarianMT의 경우 pad)
- Labels는 tokenizer의 bos 적용 여부와 관계없이 **첫번째 토큰이 bos가 아니라 실제 토큰이 와야함**.

맨 앞에 bos 삽입하고 labels값을 오른쪽으로 한칸 이동한 값(shift right)을 추가

65000, 16, 4874, 28848, 1495, 7543, 3645, 4874, 12470, 649, 130, 0, 65000, 65000

16, 4874, 28848, 1495, 7543, 3645, 4874, 12470, 649, 130, 0, 65000, 65000, 65000

Encoder-Decoder 모델 학습 시 Label Pad Token의 Loss 미 반영



train predicted

16, 3026, 17565, 1495, 7543, 3645, 2687, 1261, 649, 264, 128, 2621, 2121, 36121

labels

16, 4874, 28848, 1495, 7543, 3645, 4874, 12470, 649, 130, 0, -100, -100, -100

- labels의 **pad token**은 모델 학습(train) 시 Loss에 **반영되어서는 안됨**
- Pytorch의 Cross Entropy Loss는 이를 위해 ignore_index 인자를 제공하며 설정하지 않으면 자동으로 -100이 할당됨.
- Labels의 ignore_index에 해당하는 값은 **Loss 적용 시 반영되지 않음**

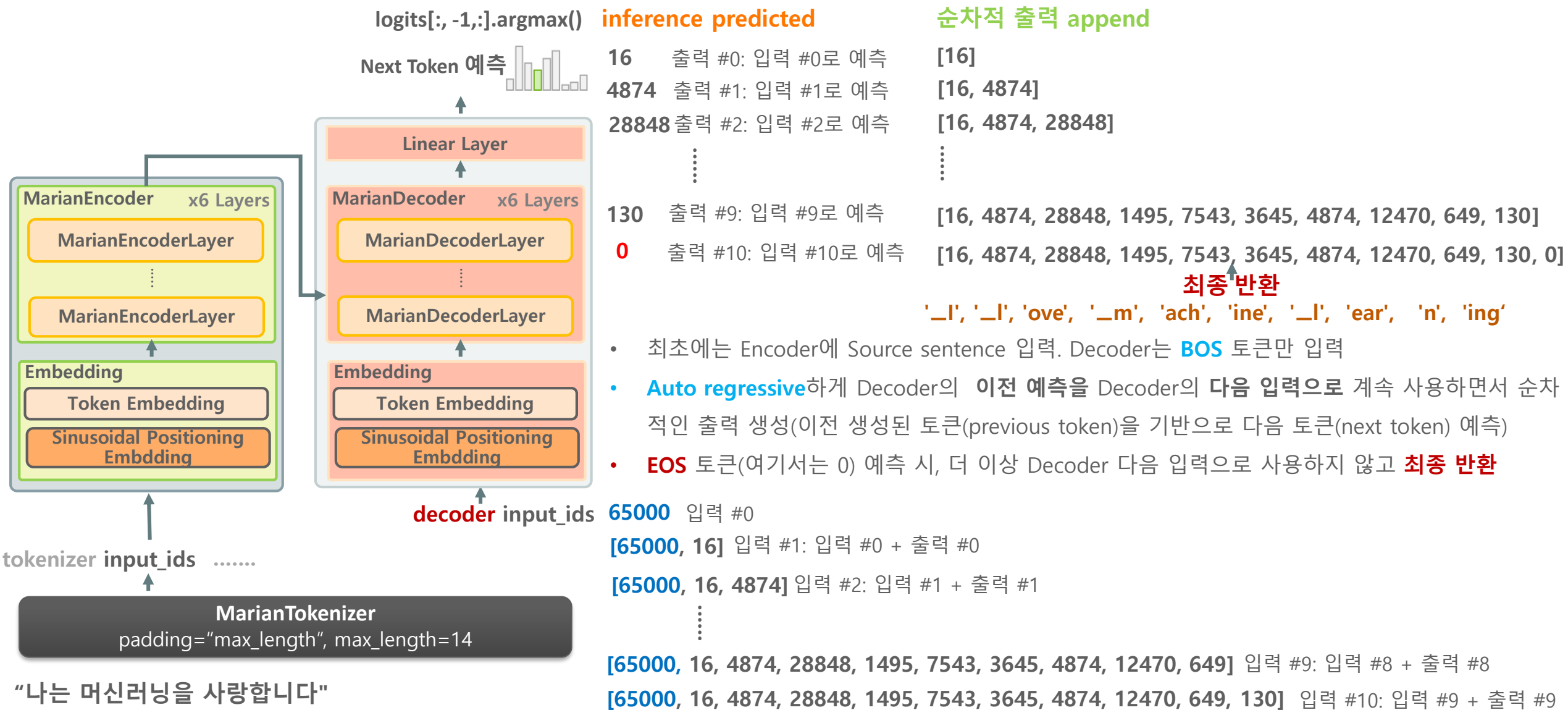
```
Loss_fn = nn.CrossEntropyLoss ( ignore_index = -100 )
```

맨 앞에 bos 삽입하고 labels값을 오른쪽으로 한칸 이동한 값(shift right) 을 추가

65000, 16, 4874, 28848, 1495, 7543, 3645, 4874, 12470, 649, 130, 0, 65000, 65000

16, 4874, 28848, 1495, 7543, 3645, 4874, 12470, 649, 130, 0, 65000, 65000, 65000

Auto regressive Inference 수행



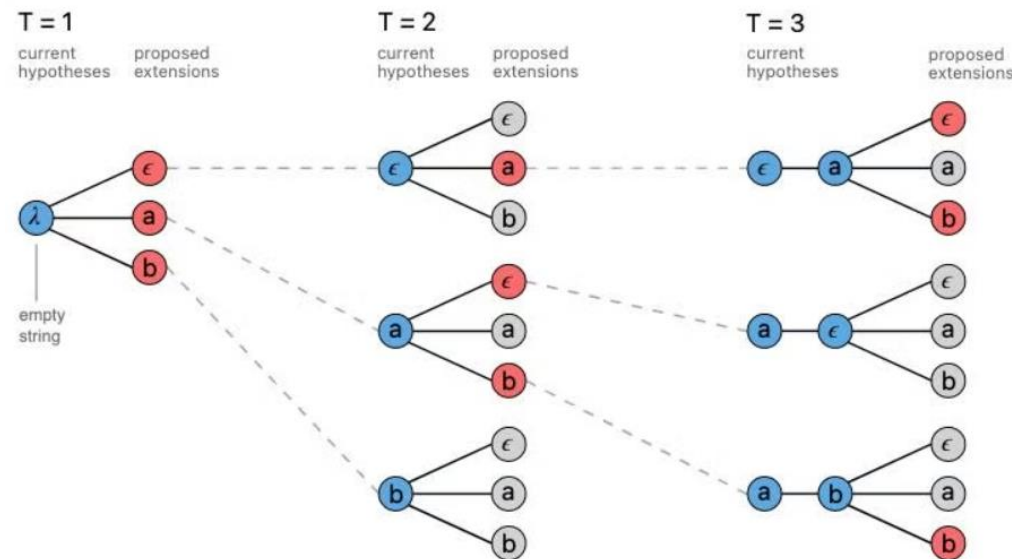
Decoding 전략

- Transformer 기반 언어모델은 하나의 토큰씩 **순차적**으로 다음 토큰을 예측하면서 문장을 완성. 이때 다음 토큰을 무조건 높은 확률을 가지는 토큰을 선택(greedy search)하면 문장에서는 **최적화된 문장이 생성되지 않은** 경우가 발생하기 쉬움.
- 보다 완성도 높은 문장을 생성하기 위해 일반적으로 아래와 같은 Decoding 전략이 있음

전략 유형	설명
Greedy Search	- 매 시점에서 가장 확률이 높은 토큰(단어) 선택. 가장 빠른 속도로 생성 - 한번 잘못된 토큰(단어)선택을 하면 이후 회복이 어려움(제대로 된 토큰을 선택하기 어려움) - 최적 문장을 만드는데 적합한 방식이 아니므로 일반적으로 선택하지 않음.
Beam Search	- 확률이 높은 다음 토큰 상위 N개의 후보를 선정하고, 이들을 기반으로 누적 확률이 높은 문장 경로를 탐색하면서 최적 문장 생성 - 속도는 느지만 보다 자연스러운 문장 품질 향상. 일반적으로 번역 모델에 적용
Top – k Sampling	상위 k개의 다음 토큰 후보군들 중에서 확률 샘플링
Top – p Sampling	누적 확률이 p 이하가 되도록 상위 단어만 남기고 확률 샘플링. 동적으로 후보수 조절

Beam Search 수행 개요

- **beam width(size)**는 auto regressive하게 다음 토큰을 생성할 때(decode) 다음 토큰의 **후보군으로** 설정하는 개수. 만약 beam width가 3이면 다음 토큰의 후보군으로 top-3 개의 후보 토큰을 선정하여 확장(expand)
- Decode step 시마다 **이전 step**에서 **선택된 beam**과 top N 후보 토큰을 결합하여 이들 중 **누적 확률이**(cumulative log probability) 가장 높은 **top beam width** 만큼의 후보들을 선택하여 **현 step의 beam**으로 등록(**beam**은 auto regressive하게 연속적으로 생성되며 이들 중 top beam width만큼을 filtering 하여 현 step의 beam으로 등록)
- EOS token이 생성되었는지 확인하고 EOS가 생성된 beam은 최종 후보군에 선착 시킴
- 모든 beam이 EOS를 선택 했거나 Decode step이 지정된 max_length에 도달할 때 까지 위를 반복
- **최종 step**에서 선택된 beam들중 가장 누적 확률이 **높은 한 개만** 최종 문장으로 반환



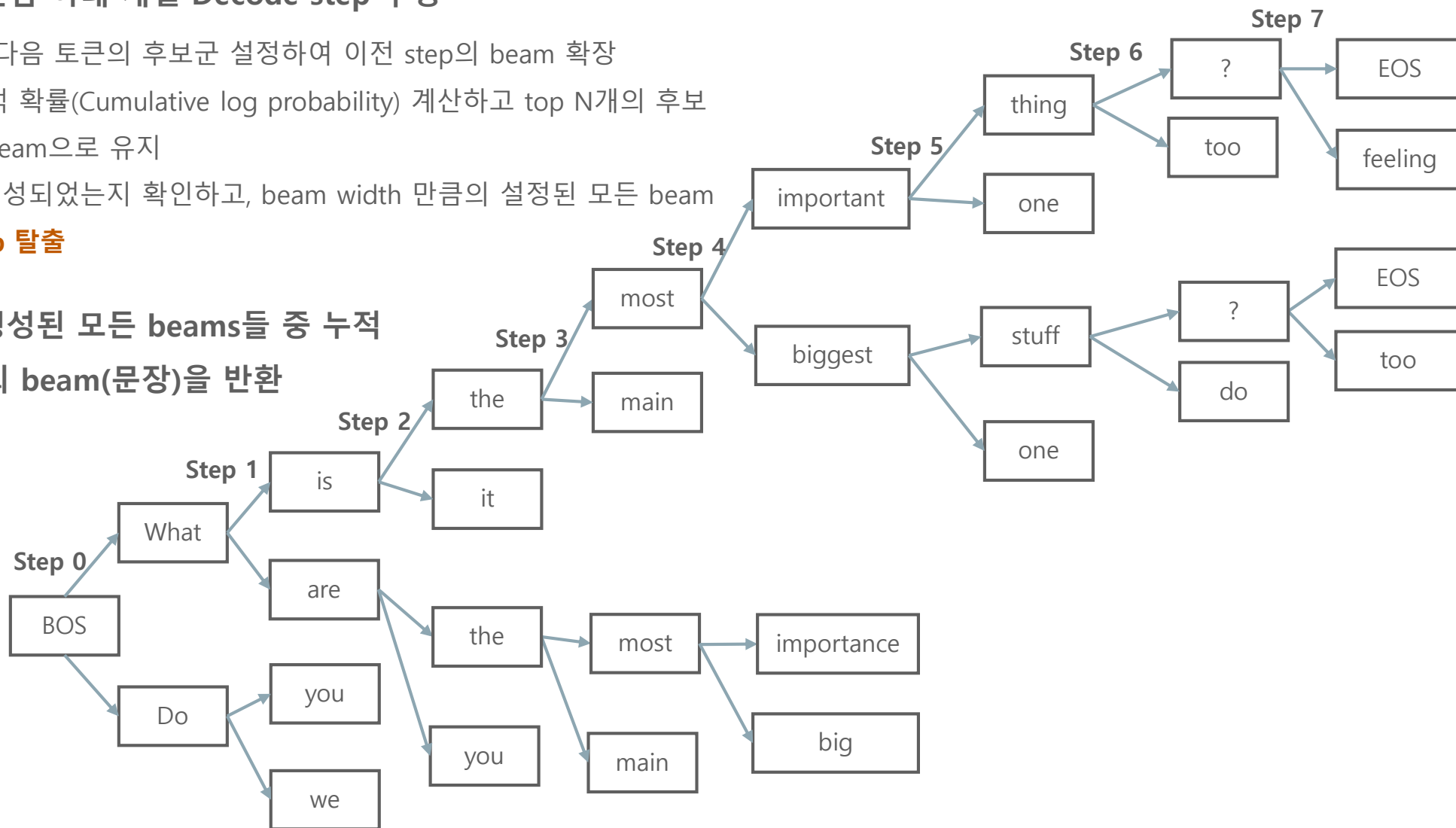
<https://pytorch.org/blog/fast-beam-search-decoding-in-pytorch-with-torchaudio-and-flashlight-text/>

Beam width=2 인 경우 Beam Search 수행 예시

1. 지정된 max_length 만큼 아래 개별 Decode step 수행

- 1.1 Beam width(size)만큼 다음 토큰의 후보군 설정하여 이전 step의 beam 확장
- 1.2 확장된 후보들에서 누적 확률(Cumulative log probability) 계산하고 top N개의 후보만 필터링하여 현 step의 beam으로 유지
- 1.3 Beam에 EOS 토큰이 생성되었는지 확인하고, beam width 만큼의 설정된 모든 beam들이 다 완료되었으면 **loop 탈출**

2. 가장 최종 step에서 생성된 모든 beams들 중 누적 확률이 가장 높은 한 개의 beam(문장)을 반환



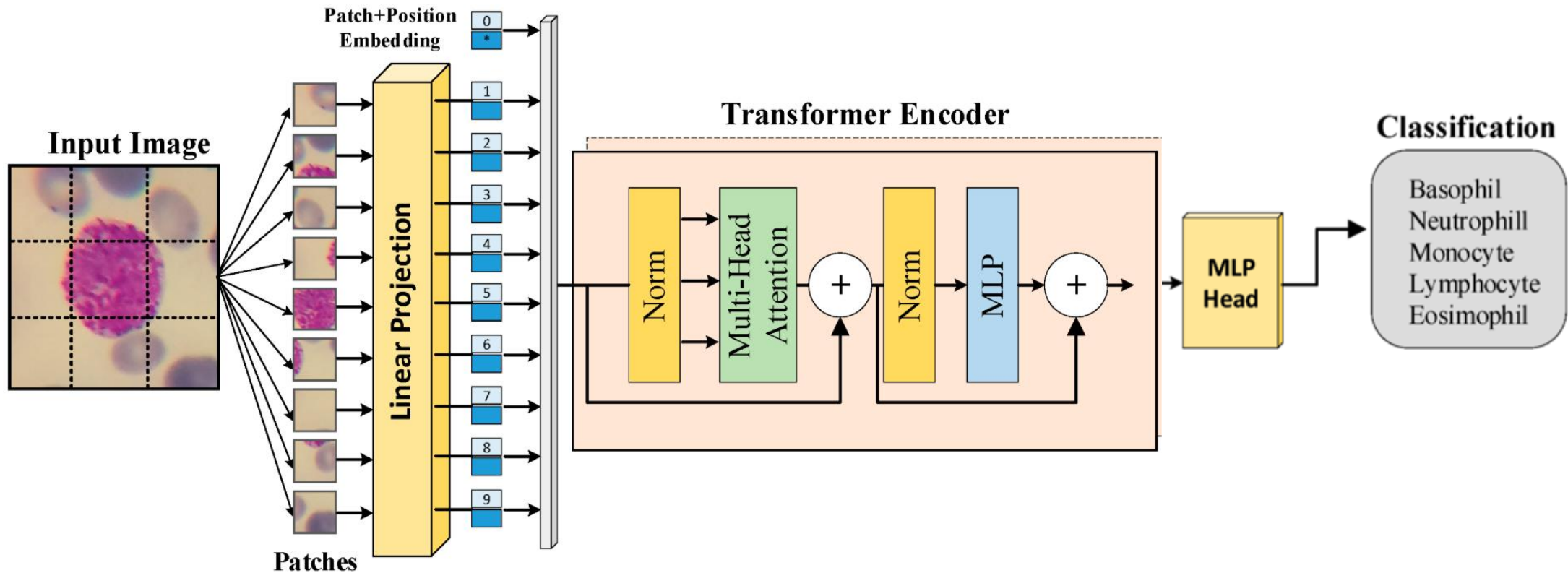
ViT(Vision Transformer) 개요

“An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”

- (**CNN을 사용하지 않고**) **Transformer 기반의 이미지 판별 모델**로 당시 최고 수준의 CNN 모델 성능을 (살짝 ^^) 뛰어 넘음
- Multi Head Self Attention만으로 CNN의 지역적 피쳐 추출(Local Feature Extraction)과 Receptive Field 확장 능력을 **전역적 문맥 학습**으로 대체
- 대규모 데이터 세트가 아닌 **작은 데이터 세트**에서는 CNN 보다 성능이 떨어짐(반드시 **대규모 데이터 세트**에서 Pretrained된 Weight 적용 필요)
- 로컬 패턴 인식이 약해 **작은 이미지**(또는 작은 객체 인식)에는 성능이 떨어지며, 고해상도 이미지에서는 **계산 비용**이 매우 커짐
- **안정적인 학습** 수행 어려움(Regularization, augmentation, 학습율 조정등의 다양한 기법 동원 필요)
- Torchvision이나 HuggingFace에서 대용량 데이터 세트로 이미 **학습된 Pretrained** 모델을 활용하여 fine-tuning 적용하는 방식으로 주로 활용
- 이미지와 텍스트를 같은 Transformer 구조안에서 **통합**할 수 있는 길을 열어서 이미지와 텍스트를 함께 다뤄야하는 **멀티모달** 영역에서 주도적인 역할 수행

ViT(Vision Transformer) 아키텍처 개요

- Transformer Encoder 모델의 입력을 문자열 토큰 임베딩에서 이미지 **패치 임베딩**으로 전환하여 이미지 판별 모델 구축
- 패치 임베딩(Patch Embedding)을 제외하고, 모델 아키텍처는 **Encoder 기반의 BERT**와 유사하며, Post Normalization이 아니라, **Pre Normalization**을 적용함(MultiHeadAttention 이전 Layer Norm, Feed Forward 이전 Layer Norm 적용)



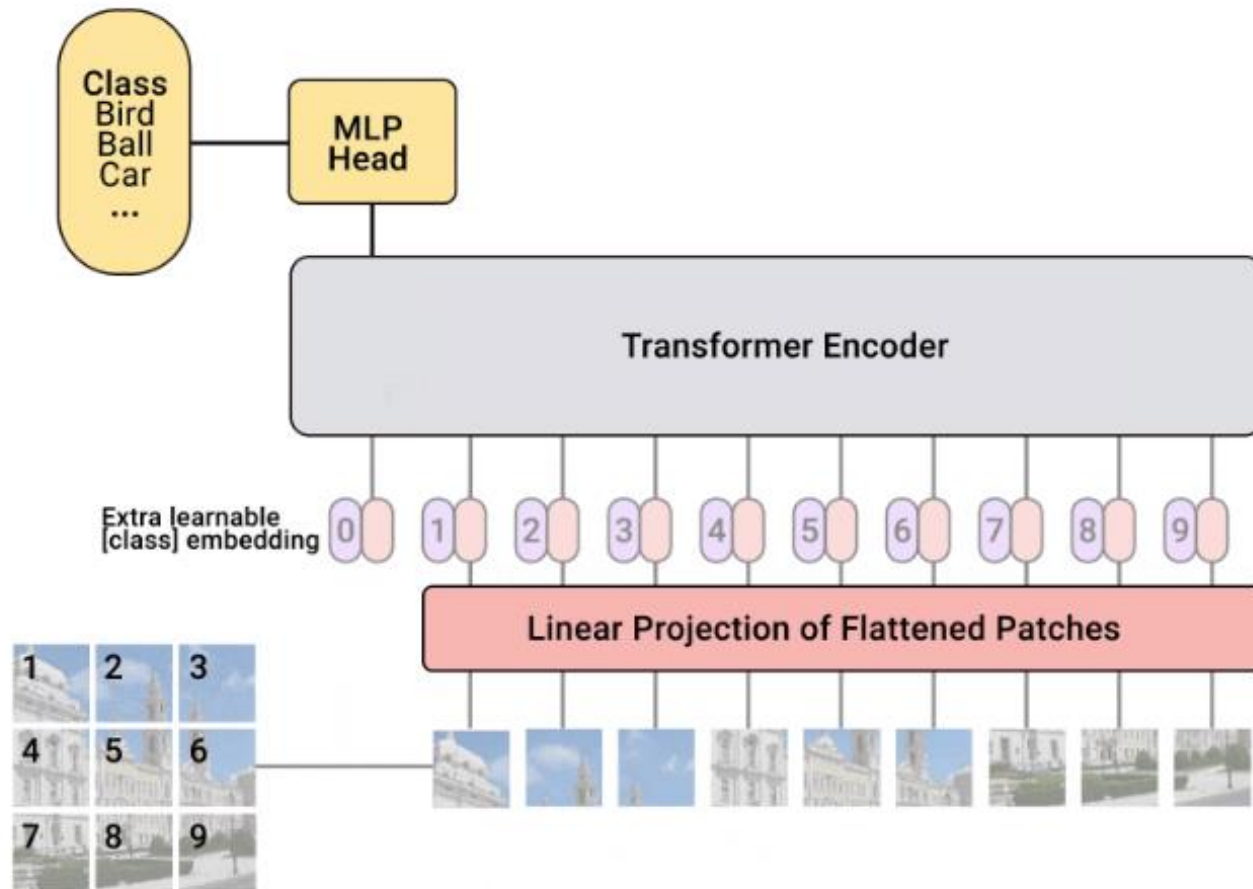
ViT(Vision Transformer) 아키텍처 개요

- Transformer Encoder 모델의 입력을 문자열 토큰 임베딩에서 이미지 **패치 임베딩**으로 전환하여 이미지 판별 모델 구축
- 패치 임베딩(Patch Embedding)을 제외하고, 모델 아키텍처는 **Encoder 기반의 BERT**와 유사하며, Post Normalization이 아니라, **Pre Normalization**을 적용함(MultiHeadAttention 이전 Layer Norm, Feed Forward 이전 Layer Norm 적용)



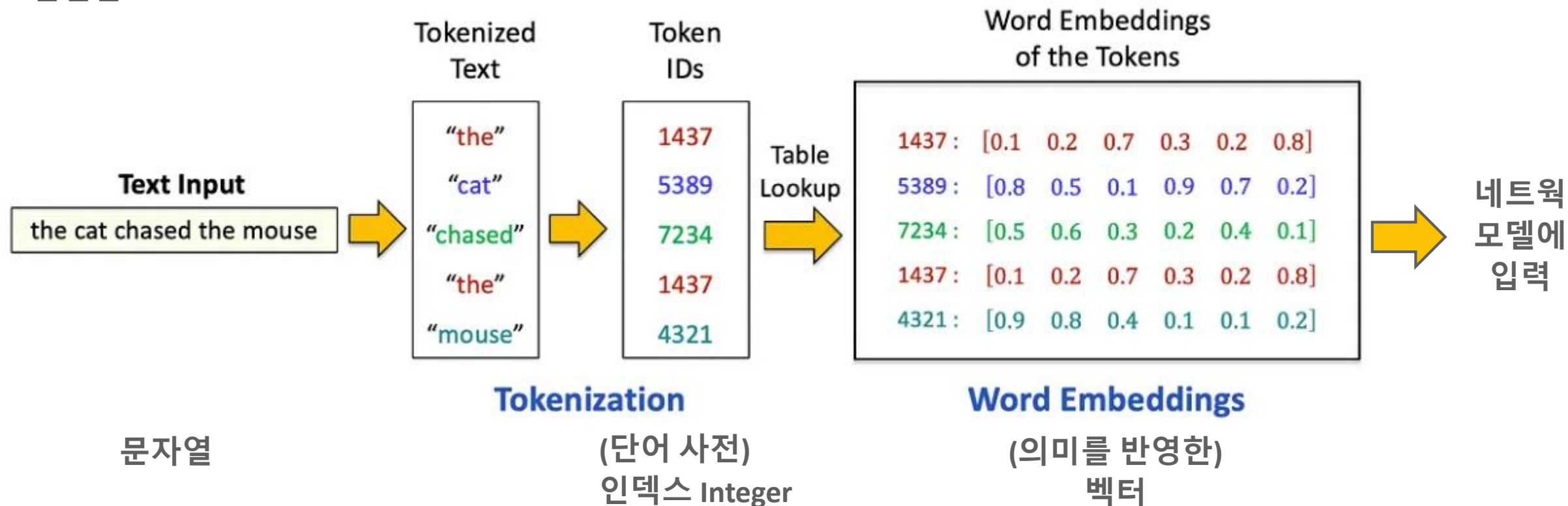
ViT(Vision Transformer) 아키텍처 개요

- Transformer Encoder 모델의 입력을 문자열 토큰 임베딩에서 이미지 **패치 임베딩**으로 전환하여 이미지 판별 모델 구축
- 패치 임베딩(Patch Embedding)을 제외하고, 모델 아키텍처는 **Encoder 기반의 BERT**와 유사하며, Post Normalization이 아니라, **Pre Normalization**을 적용함(MultiHeadAttention 이전 Layer Norm, Feed Forward 이전 Layer Norm 적용)



Tokenization(토큰화)와 Embedding 개요

- Tokenization(토큰화)는 텍스트를 **토큰** 단위로 분할 하고, 개별 토큰을 **index 값으로 매핑하는 어휘 사전** 구축 작업
- Embedding은 Tokenizer를 통해 만들어진 토큰값을 단어 의미를 가지는 다차원의 embedding vector로 생성
- Tokenizer의 **token index**를 기반으로 Embedding은 embedding vector를 반환하므로 Tokenizer와 Embedding은 Loosely 연결됨.



이미지의 Patch Embedding

문장들

"Welcome to Transformer Course"
"This is for Vision Transformer"

Tokenizer 적용

[
[115, 203,..., 31],
[223, 225,..., 51]
]

Embedding

0: [0.1 0.2 0.03]
.....
223: [0.9, 0.03, 0.1]
.....
60000: [0.1 0.0 0.2 ...]

Embedding 적용 후

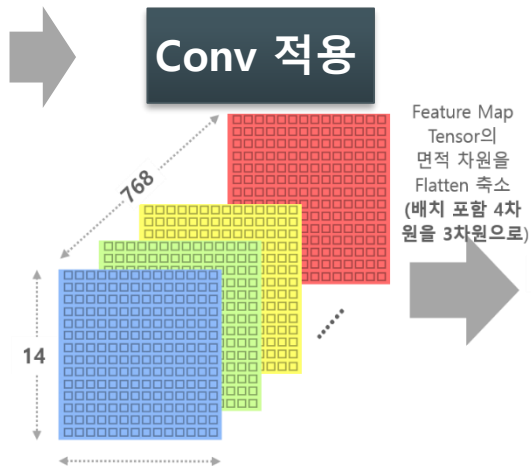
(b_size, seq_len, emb_dim)

이미지들



Patch Embedding

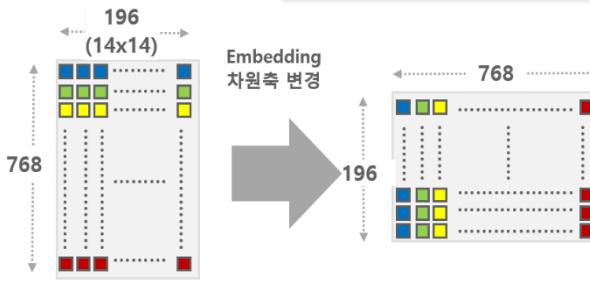
Conv 적용



Feature Map
Tensor의
면적 차원을
Flatten 축소
(배치 포함 4차
원을 3차원으로)

Flatten & Reshape

Embedding
차원축 변경



(b_size, num_patches, emb_dim)

이미지 패칭(Image Patching)

- 텍스트에서 문장을 "**단어 토큰**"으로 나누듯, 이미지를 작은 **정사각형 패치 토큰**들로 분할하는 과정
- 이렇게 분할 된 이미지 패치들은(예를 들어 16x16 크기의 패치) 추후에 ViT가 Transformer에 이미지를 넣기 위해서 각각의 이미지 패치들을 토큰 벡터로 변환 작업 수행.

Sentence to word tokens:

"hi, I am a short sentence"



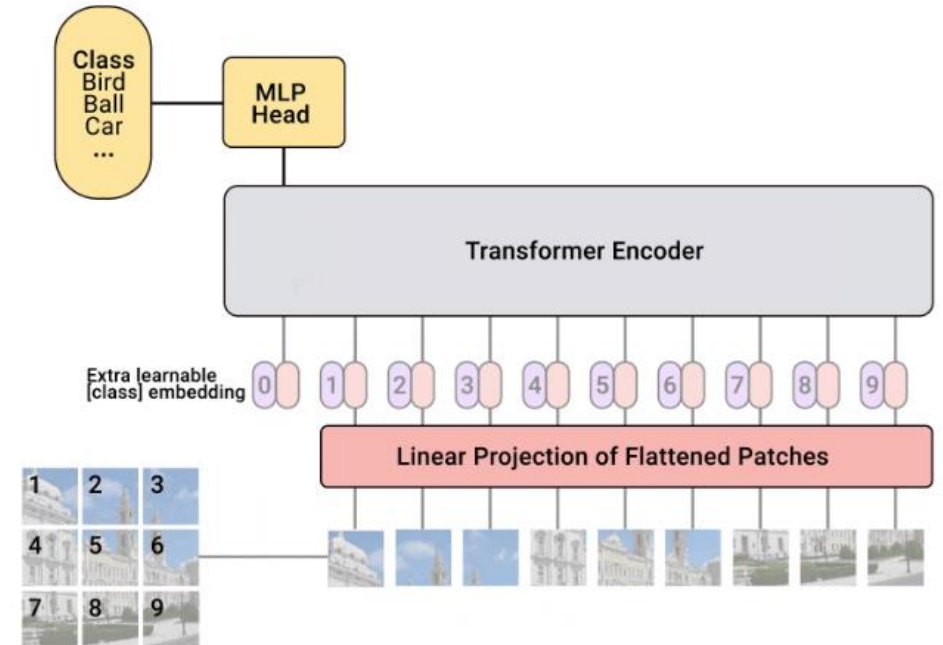
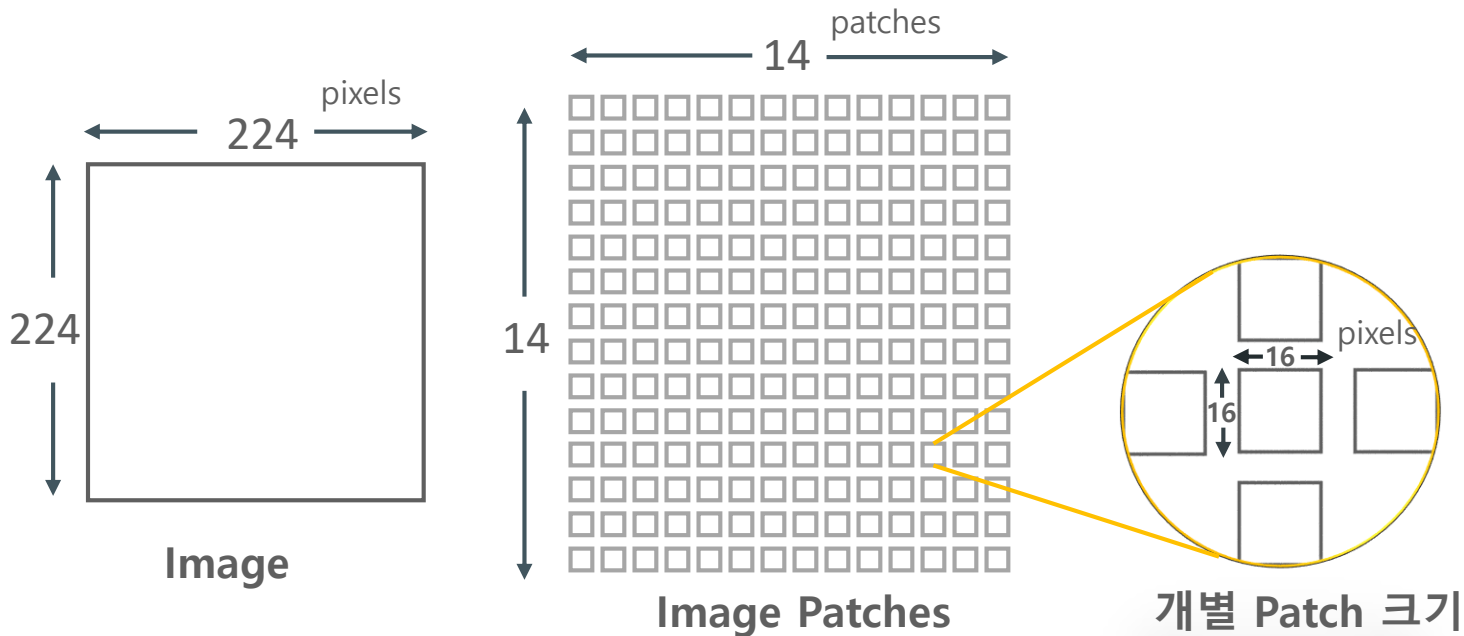
'hi' ',' 'I' 'am' 'a' 'short' 'sentence'

Image to image patches:



이미지 패칭(Image Patching)

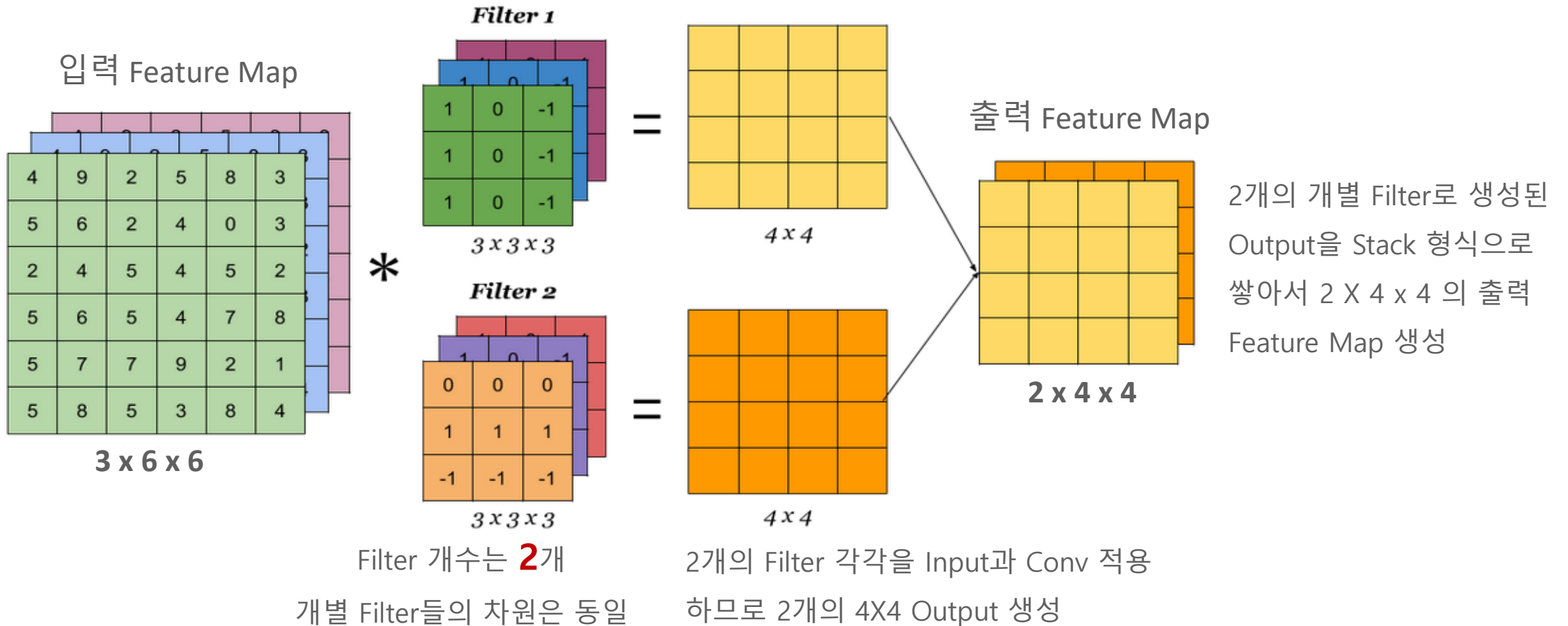
- 텍스트에서 문장을 "**단어 토큰**"으로 나누듯, 이미지를 작은 **정사각형 패치 토큰**들로 분할하는 과정
- 이렇게 분할 된 이미지 패치들은(예를 들어 16x16 크기의 패치) 추후에 ViT가 Transformer에 이미지를 넣기 위해서 각각의 이미지 패치들을 토큰 벡터로 변환 작업 수행.



224x224 Pixel 이미지를 16x16 pixel 크기를 가지는, 가로 14개, 세로 14개(총 196개)의 개별 이미지 패치들로 분할

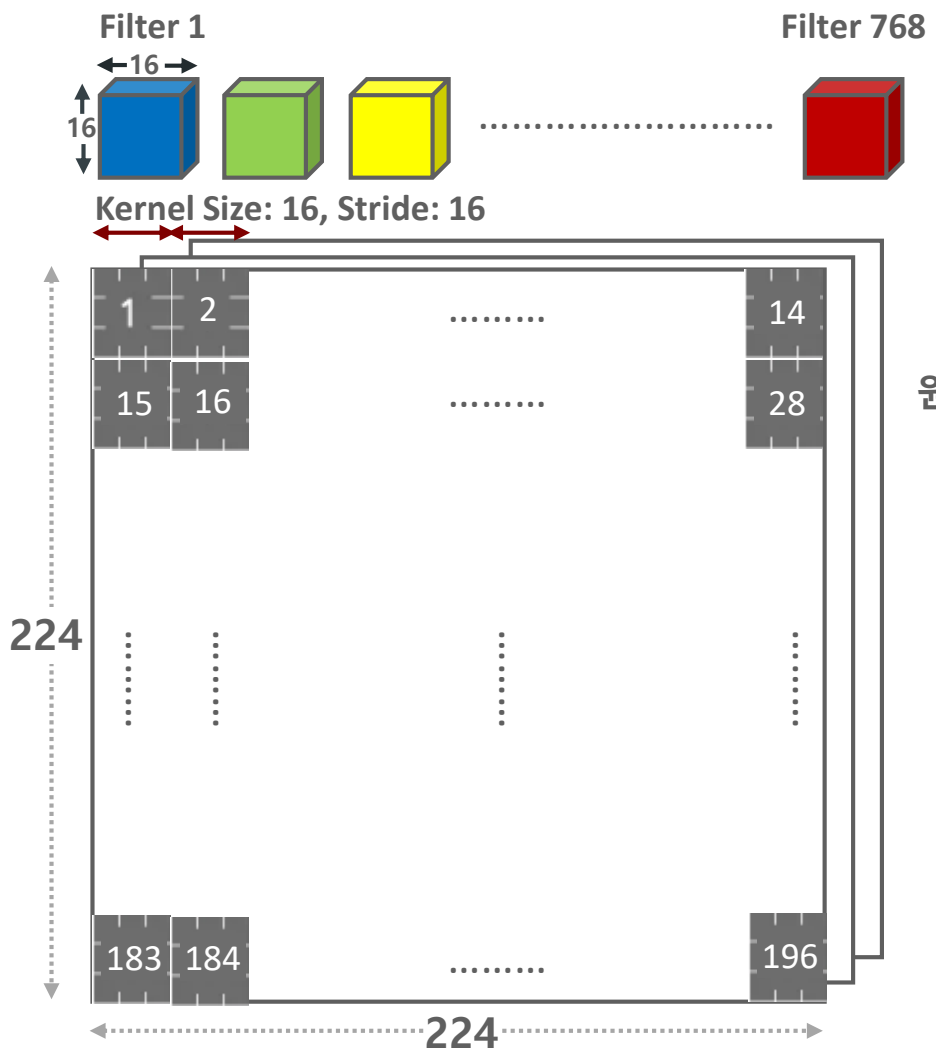
다채널 Input에 여러 개 filter들을 적용하기

출력 Feature Map의 **채널 수**는 Conv를 적용한 Filter의 **개수**로 결정됨 (단일 Filter의 채널 수가 아님)



Patch Embedding – Conv 적용

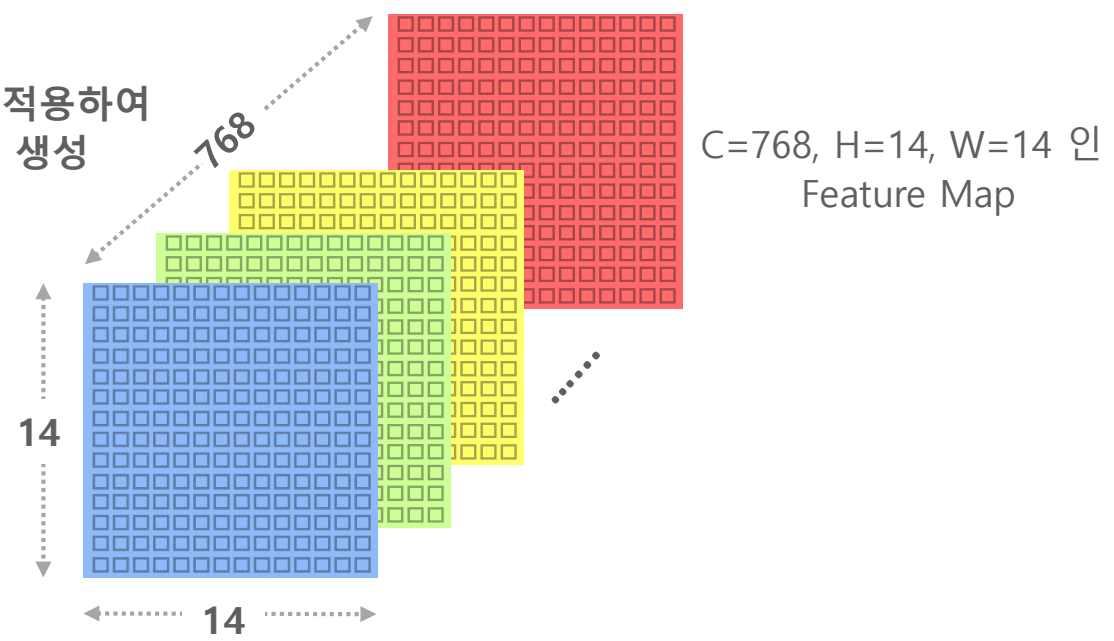
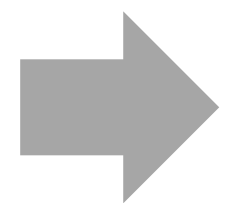
Embedding 차원수와 동일한 Conv Filter갯수와
Patch Size와 동일한 Kernel Size 적용



Patch size=16, num_patches = 14 * 14, Embedding dim=768

Conv2d(in_channels=3, out_channels=embedding_dim,
kernel_size=16, stride=16, padding=0)

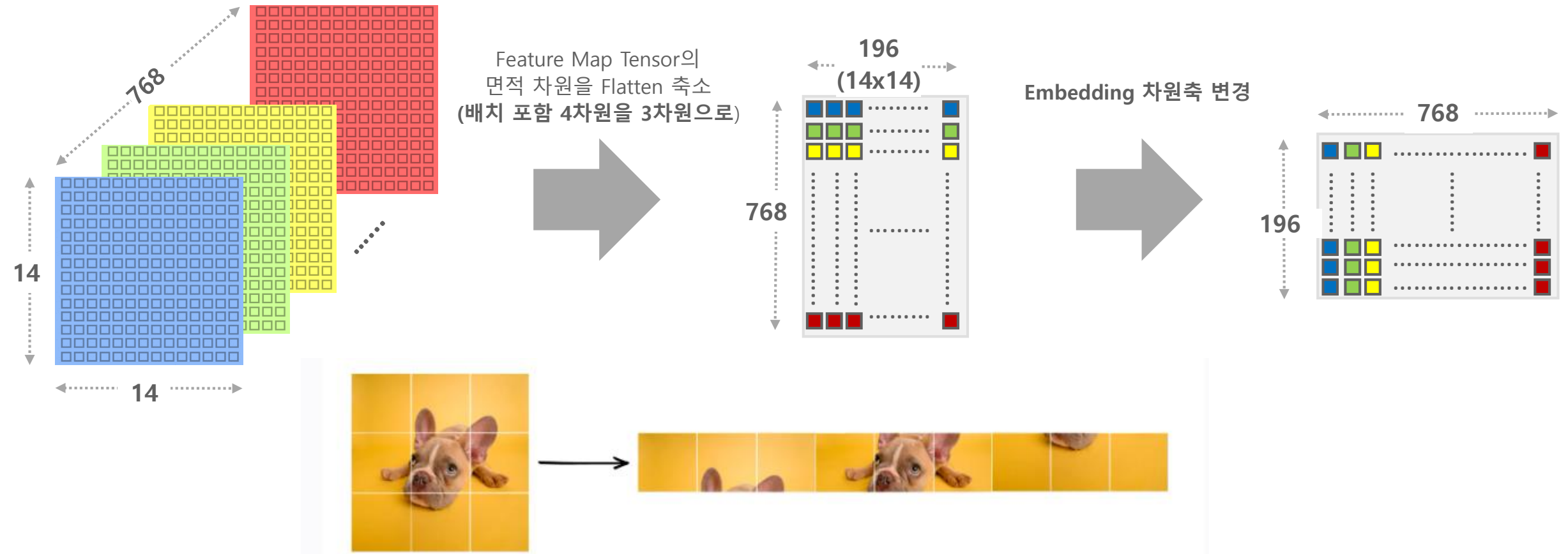
원본 이미지에 Conv 적용하여
Patch Embedding 생성



Patch Embedding – Flatten & Reshape 적용

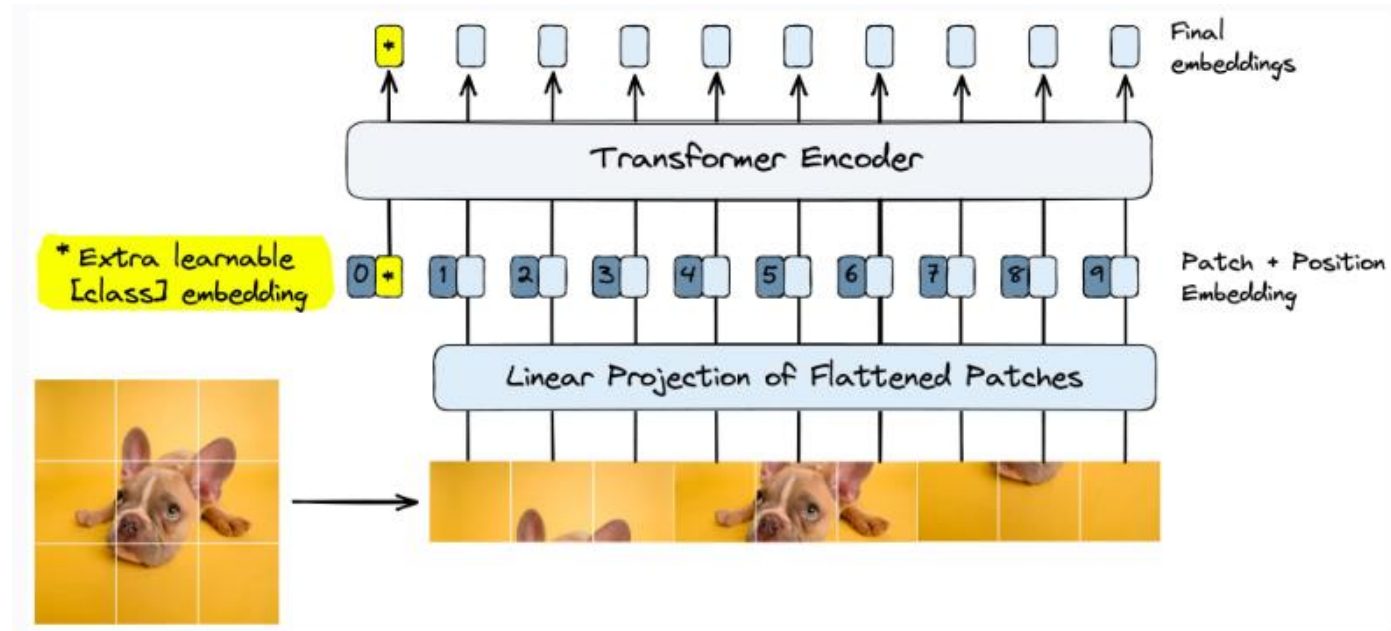
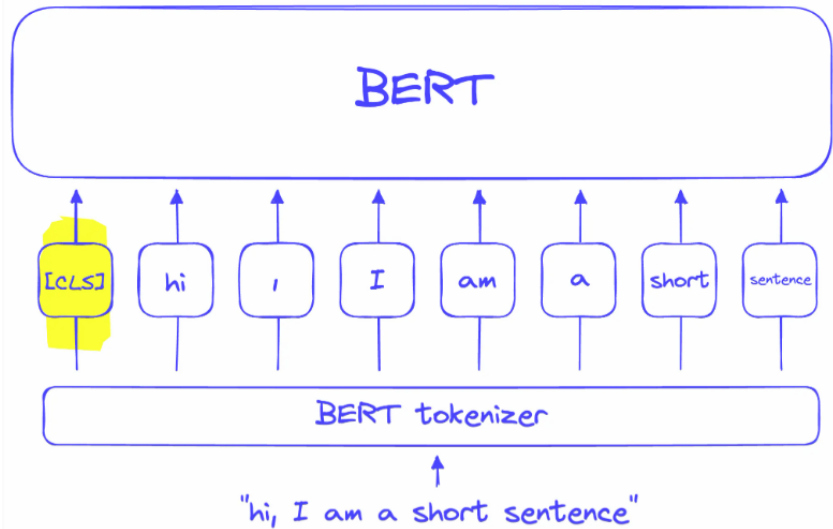
C=768, H=14, W=14 인 (B, 768, 14, 14) 크기의 Feature Map을 (B, 196[num_patches], 768[Emb_dim]) 형태로 변환하여 ViT에 입력

- ① 먼저 H, W에 해당하는 **면적 차원을 Flatten하여 축소**. 즉 (B, 768[Emb_dim], 196[H*W])으로 Reshape(배치 포함 4차원을 3차원으로)
- ② (B, 768[Emb_dim], 196[H*W]) 텐서를 (B, 196[num_patches], 768[Emb_dim]) Shape로 **차원 축 변경**

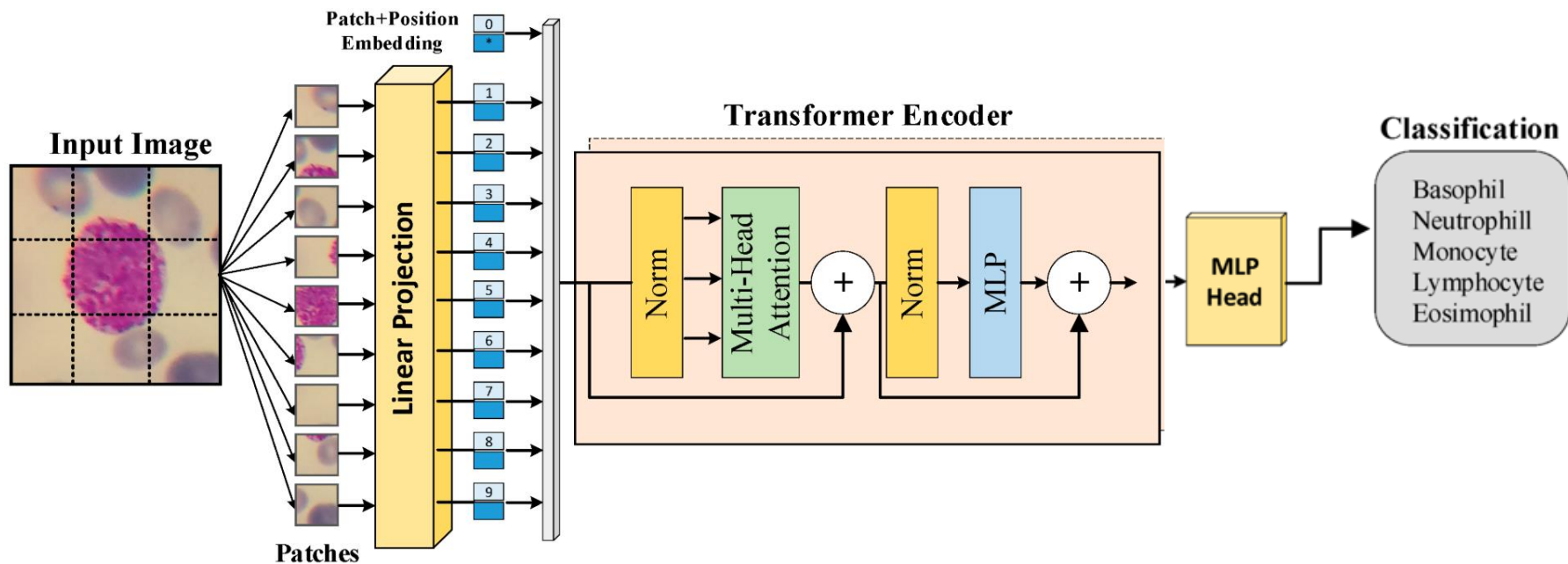


Class Embedding 추가

- 이미지 판별(Classification)을 위한 추가적인 학습 파라미터 적용 필요.
- BERT 모델의 CLS Token과 유사하게 image patching embedding 된 결과에 별도의 Class Embedding을 추가
- Class Embedding은 Transformer Encoder의 출력 후 Linear Layer 적용하여 최종 Classification Layer로 사용



ViT 모델 유형

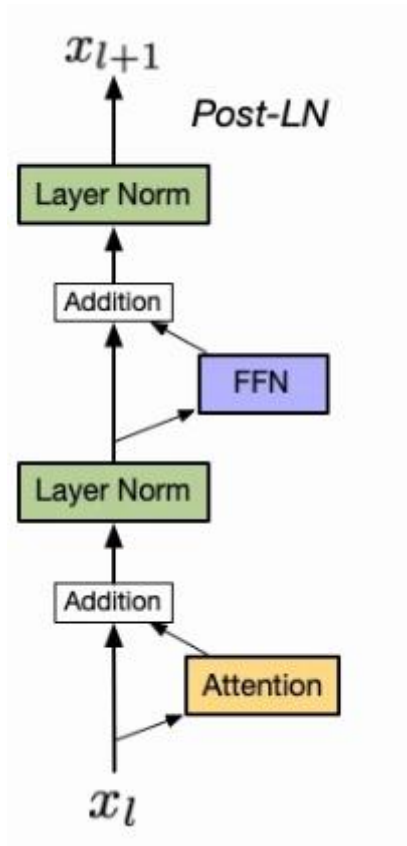


Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

Post-Norm vs Pre-Norm

Post-Norm(Normalization)

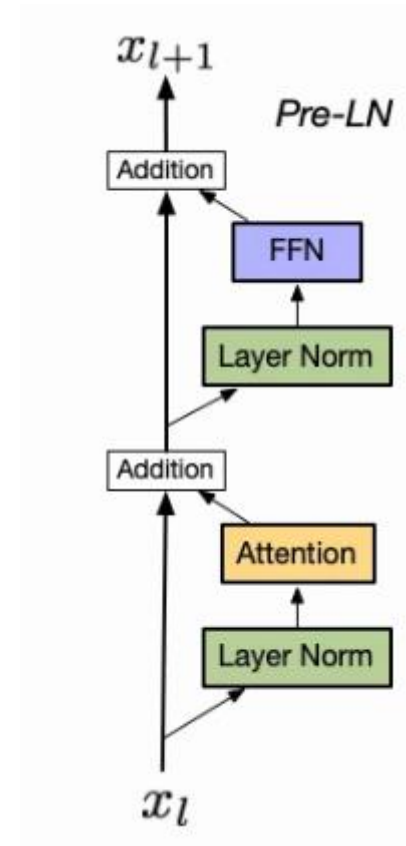


Add & Norm

Attention/Feed Forward **이후에**
residual connection 및 Layer
Norm 적용

Original Transformer, BERT

Pre-Norm(Normalization)



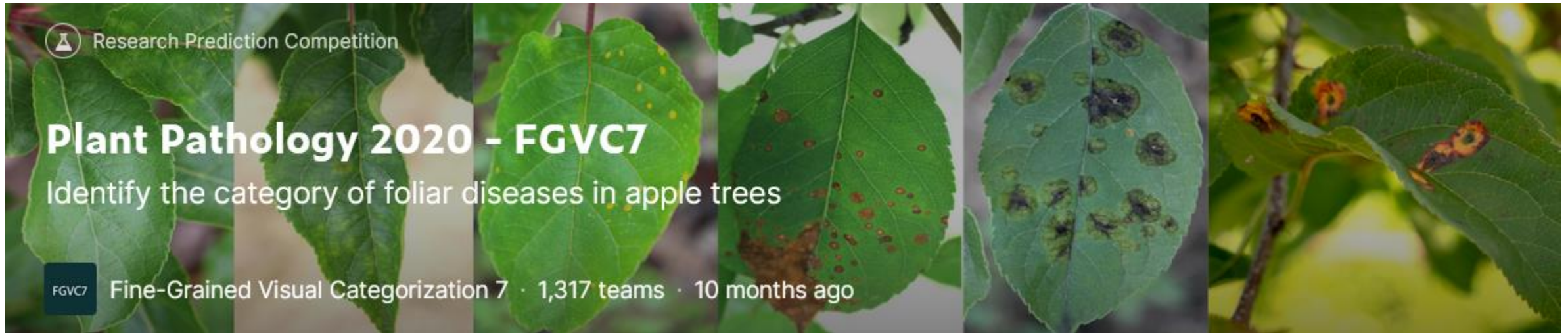
Norm & Add

Attention/Feed Forward **이전에**
Layer Norm을 적용 한 후
residual connection

ViT, GPT 등 LLM

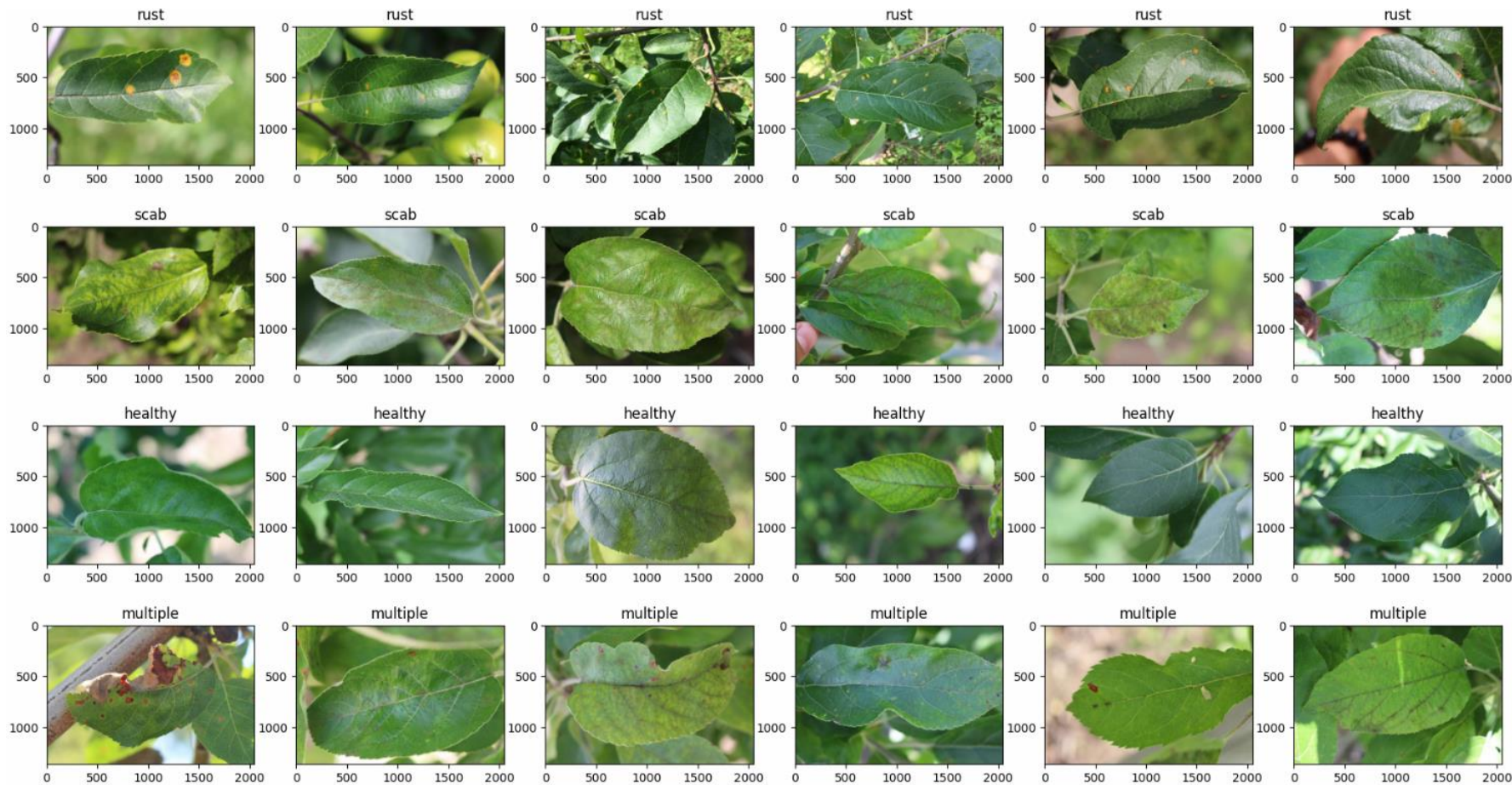
Plant Pathology - 2020

사과 나무잎 이미지에 따른 병의 유형을 판별하는 캐글 경연대회



Plant Pathology 2020 데이터 세트 개요

- 앞 이미지를 기반으로 녹병 (Rust), 딱지병/곰팡이병(scab), 복합질병(multiple_diseases), 건강(healthy) 등 4개의 유형을 판별
- 개별 이미지 사이즈는 (1365, 2048), 학습 이미지의 건수는 1821건
- 원래 성능 평가 지표는 ROC-AUC이지만, 여기서는 Accuracy(정확도)로 성능 평가



감사합니다